

TUGAS AKHIR - EF234801

Deteksi Anomali pada Data *Time Series* Konsentrasi PM2.5 dari Stasiun Pemantau Kualitas Udara Kementerian Lingkungan Hidup menggunakan REDLAMP

Alendra Rafif Athaillah

NRP 5025221297

Dosen Pembimbing

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Dosen Ko-pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Program Studi S-1 Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan.



TUGAS AKHIR - EF234801

Deteksi Anomali pada Data Time Series Konsentrasi PM2.5 dari Stasiun Pemantau Kualitas Udara Kementerian Lingkungan Hidup menggunakan REDLAMP

Alendra Rafif Athaillah

NRP 5025221297

Dosen Pembimbing

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Dosen Ko-pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Program Studi S-1 Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan.



FINAL PROJECT - EF234801

Anomaly Detection on Time-Series Data of PM2.5 Concentration from The Ministry of Environment's Air Quality Monitoring Station Using REDLAMP

Alendra Rafif Athaillah

NRP 5025221297

Advisor

Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.

NIP 197512202001122002

Co-advisor

Ilham Gurat Adillion, S.Kom., M.Eng.

NIP 1995202411009

Undergraduate Study Program of Informatics

Department of Informatics

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Halaman ini sengaja dikosongkan.

LEMBAR PENGESAHAN

**Deteksi Anomali pada Data Time Series Konsentrasi PM2.5 dari Stasiun Pemantau
Kualitas Udara Kementerian Lingkungan Hidup menggunakan REDLAMP**

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Komputer pada
Program Studi S-1 Teknik Informatika
Departemen Teknik Informatika
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh: Alendra Rafif Athaillah

NRP. 5025221297

Disetujui oleh Tim Penguji Tugas Akhir:

- | | |
|---|---------------|
| 1. Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. | Pembimbing |
| 2. Ilham Gurat Adillion, S.Kom., M.Eng. | Ko-pembimbing |
| 3. Imam Mustafa Kamal, S.ST., Ph.D. | Penguji 1 |
| 4. Dini Adni Navastara, S.Kom., M.Sc. | Penguji 2 |

SURABAYA

Juli, 2026

Halaman ini sengaja dikosongkan.

APPROVAL SHEET

Anomaly Detection on Time-Series Data of PM2.5 Concentration from The Ministry of Environment's Air Quality Monitoring Station Using REDLAMP

FINAL PROJECT

Submitted to fulfill one of the requirements
for obtaining a Bachelor of Computer Science degree at
Undergraduate Study Program of Informatics
Department of Informatics
Faculty of Intelligent Electrical and Informatics Technology
Institut Teknologi Sepuluh Nopember

By: Alendra Rafif Athaillah

NRP. 5025221297

Approved by Final Project Examiner Team:

- | | |
|---|------------|
| 1. Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. | Advisor |
| 2. Ilham Gurat Adillion, S.Kom., M.Eng. | Co-advisor |
| 3. Imam Mustafa Kamal, S.ST., Ph.D. | Examiner 1 |
| 4. Dini Adni Navastara, S.Kom., M.Sc. | Examiner 2 |

SURABAYA

July, 2026

This page is intentionally left blank.

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP : Alendra Rafif Athaillah / 5025221297
Program Studi : S-1 Teknik Informatika
Dosen Pembimbing / NIP : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. /
197512202001122002
Dosen Ko-pembimbing / NIP : Ilham Gurat Adillion, S.Kom., M.Eng. / 1995202411009

dengan ini menyatakan bahwa Tugas Akhir dengan judul “Deteksi Anomali pada Data *Time Series* Konsentrasi PM2.5 dari Stasiun Pemantau Kualitas Udara Kementerian Lingkungan Hidup menggunakan REDLAMP” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Mengetahui
Dosen Pembimbing

Surabaya, 17 Juni 2026
Mahasiswa

Prof. Dr. Eng. Chastine Fatichah, S.Kom.,
M.Kom.
NIP. 197512202001122002

Alendra Rafif Athaillah
NRP. 5025221297

Dosen Ko-pembimbing

Ilham Gurat Adillion, S.Kom., M.Eng.
NIP. 1995202411009

Halaman ini sengaja dikosongkan.

STATEMENT OF ORIGINALITY

The undersigned:

Student Name / Student ID : Alendra Rafif Athaillah / 5025221297
Study Program : Bachelor of Informatics
Advisor / Employee ID : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. /
197512202001122002
Co-advisor / Employee ID : Ilham Gurat Adillion, S.Kom., M.Eng. / 1995202411009

hereby declares that the Final Project entitled “Anomaly Detection on Time-Series Data of PM2.5 Concentration from The Ministry of Environment's Air Quality Monitoring Station Using REDLAMP” is my own work, is original, and was written in accordance with the rules of scientific writing.

If any discrepancies with this statement are found in the future, I am willing to accept sanctions in accordance with the provisions of Institut Teknologi Sepuluh Nopember.

Acknowledged
Advisor

Surabaya, 17th of June 2026
Student

Prof. Dr. Eng. Chastine Fatichah, S.Kom.,
M.Kom.
NIP. 197512202001122002

Alendra Rafif Athaillah
NRP. 5025221297

Co-advisor

Ilham Gurat Adillion, S.Kom., M.Eng.
NIP. 1995202411009

This page is intentionally left blank.

PERNYATAAN KODE ETIK
PENGUNAAN AI GENERATIF
Code of Conduct Statement: Generative AI or AI-Assisted Usage

Saya yang bertanda tangan di bawah ini:
I, the undersigned:

Nama Mahasiswa / NRP : Alendra Rafif Athaillah / 5025221297
Full Name / Student ID

Program Studi : S-1 Teknik Informatika
Study Program

Judul Tugas Akhir : Deteksi Anomali pada Data Time Series Konsentrasi
Final Project Title
PM2.5 dari Stasiun Pemantau Kualitas Udara
Kementerian Lingkungan Hidup menggunakan
REDLAMP

dengan ini menyatakan bahwa pada Tugas Akhir dengan judul di atas tersebut:
hereby declare that in the Final Project with the above title:

No.	Pernyataan <i>Statement</i>	(✓)
1	Saya hanya menggunakan AI generatif sebagai alat bantu untuk memperbaiki tata bahasa. AI generatif tidak digunakan untuk membuat isi Tugas Akhir. <i>I only used generative AI as a tool to improve the readability or language of the text in my Final Project. It was not used to generate a complete text of my work.</i>	✓
2	Saya telah memeriksa dan/atau memperbaiki seluruh bagian dari Tugas Akhir saya yang dibantu oleh AI generatif agar sesuai dengan baku mutu penulisan karya ilmiah. <i>I have reviewed and refined all aspects of my work that generative AI assists with, ensuring it adheres to the standards of academic writing.</i>	✓
3	Saya tidak menggunakan AI generatif untuk pembuatan data primer, grafik dan/atau tabel pada Tugas Akhir saya. <i>I did not use generative AI to generate primary data, figures, and/or tables in my work.</i>	✓
4	Saya telah memberikan atribusi/pengakuan terhadap alat AI yang digunakan, secara rinci pada suatu bagian pada lampiran. <i>I have acknowledged the use of generative AI in any part of the work in the specific appendix page.</i>	✓
5	Saya memastikan tidak ada plagiarisme, termasuk hal yang berasal dari penggunaan AI generatif. <i>I have ensured that there is no plagiarism issue in the work, including any parts generated by AI.</i>	✓

Surabaya, 17 Juni 2026

Mahasiswa

Alendra Rafif Athaillah
NRP. 5025221297

Halaman ini sengaja dikosongkan.

ABSTRAK

Deteksi Anomali pada Data Time Series Konsentrasi PM2.5 dari Stasiun Pemantau Kualitas Udara Kementerian Lingkungan Hidup menggunakan REDLAMP

Nama Mahasiswa / NRP : Alendra Rafif Athaillah/ 5025221297
Departemen : Teknik Informatika FTEIC - ITS
Dosen Pembimbing : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.
Dosen Ko-pembimbing : Ilham Gurat Adillion, S.Kom., M.Eng.

Abstrak

Pada data PM2.5 dari SPKU, deteksi anomali *time-series* diperlukan untuk menandai titik atau subsekuens yang menyimpang dari pola normal sebelum data digunakan dalam analisis lebih lanjut, dan karena data yang digunakan belum tentu menyediakan label anomali, diperlukan pendekatan *unsupervised* yang mampu mendeteksi penyimpangan berdasarkan pola yang dipelajari dari data. Pendekatan *unsupervised* yang telah dikembangkan masih memiliki keterbatasan dalam menangani kontaminasi anomali, *diversity gap*, *false anomalies*, dan *overconfidence* pada label. Untuk mengatasi permasalahan tersebut, penelitian ini menerapkan REDLAMP (*Robust and Explainable Detector of Time-Series Anomaly*), yaitu metode *unsupervised* yang menggabungkan *reconstruction* dan *classification* dengan pendekatan *multiclass*, untuk mendeteksi anomali pada data *time-series* konsentrasi PM2.5 dari SPKU Gelora Bung Karno milik Kementerian Lingkungan Hidup. Melalui kombinasi *reconstruction error* dan *adjusted anomaly-class score*, model menghasilkan skor anomali yang menunjukkan tingkat penyimpangan suatu *window* terhadap pola yang dipelajari, dan kinerja REDLAMP dievaluasi berdasarkan variasi ukuran *window* dan jenis pseudo-anomali hasil augmentasi. Hasil evaluasi menunjukkan bahwa REDLAMP dapat membedakan titik anomali dan titik normal, dengan VUS-ROC tertinggi sebesar 0,8489 dan VUS-PR tertinggi sebesar 0,1800. Komponen *reconstruction only* secara konsisten menghasilkan performa tertinggi dibandingkan *combined* dan *classification only*, *window* yang lebih panjang menurunkan kemampuan model dalam memberi *ranking* anomali secara global, dan konfigurasi enam kelas pseudo-anomali (K=6) memperoleh performa tertinggi dibandingkan K=2 maupun K=12. Secara keseluruhan, REDLAMP dapat diterapkan untuk mendeteksi anomali pada data PM2.5 univariat secara *unsupervised*, dengan performa optimal yang bergantung pada konfigurasi ukuran *window* dan jumlah kelas pseudo-anomali yang digunakan.

Kata Kunci: Deteksi Anomali, Time Series, PM2.5, REDLAMP, Pseudo-Anomali Multi Kelas.

Halaman ini sengaja dikosongkan.

ABSTRACT

Anomaly Detection on Time-Series Data of PM2.5 Concentration from The Ministry of Environment's Air Quality Monitoring Station Using REDLAMP

Full Name / Student ID : Alendra Rafif Athaillah/ 5025221297
Department : Informatics FTEIC - ITS
Advisor : Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom.
Co-advisor : Ilham Gurat Adillion, S.Kom., M.Eng.

Abstract

In PM2.5 data from air quality monitoring stations (SPKU), time-series anomaly detection is needed to flag points or subsequences that deviate from normal patterns before the data is used in further analysis, and because the data used does not necessarily provide anomaly labels, an unsupervised approach that can detect deviations based on patterns learned from the data is required. Existing unsupervised approaches still have limitations in handling anomaly contamination, diversity gap, false anomalies, and overconfidence in labels. To address these issues, this research applies REDLAMP (Robust and Explainable Detector of Time-Series Anomaly), an unsupervised method that combines reconstruction and classification with a multiclass approach, to detect anomalies in PM2.5 concentration time-series data from the Gelora Bung Karno air quality monitoring station under the Ministry of Environment. Through the combination of reconstruction error and adjusted anomaly-class score, the model produces an anomaly score that indicates the degree of deviation of a window from the learned pattern, and REDLAMP's performance is evaluated based on variations in window size and types of pseudo-anomalies resulting from augmentation. The evaluation results show that REDLAMP can distinguish anomalous points from normal points, with the highest VUS-ROC of 0.8489 and the highest VUS-PR of 0.1800. The reconstruction-only component consistently achieves the highest performance compared to the combined and classification-only components, longer windows reduce the model's ability to rank anomalies globally, and a configuration of six pseudo-anomaly classes ($K=6$) achieved the highest performance compared to $K=2$ and $K=12$. Overall, REDLAMP can be applied to detect anomalies in univariate PM2.5 data in an unsupervised manner, with optimal performance depending on the window size configuration and the number of pseudo-anomaly classes used.

Keywords: Anomaly detection, time-series, REDLAMP, multiclass pseudo-anomaly, PM2.5 concentration.

This page is intentionally left blank.

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh

Puji syukur kehadiran Allah SWT atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Tugas Akhir ini dengan judul **"Deteksi Anomali pada Data Time Series Konsentrasi PM2.5 Dari Stasiun Pemantau Kualitas Udara Kementerian Lingkungan Hidup Menggunakan REDLAMP"** sebagai salah satu syarat untuk memperoleh gelar Sarjana Komputer.

Penulisan Tugas Akhir ini tidak akan terlaksana dengan baik tanpa bimbingan, dukungan, dan motivasi dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. **Ibu Prof. Dr. Eng. Chastine Fatichah, S.Kom., M.Kom. dan Bapak Ilham Gurat Adillion, S.Kom., M.Eng.** selaku dosen pembimbing 1 dan 2, atas waktu dan tenaganya dalam memberikan bimbingan dan arahan selama pengerjaan Tugas Akhir ini sehingga dapat terselesaikan dengan baik.
2. **Bapak/Ibu Penguji Sidang** yang telah memberikan masukan, kritik, dan saran konstruktif demi perbaikan karya ini.
3. **Direktorat Perlindungan dan Pengelolaan Mutu Udara, Deputi Bidang Pengendalian Pencemaran dan Kerusakan Lingkungan, Kementerian Lingkungan Hidup**, yang telah menyediakan data yang digunakan dalam Tugas Akhir ini.
4. **Ayah dan Ibu** yang telah memberikan dukungan material maupun emosional, sehingga penulis dapat menyelesaikan Tugas Akhir ini.

Penulis menyadari bahwa Tugas Akhir ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Semoga hasil penelitian ini dapat bermanfaat bagi perkembangan ilmu pengetahuan dan teknologi.

Wassalamu'alaikum warahmatullahi wabarakatuh

Hormat saya,
Alendra Rafif Athaillah

Halaman ini sengaja dikosongkan.

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
APPROVAL SHEET	iii
PERNYATAAN ORISINALITAS	v
STATEMENT OF ORIGINALITY	vii
PERNYATAAN KODE ETIK PENGGUNAAN AI GENERATIF.....	ix
ABSTRAK.....	xi
ABSTRACT.....	xiii
KATA PENGANTAR	xv
DAFTAR ISI.....	xvii
DAFTAR GAMBAR	xxi
DAFTAR TABEL	xxiii
DAFTAR KODE SEMU.....	xxv
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	2
1.4 Tujuan.....	3
1.5 Manfaat.....	3
1.5.1 Manfaat Teoritis	3
1.5.2 Manfaat Praktis.....	3
1.5.3 Manfaat Sosial	3
BAB 2 TINJAUAN PUSTAKA	5
2.1 Hasil Penelitian Terdahulu	5
2.2 Dasar Teori	7
2.2.1 Pengukuran Kualitas Udara	7
2.2.2 <i>Sliding Window</i>	7
2.2.3 Deteksi Anomali <i>Time-Series</i>	8
2.2.4 <i>CNN Encoder–Decoder</i>	9
2.2.5 <i>2-Layer MLP Classifier</i>	11
2.2.6 <i>Multiclass Pseudo-Anomaly</i>	11
2.2.7 <i>Label Refurbishment</i>	14
2.2.8 <i>Objective Function REDLAMP</i>	14

2.2.9	Anomaly Scoring.....	16
2.2.10	Receiver Operating Characteristic & ROC-AUC.....	18
2.2.11	Precision–Recall Curve & AUC-PR	20
2.2.12	Volume Under the Surface (VUS).....	21
BAB 3	METODOLOGI.....	25
3.1	Alur Metodologi Penelitian	25
3.1.1	Dataset	26
3.1.2	Preprocessing.....	29
3.1.3	Pembentukan Window (Sliding Window).....	31
3.1.4	Augmentasi Data	31
3.1.5	<i>Label refurbishment</i>	35
3.1.6	Training	36
3.1.7	Inferensi	37
3.1.8	Evaluasi	38
3.1.9	Skenario Pengujian	38
3.2	Bahan dan Peralatan yang Digunakan	39
3.3	Implementasi	41
3.3.1	<i>Load Dataset</i>	41
3.3.2	Pemeriksaan <i>Timestamp</i>	41
3.3.3	Pemeriksaan <i>Missing Values</i>	43
3.3.4	Pemeriksaan <i>Static Values</i>	43
3.3.5	Pemeriksaan <i>Extreme Values</i>	44
3.3.6	Pembentukan Nilai PM2.5 bersih.....	44
3.3.7	<i>Train, Validation, Test Split</i>	45
3.3.8	Interpolasi <i>Missing Values</i>	45
3.3.9	Finalisasi Kolom Pemodelan	46
3.3.10	<i>Min-Max Normalization</i>	46
3.3.11	Pembentukan <i>Sliding Window</i>	47
3.3.12	Augmentasi Pseudo-Anomali	47
3.3.13	<i>Label Refurbishment</i>	48
3.3.14	Implementasi Model REDLAMP	49
3.3.15	Implementasi <i>Loss Function</i>	51
3.3.16	Hyperparameter Tuning.....	52
3.3.17	<i>Load Best Model</i>	53

3.3.18	Train Final dengan Best Model	53
3.3.19	Inferensi pada <i>Test Set</i>	54
3.3.20	Anomaly Scoring dan Frequent Anomaly Adjustment	54
3.3.21	Evaluasi	56
BAB 4	HASIL DAN PEMBAHASAN	59
4.1	Hasil Skenario Pengujian	59
4.1.1	Hasil Hyperparameter Tuning pada Skenario Acuan	59
4.1.2	Hasil Skenario Pengujian Menggunakan Parameter Acuan	59
4.2	Pembahasan	61
4.2.1	Performa Komponen Scoring	61
4.2.2	Analisis Hasil terhadap Ukuran Window	63
4.2.3	Analisis Hasil terhadap Jumlah Kelas Pseudo-Anomali	64
4.2.4	Analisis Prediksi Model	64
BAB 5	KESIMPULAN DAN SARAN	69
5.1	Kesimpulan	69
5.2	Saran	70
DAFTAR PUSTAKA		71
LAMPIRAN		73
BIODATA PENULIS		77

Halaman ini sengaja dikosongkan.

DAFTAR GAMBAR

Gambar 2.1. Arsitektur Encoder REDLAMP	9
Gambar 2.2. Arsitektur Decoder REDLAMP	10
Gambar 2.3. Arsitektur 2-Layer MLP	11
Gambar 2.4. Kurva ROC-AUC	18
Gambar 2.5. Kurva AUC-PR.....	20
Gambar 2.6. Kurva VUS-ROC.....	22
Gambar 2.7. Kurva VUS-PR	23
Gambar 3.1. Alur Metodologi	25
Gambar 3.2. Contoh Anomali pada Dataset.....	26
Gambar 3.3. Inkonsistensi Interval akibat Off-Grid Timestamp.....	26
Gambar 3.4. Contoh Short Gap	27
Gambar 3.5. Contoh Long Gap	27
Gambar 3.6. Sliding Window (Benson et al., 2020).....	31
Gambar 3.7. Contoh Hasil Augmentasi sebuah Window	34
Gambar 3.8. Contoh Perbandingan Hasil Augmentasi dengan Anomali pada Dataset.....	35
Gambar 4.1. Hasil Frequent Anomaly Adjustment pada $L = 24$ dan $K = 6$	65
Gambar 4.2. Window Normal dengan Skor Tertinggi	65
Gambar 4.3. Prediksi Soft Label pada Window Normal dengan Skor Tertinggi.....	65
Gambar 4.4. Prediksi Classifier pada Window Normal dengan Probabilitas yang Tersebar ...	66
Gambar 4.5. Window Anomali dengan Skor Terendah	67
Gambar 4.6. Prediksi Soft Label pada Window Anomali dengan Skor Terendah.....	67

Halaman ini sengaja dikosongkan.

DAFTAR TABEL

Tabel 2.1. Penelitian Terdahulu.....	5
Tabel 3.1. 10 Nilai PM2.5 Teratas.....	28
Tabel 3.2. Entitas Rangkaian Nol.....	28
Tabel 3.3. Entitas Rangkaian Nilai Statis.....	29
Tabel 3.4. Parameter Augmentasi Pseudo-Class.....	32
Tabel 3.5. Ruang Pencarian Hyperparameter Tuning.....	37
Tabel 3.6. Skenario Pengujian.....	39
Tabel 3.7. Perangkat Keras yang Digunakan.....	39
Tabel 3.8. Platform yang Digunakan.....	39
Tabel 3.9. Perangkat Lunak yang Digunakan.....	40
Tabel 3.10. Pustaka Pendukung yang Digunakan.....	40
Tabel 4.1. Hasil Hyperparameter Tuning Terbaik.....	59
Tabel 4.2. Hasil Seluruh Skenario.....	60
Tabel 4.3. Jumlah Window dan Cakupan Evaluasi.....	61
Tabel 4.4. Jumlah Segmen Evaluasi pada VUS.....	61
Tabel 4.5. Kontribusi Komponen Scoring pada $L = 24$	62
Tabel 4.6. Kontribusi Komponen Scoring pada $L = 48$	62
Tabel 4.7. Kontribusi Komponen Scoring pada $L = 96$	63
Tabel 4.8. Rata-rata Combined Score berdasarkan Ukuran Window.....	63
Tabel 4.9. Rata-rata Combined Score Berdasarkan Pseudo-Anomali.....	64

Halaman ini sengaja dikosongkan.

DAFTAR KODE SEMU

Kode Semu 3.1. Algoritma Augmentasi.....	33
Kode Semu 3.2. Load Dataset	41
Kode Semu 3.3. Pemeriksaan Timestamp Off-Grid.....	41
Kode Semu 3.4. Pengecekan Timestamp Duplikat	42
Kode Semu 3.5. Reindeks Timestamp	42
Kode Semu 3.6. Pemeriksaan Missing Values	43
Kode Semu 3.7. Pemeriksaan Nilai Statis	43
Kode Semu 3.8. Pemeriksaan Nilai PM2.5 Tertinggi	44
Kode Semu 3.9. Pembentukan Nilai PM2.5 Clean.....	44
Kode Semu 3.10. Train, Validation, Test Split	45
Kode Semu 3.11. Interpolasi Missing Values	45
Kode Semu 3.12. Finalisasi Clean Split	46
Kode Semu 3.13. Min-Max Normalization	46
Kode Semu 3.14. Pembentukan Window (Sliding Window).....	47
Kode Semu 3.15. Fungsi Augmentasi	48
Kode Semu 3.16. Label Refurbishment.....	48
Kode Semu 3.17. Convblock1D	49
Kode Semu 3.18. Encoder	49
Kode Semu 3.19. Decoder	50
Kode Semu 3.20. Classifier	50
Kode Semu 3.21. REDLAMP	51
Kode Semu 3.22. Loss Function.....	51
Kode Semu 3.23. Hyperparameter Tuning.....	52
Kode Semu 3.24. Loop Hyperparameter Tuning	52
Kode Semu 3.25. Load Best Model.....	53
Kode Semu 3.26. Train Akhir dengan Best Model	53
Kode Semu 3.27. Fungsi Inferensi	54
Kode Semu 3.28. Reconstruction Error	54
Kode Semu 3.29. Frequent Anomaly Adjustment.....	55
Kode Semu 3.30. Anomaly Scoring	55
Kode Semu 3.31. Endpoint Mapping	56
Kode Semu 3.32. ROC-AUC dan AUC-PR.....	56
Kode Semu 3.33. VUS Input (Guarded)	57
Kode Semu 3.34. Evaluasi Akhir	57

Halaman ini sengaja dikosongkan.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Indonesia merupakan salah satu negara dengan tingkat polusi udara yang tinggi di dunia. Pada tahun 2024, Indonesia menempati peringkat 15 pada daftar negara dengan tingkat polusi tertinggi di dunia oleh IQAIR. Dengan indeks PM_{2.5} sebesar 35,5, yang berarti tingkat Partikel Udara dengan ukuran 2,5 μm (mikrometer) di Indonesia melebihi standar WHO sebesar 7 sampai 10 kali lipat (IQAIR, 2024).

Di antara kota-kota paling berpolusi di Indonesia, Ibu Kota Indonesia, yaitu Jakarta, menempati peringkat ke-8 dengan Indeks PM_{2.5} sebesar 41.7 (IQAIR, 2024). Menurut studi *source apportionment* oleh Vital Strategies dan Institut Teknologi Bandung (2020), sumber polusi udara di Jakarta berasal dari berbagai sektor dengan kontribusi yang bervariasi. Kendaraan berbahan bakar bensin dan diesel menyumbang sekitar 32% hingga 57% terhadap konsentrasi PM_{2.5}. Selain itu, sumber non-kendaraan berkontribusi sebesar 17% hingga 46% terhadap polusi udara ambien, termasuk aktivitas antropogenik seperti pembakaran batu bara, pembakaran terbuka, konstruksi, debu jalan, serta sumber alami seperti tanah dan garam laut. Senyawa aerosol anorganik sekunder juga menyumbang sekitar 1% hingga 16%. Variasi kontribusi tersebut dipengaruhi oleh musim, aktivitas lokal, serta kondisi meteorologi yang memungkinkan polusi dari wilayah sekitar terbawa ke Jakarta oleh angin.

Variabilitas polusi udara yang tinggi memunculkan keperluan untuk memantau kualitas udara secara kontinu. Hal ini diwujudkan dengan membangun Air Quality Monitoring System atau Stasiun Pemantauan Kualitas Udara (Wangintan & Sofyan, 2019). Namun, data yang dihasilkan oleh sistem pemantauan berbasis sensor atau instrumen pengukuran tetap perlu melalui proses pengendalian kualitas. Giannoni et al. (2018) menjelaskan bahwa sistem sensor dapat mengalami malfungsi, menghasilkan pembacaan yang tidak tepat, atau mengalami korupsi data, sehingga diperlukan pengendalian kualitas otomatis untuk mendeteksi pembacaan yang anomali. Oleh karena itu, pada data PM_{2.5} dari SPKU, deteksi anomali *time-series* diperlukan untuk menandai titik atau subsekuens yang menyimpang dari pola normal sebelum data digunakan dalam analisis lebih lanjut. Selain itu, karena data yang digunakan belum tentu menyediakan label anomali untuk proses pelatihan, diperlukan pendekatan *unsupervised* yang mampu mendeteksi penyimpangan berdasarkan pola yang dipelajari dari data.

Berbagai pendekatan *unsupervised* untuk deteksi anomali *time-series* telah dikembangkan, masing-masing dengan asumsi dan mekanisme yang berbeda dalam mengenali pola anomali. Beberapa metode yang telah dikembangkan seperti *reconstruction-based model* dan *one-class classification* menganggap bahwa dataset yang digunakan terdiri atas data normal (*Normality Assumption*). Namun, kontaminasi anomali pada dataset terkadang tidak dapat dihindari. Hal ini dapat menyebabkan *over-generalization* pada model. Selain itu, asumsi bahwa anomali dapat direpresentasikan sebagai satu kelas saja (*Binary Anomaly Assumption*) tidak cukup untuk menjawab permasalahan penting dalam deteksi anomali *time-series*, yaitu *diversity gap*, kemudian permasalahan *false anomalies* (pseudo-anomali yang tidak realistis), dan juga *overconfidence* yang dapat menyebabkan ketidakstabilan performa model pada kasus *anomaly contamination* (Obata et al., 2025).

Untuk mengatasi permasalahan tersebut, diusulkan REDLAMP (*Robust and Explainable Detector of Time-Series Anomaly*). REDLAMP merupakan sebuah metode *unsupervised* yang menggabungkan *reconstruction* dan *classification* pada kasus deteksi anomali dengan pendekatan *multiclass*. REDLAMP mengatasi permasalahan *diversity gap* dengan

mengaumentasi data agar menghasilkan pseudo-anomali yang beragam. Selain itu, label pada masing-masing pseudo-anomali berupa *soft labels* sehingga mencegah *overconfidence* yang dapat disebabkan oleh label *noise*, anomaly contamination, dan false anomalies. Arsitektur REDLAMP mampu memberikan skor anomali berdasarkan *Reconstruction Error* (MSE) dari *decoder* dan *adjusted anomaly-class score* dari *classifier* (Obata et al., 2025).

Penelitian ini mengkaji penerapan REDLAMP untuk mendeteksi anomali pada data *time-series* yang dihasilkan oleh SPKU milik Kementerian Lingkungan Hidup. REDLAMP dirancang untuk mengurangi dampak *anomaly contamination* dan *false anomalies* melalui penggunaan pseudo-anomali *multiclass* serta *soft labels* pada proses pelatihan. Melalui kombinasi *reconstruction error* dan *adjusted anomaly-class score*, model menghasilkan skor anomali yang menunjukkan tingkat penyimpangan suatu *window* terhadap pola yang dipelajari. Keluaran *classifier* juga dapat digunakan untuk memberikan indikasi kedekatan pola suatu *window* terhadap kelas pseudo-anomali tertentu.

1.2 Rumusan Masalah

Dalam penerapan deteksi anomali pada data Konsentrasi PM2.5 dari SPKU milik Kementerian Lingkungan Hidup menggunakan metode REDLAMP, terdapat beberapa persoalan yang perlu dijawab, yaitu:

1. Bagaimana cara mendeteksi anomali pada data *time-series* konsentrasi PM2.5 yang dihasilkan oleh SPKU milik Kementerian Lingkungan Hidup yang tidak memiliki label anomali?
2. Bagaimana menerapkan metode REDLAMP sebagai pendekatan *unsupervised multiclass anomaly detection* pada data tersebut?
3. Bagaimana kinerja REDLAMP dalam mendeteksi anomali pada data konsentrasi PM2.5 ketika dibandingkan berdasarkan variasi ukuran *window* dan variasi jenis pseudo-anomali hasil augmentasi?

1.3 Batasan Masalah

Pada penelitian ini, terdapat beberapa batasan yang ditetapkan untuk memastikan bahwa hasil penelitian relevan dan proses pengembangan yang berada pada lingkup yang dapat dikelola oleh peneliti. Adapun batasan yang ditetapkan untuk membahas permasalahan penelitian ini adalah:

1. Data yang digunakan merupakan data Konsentrasi PM2.5 dari Stasiun Pemantauan Kualitas Udara (SPKU) yang berada pada Gelanggang Olahraga Bung Karno, Jakarta, yang dikelola oleh Kementerian Lingkungan Hidup.
2. Variabel yang dianalisis dibatasi pada polutan PM2.5 sebagai representasi data univariat.
3. Rentang waktu data yang digunakan dibatasi dari tahun 2021-2023 sesuai dengan ketersediaan data yang dihasilkan oleh Stasiun Pemantau Kualitas Udara milik Kementerian Lingkungan Hidup tersebut.
4. Penelitian hanya berfokus pada deteksi anomali secara *unsupervised*, tanpa melakukan klasifikasi kualitas udara atau prediksi konsentrasi polutan.
5. Metode yang digunakan dibatasi pada arsitektur REDLAMP, tanpa membandingkannya dengan metode deteksi anomali lain.

6. Evaluasi model dibatasi pada pengaruh variasi *window size* dan variasi jumlah kelas pseudo-anomali hasil augmentasi yang tersedia dalam mekanisme REDLAMP.
7. Penelitian tidak membahas faktor eksternal seperti meteorologi (suhu, kelembapan, kecepatan angin) atau sumber emisi, karena fokus utama adalah deteksi anomali secara univariat.

1.4 Tujuan

Penelitian ini bertujuan untuk menerapkan deteksi anomali *unsupervised* pada data *time-series*. Adapun rincian tujuan dari penelitian ini adalah sebagai berikut:

1. Mendeteksi anomali pada data *time-series* konsentrasi PM2.5 yang dihasilkan oleh SPKU milik Kementerian Lingkungan Hidup.
2. Menerapkan metode REDLAMP sebagai pendekatan *unsupervised multiclass anomaly detection* pada data konsentrasi PM2.5 univariat untuk menghasilkan skor anomali berdasarkan *Reconstruction Error* dan *adjusted anomaly-class score* dari probabilitas keluaran *classifier* setelah *Frequent Anomaly Adjustment*.
3. Meninjau kinerja REDLAMP dalam mendeteksi anomali pada data konsentrasi PM2.5 berdasarkan variasi ukuran *window* dan variasi jenis pseudo-anomali hasil augmentasi.

1.5 Manfaat

Penelitian ini diharapkan dapat bermanfaat, baik secara teoritis maupun praktis, dalam penerapan deteksi anomali *time-series unsupervised*. Adapun manfaat penelitian ini adalah sebagai berikut.

1.5.1 Manfaat Teoritis

1. Menambah wawasan dalam bidang deteksi anomali *time-series*, khususnya pada pendekatan *unsupervised*.
2. Menjadi referensi ilmiah mengenai penerapan metode REDLAMP untuk deteksi anomali pada data lingkungan, khususnya data konsentrasi PM2.5 univariat.
3. Memberikan pemahaman mengenai pengaruh variasi ukuran *window* dan jenis pseudo-anomali terhadap kinerja model REDLAMP dalam deteksi anomali *time-series* secara *unsupervised*.

1.5.2 Manfaat Praktis

1. Menyediakan metode deteksi anomali yang dapat membantu meningkatkan kualitas data konsentrasi PM2.5 dari SPKU Gelora Bung Karno.
2. Memberikan alat bantu bagi instansi pengelola *monitoring* kualitas udara untuk menandai kandidat anomali pada data sebelum dipublikasikan atau digunakan dalam analisis lanjutan.

1.5.3 Manfaat Sosial

1. Mendukung penyediaan informasi kualitas udara yang lebih akurat kepada masyarakat, khususnya Masyarakat DKI Jakarta.
2. Membantu pemerintah dalam meningkatkan efektivitas pengelolaan lingkungan.

Halaman ini sengaja dikosongkan.

BAB 2

TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Pada subbab ini akan dijelaskan mengenai beberapa penelitian terdahulu yang memiliki kesamaan atau keterkaitan dengan penelitian ini.

Tabel 2.1. Penelitian Terdahulu

Peneliti & Tahun Penelitian	Fokus Penelitian	Gap Terhadap Penelitian
<i>Anomaly Detection in Univariate Time-Series: A Survey on the State-of-the-Art</i> (Braei & Wagner, 2020)	Melakukan survei dan evaluasi terhadap 20 metode deteksi anomali deret waktu univariat yang mencakup pendekatan statistik, <i>machine learning</i> , dan <i>deep learning</i> . Metode-metode tersebut dibandingkan pada beberapa <i>dataset</i> publik berdasarkan akurasi deteksi dan waktu komputasi untuk mengidentifikasi kelebihan, keterbatasan, serta kesesuaian metode terhadap karakteristik data tertentu.	Penelitian tersebut memberikan gambaran umum mengenai metode deteksi anomali <i>time series</i> univariat, tetapi belum secara khusus menganalisis pengaruh variasi panjang <i>window</i> terhadap performa.
<i>Anomaly Detection in Time Series: A Comprehensive Evaluation</i> (Schmidl et al., 2022)	Melakukan evaluasi komprehensif terhadap 71 algoritma deteksi anomali pada 976 <i>dataset time series</i> univariat dan multivariat. Algoritma dikelompokkan berdasarkan jenis pembelajaran dan keluarga metode, seperti <i>forecasting</i> , <i>reconstruction</i> , <i>distance</i> , <i>encoding</i> , <i>distribution</i> , dan <i>isolation tree</i> . Evaluasi dilakukan menggunakan beberapa metrik, termasuk AUC-ROC, AUC-PR, dan metrik berbasis rentang, untuk menganalisis efektivitas, efisiensi, robustness, serta kesesuaian algoritma terhadap karakteristik data dan jenis anomali yang berbeda.	Penelitian ini mengakui bahwa <i>window size</i> merupakan parameter yang bergantung pada karakteristik dataset, tetapi belum secara khusus mengevaluasi pengaruh variasi <i>window size</i> terhadap performa satu arsitektur tertentu.
<i>Neural Contextual Anomaly Detection for Time Series</i> (Carmona et al., 2021)	Mengusulkan NCAD, yaitu metode deteksi anomali berbasis <i>window</i> yang	Penelitian ini telah memanfaatkan <i>synthetic anomalies</i> untuk membantu

	membandingkan representasi <i>context window</i> dan <i>suspect window</i> menggunakan Temporal Convolutional Network. Model dilatih dengan bantuan <i>synthetic anomalies</i> yang dihasilkan melalui <i>Contextual Outlier Exposure</i> , <i>point-outlier injection</i> , dan <i>mixup</i> , sehingga dapat digunakan pada skenario <i>unsupervised</i> , <i>semi-supervised</i> , maupun <i>supervised</i> untuk data univariat dan multivariat.	model membedakan pola normal dan anomali, tetapi pseudo-anomali tersebut masih diperlakukan sebagai satu kelas anomali dalam kerangka klasifikasi biner.
<i>A Deep Neural Network for Unsupervised Anomaly Detection and Diagnosis in Multivariate Time Series Data</i> (Zhang et al., 2018)	Mengusulkan MSCRED, yaitu metode deteksi dan diagnosis anomali pada data deret waktu multivariat dengan membentuk <i>multi-scale signature matrices</i> , mempelajari korelasi antarsensor menggunakan convolutional encoder dan <i>attention-based</i> ConvLSTM, kemudian merekonstruksi <i>signature matrices</i> melalui <i>convolutional decoder</i> . Residual hasil rekonstruksi digunakan untuk menghasilkan skor anomali serta membantu identifikasi akar penyebab dan tingkat keparahan anomali.	Penelitian ini menghasilkan skor anomali berdasarkan residual hasil rekonstruksi, sehingga proses deteksi masih bergantung pada reconstruction-based scoring dan belum menggabungkannya dengan informasi dari classifier.
<i>Revisiting Time Series Outlier Detection: Definitions and Benchmarks</i> (Lai et al., 2021)	Mengusulkan pengelompokan anomali <i>time series</i> yang mengategorikan anomali menjadi lima jenis: <i>point-wise (global, contextual)</i> dan <i>pattern-wise (shapelet, seasonal, trend)</i> , disertai kriteria sintesis umum untuk masing-masing jenis berdasarkan analisis spektral terhadap struktur <i>shapelet</i> , <i>seasonality</i> , dan <i>trend</i> dari data <i>time series</i> .	Penelitian tersebut berhenti pada perumusan dan kriteria sintesis anomali untuk keperluan <i>benchmarking</i> berbagai algoritma deteksi anomali klasik dan <i>deep learning</i> , tanpa mengusulkan arsitektur model deteksi anomali baru maupun skema pelatihan berbasis klasifikasi <i>multiclass</i> seperti REDLAMP dan penelitian ini.
<i>Robust and Explainable Detector of Time Series</i>	Mengusulkan REDLAMP, yaitu metode deteksi anomali	Penelitian ini telah mengevaluasi pengaruh jumlah

<i>Anomaly via Augmenting Multiclass Pseudo-Anomalies</i> (Obata et al., 2025)	deret waktu <i>unsupervised</i> yang menggabungkan <i>masked reconstruction</i> dan <i>multiclass classification</i> . Metode ini menghasilkan berbagai kelas pseudo-anomali melalui <i>data-augmentation</i> , menggunakan <i>soft labels</i> untuk mengurangi <i>overconfidence</i> akibat <i>anomaly contamination</i> dan <i>false anomalies</i> , serta menggabungkan <i>reconstruction error</i> dengan <i>adjusted anomaly-class score</i> sebagai skor anomali.	pseudo-anomali terhadap performa REDLAMP, tetapi belum secara khusus menganalisis pengaruh variasi <i>window size</i> terhadap kinerja model.
---	---	---

2.2 Dasar Teori

Pada subbab ini akan dijelaskan mengenai teori-teori yang terkait dengan penelitian ini.

2.2.1 Pengukuran Kualitas Udara

2.2.1.1 Sistem Pemantauan Kualitas Udara (AQMS / SPKU)

Stasiun Pemantau Kualitas Udara (SPKU) merupakan instrumen yang dirancang untuk mengukur konsentrasi pencemar udara ambien serta parameter meteorologi pendukung seperti suhu udara, kelembaban, radiasi, serta arah dan kecepatan angin. Secara teknis, operasional SPKU dapat dibedakan berdasarkan teknologi pengukurannya, yaitu menggunakan *analyzer* atau sensor. SPKU berbasis *analyzer* memiliki tingkat akurasi yang lebih tinggi karena setiap parameter kimia udara diukur menggunakan instrumen khusus yang terpisah. Sebaliknya, SPKU berbasis sensor umumnya memiliki tingkat akurasi yang lebih rendah karena sering kali menggunakan satu unit deteksi untuk berbagai parameter tanpa indikator pemisah yang spesifik. Data yang dihasilkan oleh SPKU ditarik melalui *server* pusat (*regional center*) untuk direkapitulasi dalam sistem pelaporan harian, mingguan, hingga tahunan guna memantau kondisi udara ambien secara kontinu (Damayanti & Handriyono, 2022).

2.2.1.2 Particulate Matter PM2.5

Udara ambien terdiri dari campuran berbagai gas, debu, dan partikel-partikel lain. Salah satu parameter krusial dalam pengukuran kualitas udara adalah PM2.5. PM2.5 adalah pengukuran konsentrasi partikel dengan diameter aerodinamis kurang dari 2,5 mikrometer. Berbeda dengan parameter gas seperti CO atau SO₂, PM2.5 sering kali menjadi parameter penentu dalam status mutu udara. Sebagai contoh, dalam evaluasi di SPKU Tandes, parameter PM2.5 tercatat pernah mencapai kategori "Sedang" (nilai ISPU 51-100) meskipun parameter pencemar lainnya berada dalam kategori "Baik". Hal ini menunjukkan signifikansi pengukuran PM2.5 sebagai indikator polusi yang sensitif terhadap aktivitas masyarakat dan kondisi lingkungan (Damayanti & Handriyono, 2022).

2.2.2 Sliding Window

Teknik sliding window merupakan metode fundamental dalam pra-pemrosesan data untuk mengubah urutan data deret waktu yang kontinu menjadi serangkaian sub-urutan (subsequences) diskrit dengan ukuran tetap. Dalam pendekatan ini, sebuah jendela dengan

panjang L (atau sering dinotasikan sebagai w) digerakkan di sepanjang sumbu waktu dari data input (Braei & Wagner, 2020; Schmidl et al., 2022). Metode ini memungkinkan algoritma pembelajaran mesin untuk memproses data temporal yang panjang dengan memecahnya menjadi segmen-segmen lokal yang lebih mudah dikelola.

Secara matematis, jika diberikan suatu deret waktu seperti pada Persamaan 2.1.

$$X = \{x_1, x_2, \dots, x_T\} \quad (2.1)$$

Di mana:

- x_i menyatakan nilai data pada indeks waktu ke- i ,
- T menyatakan panjang total deret waktu.

maka teknik *sliding window* akan menghasilkan sekumpulan instance pelatihan. Setiap instance dinyatakan seperti pada Persamaan 2.2

$$X(t) = \{x_{t-L+1}, x_{t-L+2}, \dots, x_t\} \quad (2.2)$$

Di mana:

- $X(t)$ menyatakan *window* yang berakhir pada indeks waktu t ,
- t menyatakan indeks waktu saat ini,
- L merupakan ukuran *window*.

Pada umumnya, pergeseran *window* dilakukan dengan *stride* sebesar satu, yaitu *window* digeser satu *time step* ke depan untuk setiap instance baru. Pendekatan ini menghasilkan *overlap* antar *instance* yang berdekatan, sehingga kontinuitas informasi temporal tetap terjaga (Obata et al., 2025; Schmidl et al., 2022).

2.2.3 Deteksi Anomali *Time-Series*

2.2.3.1 Definisi

Deteksi anomali pada deret waktu (*Time-Series Anomaly Detection*) merupakan proses mengidentifikasi titik data atau subsekuens yang menyimpang dari pola yang sering muncul berdasarkan karakteristik, model, embedding, atau ukuran kesamaan tertentu. Anomali dapat menunjukkan kejadian penting, seperti kegagalan sistem, kerusakan struktural, maupun gangguan ekosistem. Karena anomali umumnya belum diketahui sebelumnya, pendekatan supervised relatif jarang digunakan dalam praktik. Pendekatan unsupervised memungkinkan proses deteksi dilakukan tanpa membutuhkan label atau pengetahuan awal mengenai lokasi anomali (Schmidl et al., 2022).

2.2.3.2 Pengelompokan Anomali

Secara teoritis, anomali dalam deret waktu dapat dikategorikan ke dalam tiga jenis utama sebagai berikut:

- Anomali Titik (*Point Anomalies*)

Anomali Titik terjadi ketika satu titik data menyimpang secara ekstrem dari distribusi umum data, misalnya transaksi kredit bernilai besar yang berbeda dari transaksi lainnya (Braei & Wagner, 2020).

- Anomali Kolektif (*Collective Anomalies*)

Anomali kolektif terjadi ketika sekumpulan titik data dianggap abnormal jika dilihat sebagai satu kesatuan urutan, meskipun masing-masing titik secara individual dapat tampak normal (Braei & Wagner, 2020).

- Anomali Kontekstual (*Contextual Anomalies*)

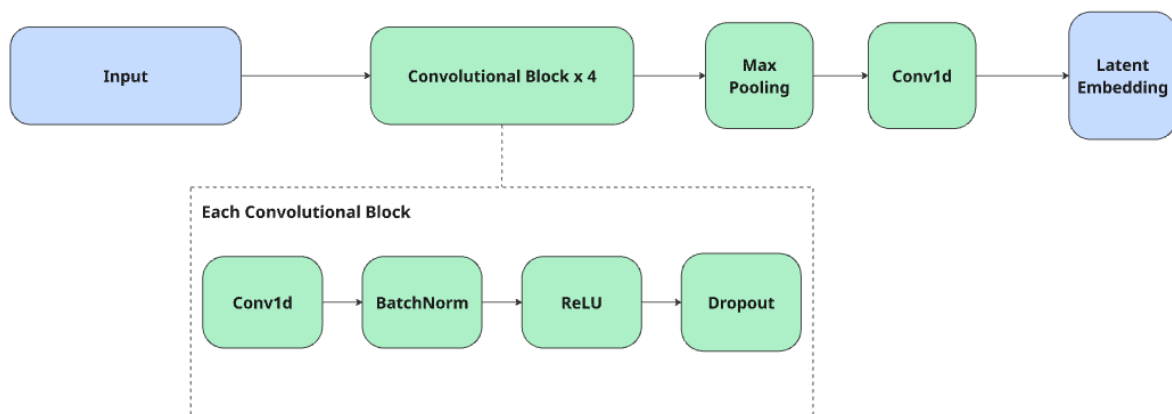
Anomali kontekstual merujuk pada titik data yang dianggap normal dalam satu konteks tertentu, namun menjadi anomali dalam konteks lainnya, misalnya suhu 35°C yang normal pada musim panas tetapi tidak wajar pada musim dingin (Braei & Wagner, 2020).

2.2.4 CNN Encoder–Decoder

2.2.4.1 1-D CNN pada Data *Time-Series*

Convolutional Neural Network (CNN) telah diadaptasi secara luas untuk analisis data deret waktu (*time series*). Berbeda dengan CNN pada citra yang umumnya menerapkan konvolusi dua dimensi, CNN pada deret waktu menggunakan operasi konvolusi satu dimensi sepanjang sumbu waktu atau 1D-CNN. Arsitektur ini digunakan untuk mengekstraksi pola temporal lokal dari data (Braei & Wagner, 2020).

2.2.4.2 Arsitektur *Encoder*

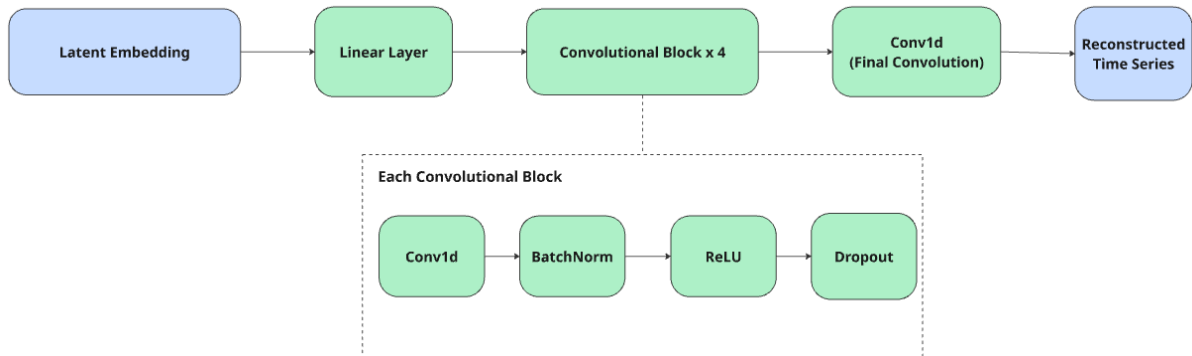


Gambar 2.1. Arsitektur Encoder REDLAMP

Encoder bertugas memetakan input *time-series* menjadi representasi laten (*latent space*) yang berdimensi lebih rendah. Dalam metode berbasis rekonstruksi, representasi laten digunakan untuk menyimpan informasi penting dari *input* yang diperlukan dalam proses rekonstruksi (Schmidl et al., 2022). Model REDLAMP menggunakan *encoder* yang terdiri dari serangkaian blok konvolusional. Setiap blok dilengkapi dengan lapisan konvolusi 1D, *Batch Normalization*, fungsi aktivasi ReLU, dan *Dropout* untuk mencegah *overfitting*. Output dari

blok-blok ini kemudian diproses lebih lanjut melalui lapisan *max-pooling* dan konvolusi tambahan untuk menghasilkan vektor *embedding* akhir (Obata et al., 2025).

2.2.4.3 Arsitektur Decoder



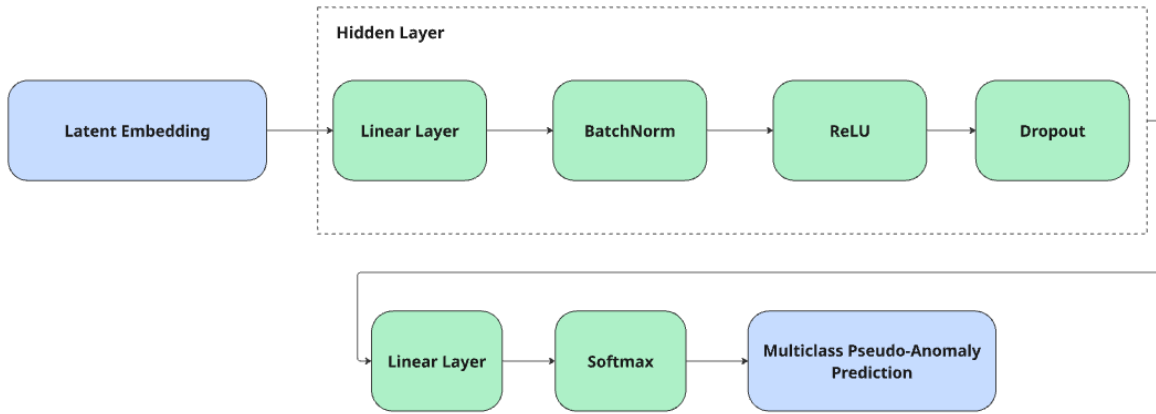
Gambar 2.2. Arsitektur Decoder REDLAMP

Decoder memiliki struktur yang umumnya simetris atau berkebalikan dengan *encoder*. Fungsi utamanya adalah merekonstruksi kembali data deret waktu dari representasi laten yang dihasilkan oleh *encoder*. Proses ini melibatkan operasi rekonstruksi untuk mengembalikan dimensi data ke ukuran aslinya. Dalam konteks deteksi anomali, *decoder* dilatih untuk memprediksi rekonstruksi data dengan meminimalisir perbedaan antara *output* dan *input* asli. Pada REDLAMP, proses rekonstruksi menggunakan masked reconstruction loss. *Anomaly mask* digunakan untuk menandai bagian *window* yang telah dimodifikasi melalui *data augmentation*. Kesalahan rekonstruksi hanya dihitung pada bagian yang tidak ditandai sebagai pseudo-anomali. Dengan demikian, *decoder* diarahkan untuk mempelajari rekonstruksi bagian data yang bebas dari *pseudo-anomali* tanpa dipaksa merekonstruksi pola anomali buatan (Obata et al., 2025).

2.2.4.4 Prinsip Deteksi Anomali Berbasis Rekonstruksi

Prinsip dasar deteksi anomali menggunakan arsitektur ini adalah *encoder* dilatih untuk menangkap pola yang dipelajari dari data pelatihan, sehingga *decoder* dapat merekonstruksinya dengan *error* yang rendah. Sebaliknya, ketika input yang diterima menyimpang, hasil rekonstruksinya cenderung memiliki *error* yang lebih tinggi. Nilai penyimpangan atau *error* rekonstruksi inilah yang kemudian digunakan sebagai skor anomali (Braei & Wagner, 2020; Schmidl et al., 2022).

2.2.5 2-Layer MLP Classifier



Gambar 2.3. Arsitektur 2-Layer MLP

Multi-Layer Perceptron (MLP) merupakan jaringan saraf *feed-forward* yang terdiri atas lapisan-lapisan *fully connected* dan menggunakan fungsi aktivasi untuk memodelkan hubungan non-linear pada data (Braei & Wagner, 2020). Pada REDLAMP, MLP tidak menerima window deret waktu secara langsung, melainkan menerima representasi laten atau *embedding* yang dihasilkan oleh encoder. REDLAMP menggunakan *2-layer MLP* sebagai classifier untuk memprediksi kelas dari setiap embedding. Kelas tersebut mencakup kelas normal dan berbagai kelas pseudo-anomali yang dihasilkan melalui data augmentation. Dengan demikian, classifier berperan dalam mempelajari pemisahan antar kelas pada ruang laten dan mendukung pendekatan klasifikasi multikelas yang digunakan oleh REDLAMP (Obata et al., 2025).

2.2.6 Multiclass Pseudo-Anomaly

Pendekatan pseudo-anomali *multiclass* bertujuan untuk memperkaya representasi pola anomali dengan cara menghasilkan berbagai jenis anomali (*pseudo-anomalies*) melalui augmentasi data. Berbeda dengan pendekatan *binary anomaly assumption* yang hanya membedakan data normal dan anomali, pendekatan ini mengasumsikan bahwa anomali memiliki beragam karakteristik dan bentuk. Setiap jenis augmentasi diperlakukan sebagai satu kelas anomali yang berbeda, sehingga model klasifikasi dapat mempelajari batas keputusan yang lebih kaya dan robust terhadap variasi pola anomali (Obata et al., 2025).

Augmentasi pseudo-anomali pada REDLAMP mengadopsi strategi injeksi anomali sintetis dari Goswami et al. (2023). Strategi ini dirancang untuk menghasilkan anomali dengan beragam karakteristik yang mencakup lima jenis anomali deret waktu menurut taksonomi Lai et al. (2021), yaitu global, contextual, shapelet, seasonal, dan trend (Obata et al., 2025). Untuk mencakup kelima jenis tersebut, dirancang sejumlah augmentasi yang masing-masing meniru salah satu bentuk penyimpangan. Secara keseluruhan terdapat 12 kelas, yang terdiri atas 1 kelas normal dan 11 kelas pseudo-anomali.

Misalkan data *time-series* multivariat dinyatakan seperti pada Persamaan 2.3

$$X^{(t,k)} = f^{(k)}(X^{(t)}, dim, st, ed) \quad (2.3)$$

dengan:

- t menyatakan indeks waktu
- k menyatakan indeks kelas augmentasi
- dim menyatakan indeks dimensi/fitur yang dimodifikasi,
- st dan ed menyatakan indeks awal dan akhir segmen yang mengalami augmentasi, dengan $st, ed \in \{1, \dots, L\}$ dan $st < ed$.

Maka, bentuk fungsi $f^{(k)}$ untuk masing-masing kelas dijabarkan sebagai berikut:

- Normal: Kelas normal menggunakan data asli tanpa modifikasi.

$$X_{dim}^{(t,k)} = X_{dim}^{(t)} \quad (2.4)$$

- Spike: Menambahkan lonjakan pada satu titik waktu tertentu.

$$X_{dim,st}^{(t,k)} = X_{dim,st}^{(t)} + a, \text{ where } a \sim N(0,1) \quad (2.5)$$

- Flip: Membalik urutan data pada rentang waktu tertentu, mensimulasikan gangguan temporal.

$$X_{dim,st:ed}^{(t,k)} = X_{dim,st:ed}^{(t)}[:, -1] \quad (2.6)$$

- Speedup: Mengubah frekuensi data pada suatu segmen, baik dengan mempercepat ($2\times$) maupun memperlambat ($1/2\times$).

$$X_{dim,st:ed}^{(t,k)} = X_{dim,st:ed+length}^{(t)}[:, 2] \text{ if } 2x \quad (2.7)$$

$$X_{dim,st:ed}^{(t,k)} = \text{Stretch}(X_{dim,st:ed-(length/2)}^{(t)}; [length/2] \rightarrow [length]) \text{ if } 1/2x \quad (2.8)$$

- Noise: Menambahkan noise berdasarkan distribusi normal.

$$X_{dim,st:ed}^{(t,k)} = X_{dim,st:ed}^{(t)} + a, \text{ where } a \sim N(0,0.1I) \quad (2.9)$$

- Cutoff: Mengganti seluruh nilai dalam rentang tertentu dengan satu nilai konstan acak.

$$X_{dim,st:ed}^{(t,k)} = \text{Uni}(\{\min(X_{dim,st:ed}^{(t)}), \dots, \max(X_{dim,st:ed}^{(t)})\}, 1) \quad (2.10)$$

- Average: Menerapkan moving average sesuai ukuran window.

$$X_{dim,st:ed}^{(t,k)} = MovingAverage(X_{dim,st:ed}^{(t)})^{w=L/5} \quad (2.11)$$

- Scale: Mengalikan nilai dengan faktor skala acak.

$$X_{dim,st:ed}^{(t,k)} = a * X_{dim,st:ed}^{(t)}, where a \sim N(1,1) \quad (2.12)$$

- Wander: Menambahkan tren linear dan pergeseran baseline.

$$X_{dim,st:ed}^{(t,k)} = X_{dim,st:ed}^{(t)} + np.linspace(0, a, length), \quad (2.13)$$

$$X_{dim,st}^{(t,k)} += a, where a \sim N(0,1) \quad (2.14)$$

- Contextual: Menggabungkan scaling dan shifting untuk menghasilkan anomali kontekstual.

$$X_{dim,st:ed}^{(t,k)} = a * X_{dim,st:ed}^{(t)} + b, where a \sim N(1,1), b \sim N(0,1) \quad (2.15)$$

- Upside-Down: Membalik nilai terhadap rata-rata segmen.

$$X_{dim,st:ed}^{(t,k)} = 2 * Mean(X_{dim,st:ed}^{(t)}) - X_{dim,st:ed}^{(t)} \quad (2.16)$$

- Mixture: Mensimulasikan anomali akibat pencampuran konteks temporal, yaitu ketika suatu segmen data berasal dari distribusi waktu lain yang tidak sesuai dengan konteks saat ini.

$$X_{dim,st:ed}^{(t,k)} = X_{dim,st:ed}^{(t')}, where t' \in \{L, \dots, T\} \quad (2.17)$$

Seluruh jenis pseudo-anomali beserta formulasi matematisnya pada penelitian ini mengacu pada strategi augmentasi data yang diperkenalkan dalam metode REDLAMP oleh Obata et al. (2025).

2.2.7 Label Refurbishment

Label refurbishment adalah proses mengonversi *hard labels* menjadi *soft labels*. Teknik ini digunakan untuk meningkatkan performa klasifikasi pada kondisi data dengan label yang mengandung *noise* (seperti *anomaly contamination* atau *false anomalies*). Karena pseudo-anomali pada REDLAMP dihasilkan dari data normal melalui augmentasi, pendekatan *backward correction* diterapkan untuk memperhitungkan probabilitas kesalahan pelabelan antar kelas (Obata et al., 2025).

Secara matematis, proses *label refurbishment* pada REDLAMP dirumuskan sebagai berikut. Misalkan $y(t, k)$ merupakan *hard label* untuk data pada waktu t dan kelas k , maka *soft label* hasil koreksi dinyatakan pada Persamaan 2.18 dan Persamaan 2.19.

$$\tilde{y}(t, k) = (1 - \alpha - K\beta) \cdot y(t, k) + \alpha + \beta, \text{ jika } k = 1 \quad (2.18)$$

$$\tilde{y}(t, k) = (1 - \alpha - K\beta) \cdot y(t, k) + \beta, \quad \text{jika } k \neq 1 \quad (2.19)$$

Di mana:

- α merepresentasikan probabilitas bahwa suatu pseudo-anomali salah diklasifikasikan sebagai data normal,
- β menyatakan probabilitas bahwa suatu kelas salah diklasifikasikan sebagai kelas lainnya
- k adalah jumlah kelas. $k = 1$ merepresentasikan kelas normal.

2.2.8 Objective Function REDLAMP

Pada metode REDLAMP, fungsi objektif dirancang untuk membentuk representasi laten yang kaya dan terpisah dengan baik antar kelas pseudo-anomali. Fungsi objektif ini mengombinasikan dua komponen utama, yaitu *masked reconstruction loss* dan *cross-entropy loss*, yang masing-masing berperan dalam mempelajari perilaku normal data serta memisahkan berbagai jenis pseudo-anomali dalam ruang laten (Obata et al., 2025).

2.2.8.1 Masked Reconstruction Loss

Masked reconstruction loss bertujuan untuk meminimalkan kesalahan rekonstruksi hanya pada bagian data yang bebas dari anomali. Decoder memprediksi hasil rekonstruksi $\hat{X}(t, k)$ dari data hasil augmentasi $X(t, k)$ berdasarkan representasi laten yang dihasilkan oleh *encoder*. Untuk mencegah model mempelajari pola anomali buatan yang disisipkan selama proses augmentasi, digunakan sebuah *anomaly mask* $W(t, k)$ yang menandai lokasi anomali pada data. Dengan demikian, kesalahan rekonstruksi hanya dihitung pada area yang dianggap normal, sehingga representasi laten yang dipelajari lebih merefleksikan perilaku normal data (Obata et al., 2025).

Secara matematis, *masked reconstruction loss* didefinisikan pada Persamaan 2.20.

$$L_MSE = || (1 - W^{(t,k)}) \circ (X^{(t,k)} - \hat{X}^{(t,k)}) ||_2^2 \quad (2.20)$$

Di mana:

- L_MSE adalah masked reconstruction loss, yaitu galat rekonstruksi yang hanya dihitung pada bagian data yang bebas anomali;
- $W^{(t,k)}$ adalah anomaly mask untuk window ke-t dengan augmentasi jenis ke-k, bernilai 1 pada titik yang dimodifikasi augmentasi (anomali) dan 0 pada titik yang bebas anomali;
- $X^{(t,k)}$ adalah window input ke-t setelah diterapkan augmentasi jenis ke-k, yang menjadi acuan rekonstruksi;
- $\hat{X}^{(t,k)}$ adalah hasil rekonstruksi window tersebut yang dihasilkan oleh decoder.

2.2.8.2 Cross-Entropy Loss

Selain rekonstruksi, REDLAMP juga menggunakan mekanisme klasifikasi multikelas untuk memisahkan pseudo-anomali dalam ruang laten. *Classifier* memprediksi label $\hat{y}(t,k)$ dari data berdasarkan *embedding* yang dihasilkan oleh *encoder*. *Cross-entropy loss* dihitung dengan membandingkan label prediksi tersebut dengan *soft label* hasil label *refurbishment* $\tilde{y}(t,k)$. *Loss* ini berperan untuk mencegah model melakukan prediksi yang terlalu yakin terhadap data yang mengandung *false anomaly* atau kontaminasi anomali.

Cross-entropy loss dirumuskan seperti pada Persamaan 2.21

$$L_CE = - \sum_{i=1}^K (\tilde{y}_i^{(t,k)} \log(\hat{y}_i^{(t,k)})) \quad (2.21)$$

Di mana:

- i menyatakan indeks kelas yang dijumlahkan, dengan $i = 1, \dots, K$,
- K menyatakan jumlah total kelas (1 kelas normal dan $K - 1$ kelas pseudo-anomali),
- $\tilde{y}_i^{(t,k)}$ menyatakan elemen ke- i dari *soft label* hasil label *refurbishment*,
- $\hat{y}_i^{(t,k)}$ menyatakan probabilitas prediksi kelas ke- i yang dihasilkan *classifier*.

2.2.8.3 Total Loss

Objective Function akhir pada REDLAMP merupakan kombinasi berbobot dari *masked reconstruction loss* dan *cross-entropy loss*. Kombinasi ini memungkinkan model untuk

secara simultan mempelajari pola normal melalui rekonstruksi serta memisahkan berbagai jenis pseudo-anomali melalui klasifikasi multikelas.

Total loss didefinisikan seperti pada Persamaan 2.22.

$$L = \gamma \cdot L_{CE} + (1 - \gamma) \cdot L_{MSE} \quad (2.22)$$

Di mana:

- L_{CE} menyatakan *cross-entropy loss* sebagaimana didefinisikan pada Persamaan 2.21,
- L_{MSE} menyatakan *masked reconstruction loss* sebagaimana didefinisikan pada Persamaan 2.20,
- γ merupakan *hyperparameter* yang mengatur kontribusi relatif antara *loss* klasifikasi dan *loss* rekonstruksi.

2.2.9 Anomaly Scoring

Pada tahap inferensi, REDLAMP menghasilkan skor anomali untuk setiap window pada data *test*. Fungsi ini dirancang untuk mengintegrasikan *reconstruction error* dan *adjusted anomaly-class score*, sehingga model mampu mendeteksi berbagai jenis anomali dengan karakteristik yang berbeda. Skor anomali diharapkan bernilai besar ketika data yang dianalisis mengandung anomali (Obata et al., 2025).

2.2.9.1 Reconstruction Error

REDLAMP dilatih untuk merekonstruksi bagian data yang bebas dari anomali. Oleh karena itu, model diharapkan mampu menghasilkan rekonstruksi yang akurat pada data normal, namun mengalami kesulitan pada bagian data yang mengandung anomali. Tingkat *Reconstruction Error* ini digunakan sebagai salah satu indikator anomali.

Reconstruction error dihitung dengan membandingkan data masukan $X(t)$ dengan hasil rekonstruksi $\hat{X}(t)$ menggunakan norma Euclidean seperti pada Persamaan 2.23.

$$s_{MSE}(t) = ||X(t) - \hat{X}(t)||_2^2 \quad (2.23)$$

Di mana:

- $s_{MSE}(t)$ adalah skor rekonstruksi (*reconstruction score*) untuk window ke- t , yang menjadi indikator anomali berdasarkan besar galat rekonstruksi.
- $X(t)$ adalah window input ke- t (tanpa augmentasi, karena tahap inferensi).
- $\hat{X}(t)$ adalah hasil rekonstruksi window tersebut yang dihasilkan oleh decoder.

2.2.9.2 Adjusted Anomaly-Class Score

Selain *Reconstruction Error*, REDLAMP juga memanfaatkan hasil klasifikasi pseudo-anomali. Namun, beberapa jenis pseudo-anomali hasil augmentasi dapat bersifat *pseudo-normal*, yaitu tidak secara signifikan melanggar normalitas data. Untuk mengatasi hal tersebut, diperkenalkan mekanisme *Frequent Anomaly Adjustment* (FAA), yang menyesuaikan skor kelas anomali berdasarkan frekuensi kemunculannya pada data uji.

Frequent Anomaly Adjustment didefinisikan pada Persamaan 2.24 dan 2.25

$$FAA(\hat{y}_{i(t)}) = 0, \quad \text{if } \left(\frac{1}{(T - L + 1)} \right) * \sum_{\{j=L\}}^T \hat{y}_{i(j)} > \delta \quad (2.24)$$

$$FAA(\hat{y}_i^{(t)}) = \hat{y}_i^{(t)}, \text{otherwise} \quad (2.25)$$

di mana:

- $\hat{y}_i^{(t)}$ menyatakan probabilitas prediksi kelas ke- i pada *window* ke- t ,
- i menyatakan indeks kelas pseudo-anomali,
- j menyatakan indeks *window* yang dijumlahkan dalam perhitungan rata-rata,
- L menyatakan ukuran *window*,
- T menyatakan jumlah total titik waktu pada *test set*,
- δ menyatakan ambang batas (*threshold*) yang menentukan apakah suatu kelas pseudo-anomali dianggap sering muncul (*pseudo-normal class*).

Setelah penyesuaian ini diterapkan, *adjusted anomaly-class score* dihitung sebagai jumlah skor kelas anomali (tidak termasuk kelas normal). Perhitungan *adjusted anomaly-class score* didefinisikan pada Persamaan 2.26

$$s_{CE}(t) = \sum_{i=2}^K FAA(\hat{y}_i^{(t)}) \quad (2.26)$$

di mana:

- K menyatakan jumlah total kelas,
- Penjumlahan dimulai dari $i = 2$, karena $i = 1$ merepresentasikan kelas normal, yang tidak diikutsertakan dalam perhitungan skor anomali.

Kelas pseudo-anomali dengan rata-rata probabilitas prediksi yang melebihi ambang batas δ dianggap sebagai pseudo-normal class, sehingga kontribusi kelas tersebut dihilangkan dari *adjusted anomaly-class score*. Pada REDLAMP, nilai δ ditetapkan sebesar 0,05 secara *default* (Obata et al., 2025).

2.2.9.3 Total Score

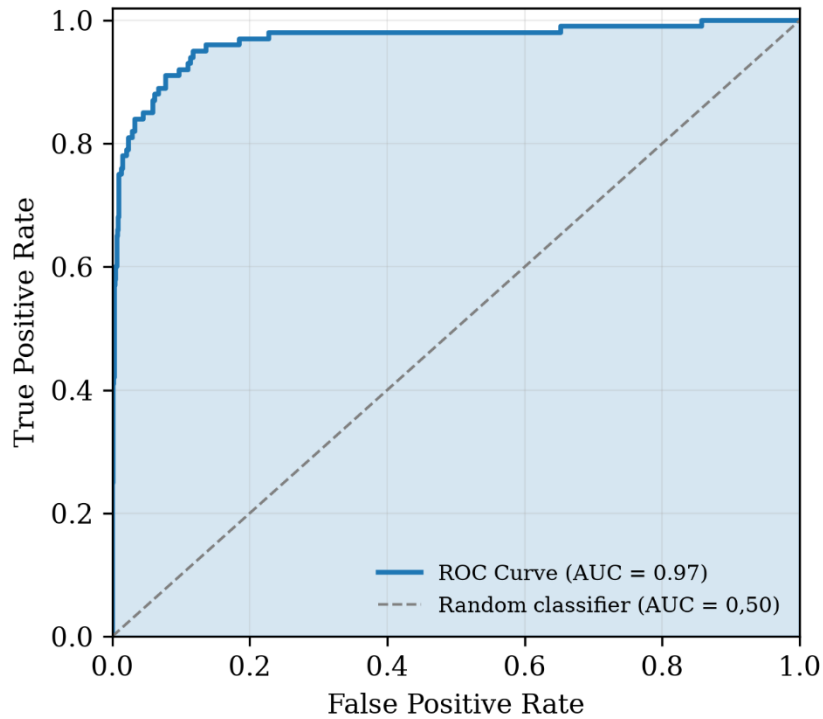
Skor anomali akhir pada REDLAMP diperoleh dengan menggabungkan *reconstruction error* dan *adjusted anomaly-class score*. Sebelum digabungkan, kedua skor dinormalisasi menggunakan *min-max normalization* agar berada pada skala yang sebanding. *Total anomaly score* kemudian dihitung sebagai rata-rata dari kedua skor yang telah dinormalisasi (Obata et al., 2025). Perhitungan *total score* didefinisikan pada Persamaan 2.27

$$s(t) = \frac{1}{2} \cdot \text{MinMax}_T(s_MSE(t)) + \frac{1}{2} \cdot \text{MinMax}_T(s_CE(t)) \quad (2.27)$$

di mana:

- $s_{MSE}(t)$ menyatakan *reconstruction error* pada *window* ke- t sebagaimana didefinisikan pada Persamaan 2.23,
- $s_{CE}(t)$ menyatakan *adjusted anomaly-class score* pada *window* ke- t sebagaimana didefinisikan pada Persamaan 2.26.

2.2.10 Receiver Operating Characteristic & ROC-AUC



Gambar 2.4. Kurva ROC-AUC

Receiver Operating Characteristic Curve merupakan kurva evaluasi klasifikasi biner yang menunjukkan hubungan antara *true positive rate* dan *false positive rate* pada berbagai nilai *threshold*. Dalam penelitian ini, titik anomali diperlakukan sebagai kelas positif, sedangkan titik normal diperlakukan sebagai kelas negatif.

True positive rate (TPR) dirumuskan pada Persamaan 2.28

$$\text{True Positive Rate} = \frac{TP}{TP + FN} \quad (2.28)$$

Dan *False positive rate (FPR)* dirumuskan pada Persamaan 2.29

$$\text{False Positive Rate} = \frac{FP}{FP + TN} \quad (2.29)$$

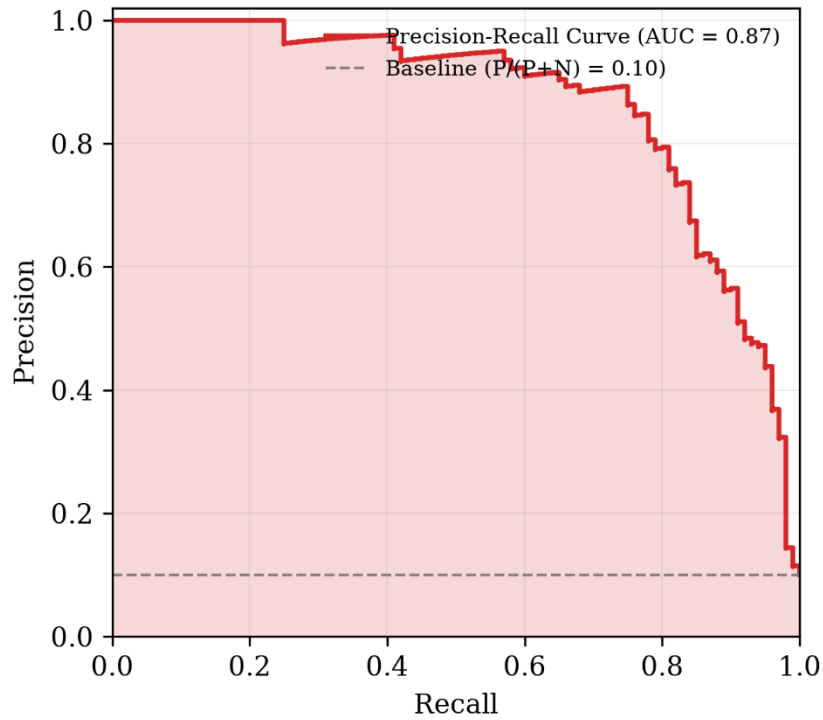
Di mana:

- *TP* merupakan jumlah titik positif yang diprediksi sebagai positif
- *TN* merupakan jumlah titik negatif yang diprediksi sebagai negatif.
- *FN* merupakan jumlah titik positif yang diprediksi sebagai negatif
- *FP* merupakan jumlah titik negatif yang diprediksi sebagai positif

ROC Curve dibentuk dengan menempatkan *FPR* pada sumbu horizontal dan *TPR* pada sumbu vertikal. Setiap nilai *threshold* terhadap skor prediksi menghasilkan pasangan nilai *FPR* dan *TPR* yang berbeda. Dengan demikian, *ROC Curve* menunjukkan kinerja model pada seluruh rentang *threshold*, bukan hanya pada satu *threshold* tertentu.

Kualitas performa model dapat dibaca langsung dari bentuk kurvanya. Pada Gambar 2.4, model yang bekerja dengan baik sudah mencapai true positive rate yang tinggi ketika false positive rate-nya masih rendah, sehingga kurvanya melengkung menjauhi garis diagonal. Model yang performanya buruk justru menghasilkan kurva yang merapat ke garis diagonal dari titik (0, 0) hingga titik (1, 1), yaitu garis dasar ROC yang mewakili classifier acak. Ringkasan numerik dari kurva tersebut adalah Area Under the ROC Curve atau ROC-AUC, yaitu luas area di bawah ROC Curve. Nilai ROC-AUC sebesar 0,5 diperoleh ketika kurva berimpit dengan garis diagonal, sedangkan nilai 1 menunjukkan classifier sempurna. Dengan demikian, ROC-AUC dapat digunakan untuk membandingkan kinerja beberapa model secara keseluruhan pada berbagai threshold (Saito & Rehmsmeier, 2015).

2.2.11 Precision–Recall Curve & AUC-PR



Gambar 2.5. Kurva AUC-PR

Precision–Recall Curve merupakan kurva evaluasi yang menunjukkan hubungan antara *precision* dan *recall* pada berbagai nilai *threshold*. Kurva ini digunakan pada klasifikasi biner yang membagi data menjadi kelas positif dan negatif. Dalam penelitian ini, titik anomali diperlakukan sebagai kelas positif, sedangkan titik normal diperlakukan sebagai kelas negatif.

Precision menunjukkan proporsi prediksi positif yang diklasifikasikan dengan benar. Precision dirumuskan seperti pada Persamaan 2.30

$$Precision = \frac{TP}{TP + FP} \quad (2.30)$$

Sementara itu, *recall* menunjukkan proporsi data positif aktual yang berhasil diklasifikasikan sebagai positif. Recall dirumuskan seperti pada Persamaan 2.31

$$Recall = \frac{TP}{TP + FN} \quad (2.31)$$

Precision–Recall Curve dibentuk dengan menempatkan *recall* pada sumbu horizontal dan *precision* pada sumbu vertikal. Setiap titik pada kurva merepresentasikan pasangan nilai *precision* dan *recall* yang diperoleh dari suatu nilai *threshold* terhadap skor prediksi. Dengan demikian, kurva tersebut memberikan gambaran kinerja model pada berbagai nilai *threshold*, bukan hanya pada satu *threshold* tertentu (Saito & Rehmsmeier, 2015).

Area Under the Precision–Recall Curve atau AUC-PR merupakan luas area di bawah kurva *precision–recall* dan dapat digunakan untuk membandingkan kinerja beberapa model.

Nilai AUC-PR yang lebih besar menunjukkan kurva yang secara umum memiliki precision lebih tinggi pada berbagai tingkat recall. Berbeda dengan ROC *Curve* yang memiliki garis dasar tetap, garis dasar Precision–Recall Curve ditentukan oleh proporsi kelas positif terhadap seluruh data, seperti pada Persamaan 2.32

$$PR\ Baseline = \frac{P}{P + N} \quad (2.32)$$

Dengan:

- P sebagai jumlah data positif
- N sebagai jumlah data negatif.

Oleh karena itu, nilai dasar AUC-PR dipengaruhi oleh distribusi kelas dalam dataset. Pada data seimbang, nilai dasarnya sebesar 0,5, sedangkan pada data tidak seimbang nilainya mengikuti proporsi kelas positif terhadap keseluruhan data (Saito & Rehmsmeier, 2015).

Posisi garis dasar tersebut menentukan cara kurva dibaca. Semakin tidak seimbang data, semakin rendah pula garis dasarnya, sehingga nilai precision yang terlihat rendah secara mutlak dapat tetap bermakna apabila garis dasarnya jauh lebih rendah. Penilaian performa karenanya dilakukan dengan membandingkan kurva terhadap garis dasarnya, bukan terhadap nilai mutlaknya. Selain itu, garis dasar yang rendah juga mengindikasikan tingkat kesulitan dalam membangun model yang baik pada data tersebut (Saito & Rehmsmeier, 2015).

2.2.12 Volume Under the Surface (VUS)

Volume Under the Surface (VUS) merupakan metrik evaluasi untuk deteksi anomali *time series* yang dikembangkan sebagai perluasan dari ROC-AUC dan AUC-PR. ROC-AUC dan AUC-PR mengevaluasi skor anomali pada berbagai nilai *threshold*, tetapi memperlakukan setiap titik waktu secara independen. Pendekatan tersebut belum sepenuhnya mempertimbangkan anomali berbasis rentang yang berlangsung pada beberapa titik waktu secara berurutan (Paparrizos et al., 2022).

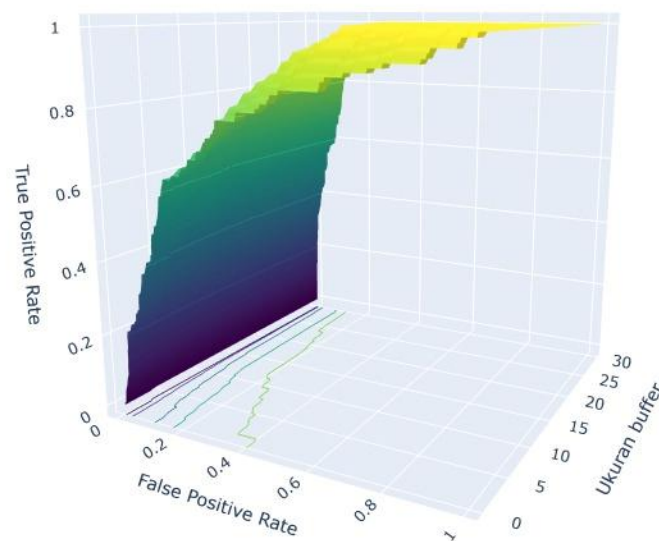
Pada anomali berbasis rentang, dapat terjadi ketidaksesuaian antara rentang anomali yang diberi label dan lokasi skor anomali yang dihasilkan oleh metode deteksi. Ketidaksesuaian tersebut dapat disebabkan oleh perbedaan strategi pelabelan maupun penggunaan *sliding window* yang menyebabkan skor anomali muncul di sekitar batas awal atau akhir rentang anomali. Oleh karena itu Paparrizos et al. (2022) memperkenalkan wilayah *buffer* pada batas rentang anomali untuk memberikan toleransi terhadap pergeseran antara skor anomali dan label aktual.

Wilayah *buffer* mengubah label biner menjadi label kontinu. Titik yang berada di dalam rentang anomali tetap memiliki nilai label sebesar satu, sedangkan titik di wilayah *buffer* memperoleh nilai yang menurun secara bertahap ketika semakin jauh dari batas anomali. Dengan demikian, skor anomali yang berada di sekitar batas rentang anomali masih dapat memperoleh sebagian nilai evaluasi. Label kontinu tersebut selanjutnya digunakan untuk

menghitung nilai *true positive rate*, *false positive rate*, dan *precision* yang telah disesuaikan untuk anomali berbasis rentang (Paparrizos et al., 2022).

Untuk satu ukuran *buffer*, perubahan nilai *threshold* terhadap skor anomali menghasilkan *Range-AUC-ROC* atau *Range-AUC-PR*. Namun, hasil evaluasinya masih bergantung pada ukuran *buffer* yang dipilih. VUS mengatasi ketergantungan tersebut dengan menghitung performa pada berbagai nilai ambang batas dan berbagai ukuran *buffer*. Kumpulan nilai evaluasi tersebut membentuk suatu permukaan, sedangkan volume di bawah permukaan tersebut digunakan sebagai nilai VUS (Paparrizos et al., 2022).

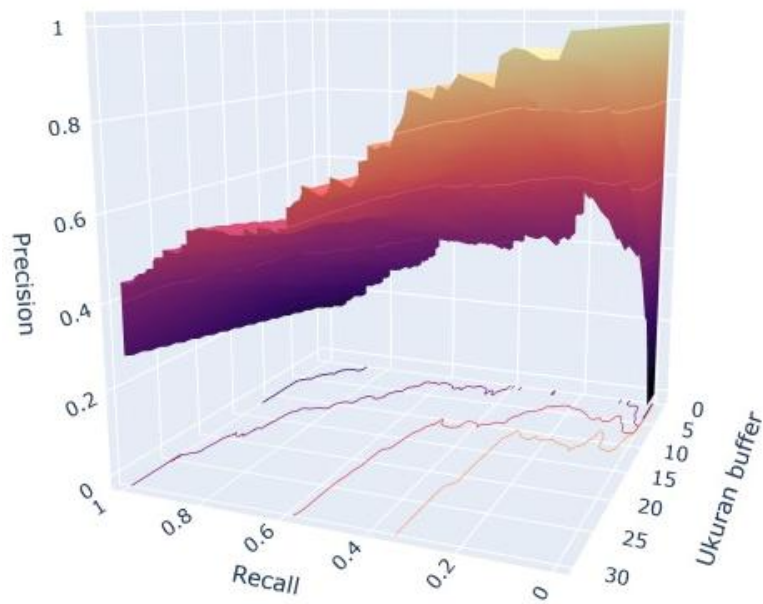
2.2.12.1 VUS-ROC



Gambar 2.6. Kurva VUS-ROC

VUS-ROC merupakan perluasan dari AUC-ROC yang mempertimbangkan berbagai ukuran *buffer*. Pada setiap ukuran *buffer*, nilai *true positive rate* dan *false positive rate* dihitung pada berbagai *threshold* untuk membentuk sebuah kurva ROC. Proses tersebut diulang dengan mengubah ukuran *buffer* dari nol hingga batas maksimum yang ditentukan. Kurva-kurva yang dihasilkan membentuk permukaan dengan dimensi *false positive rate*, *true positive rate*, dan ukuran *buffer*. Volume di bawah permukaan tersebut disebut VUS-ROC (Paparrizos et al., 2022).

2.2.12.2 VUS-PR



Gambar 2.7. Kurva VUS-PR

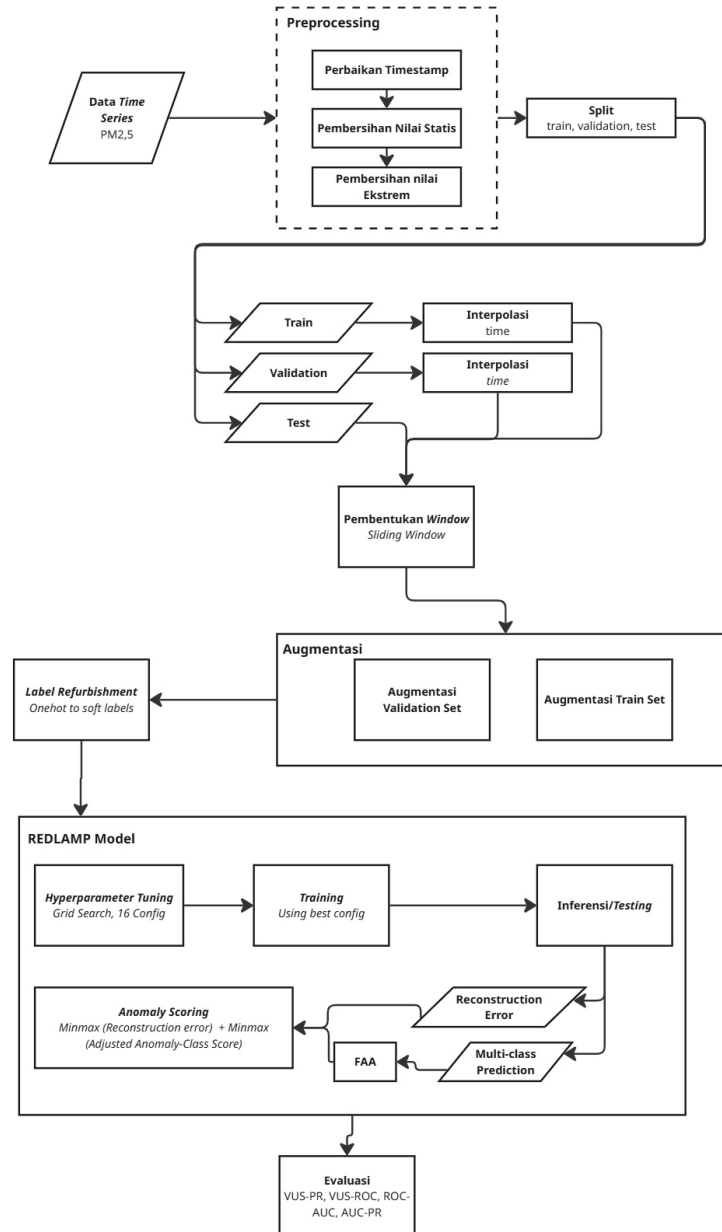
VUS-PR merupakan perluasan dari AUC-PR yang menghitung hubungan antara *precision* dan *recall* pada berbagai nilai ambang batas serta ukuran *buffer*. Pada setiap ukuran *buffer*, pasangan nilai *precision* dan *recall* membentuk sebuah *Precision–Recall Curve*. Pengulangan perhitungan pada berbagai ukuran *buffer* menghasilkan permukaan dengan dimensi *recall*, *precision*, dan ukuran *buffer*. Volume di bawah permukaan tersebut disebut VUS-PR (Paparrizos et al., 2022).

Karena dibangun berdasarkan *precision* dan *recall*, VUS-PR tetap mempertimbangkan proporsi prediksi anomali yang benar dan proporsi anomali aktual yang berhasil dideteksi. Pada saat yang sama, penggunaan berbagai ukuran *buffer* memungkinkan evaluasi mempertimbangkan anomali berbasis rentang dan ketidaksesuaian kecil antara lokasi skor anomali dengan label.

Halaman ini sengaja dikosongkan.

BAB 3 METODOLOGI

3.1 Alur Metodologi Penelitian



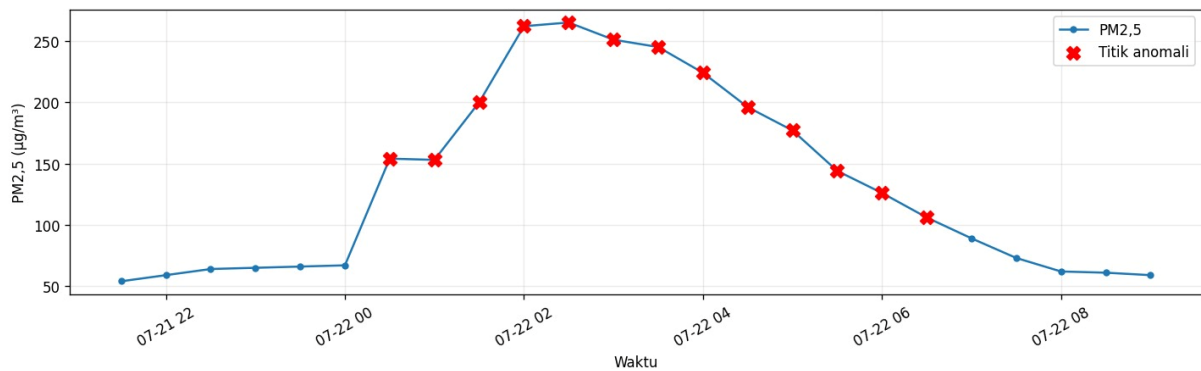
Gambar 3.1. Alur Metodologi

Secara garis besar, alur metodologi pada Gambar 3.1 dimulai dari data PM2.5 yang berupa deret waktu, dilanjutkan dengan preprocessing untuk membersihkan dan menyiapkan data, pembentukan window menggunakan sliding window, augmentasi untuk membentuk kelas pseudo-anomali, training model REDLAMP, hingga evaluasi performa deteksi.

3.1.1 Dataset

3.1.1.1 Informasi Umum Dataset

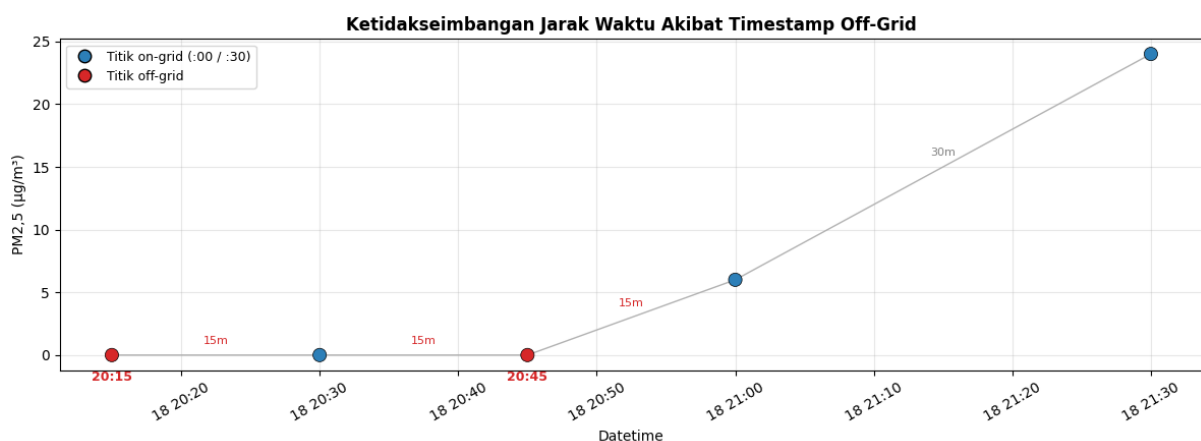
Dataset memiliki jumlah baris sebanyak 50.972. *Datetime minimum* yang ada pada dataset ini berada pada tanggal 1 Januari 2021, Pukul 00:00:00. Sementara *Datetime maximum* berada pada 31 Desember 2023, Pukul 23:30:00. Label anomali diambil dari data-data yang ditandai invalid oleh sistem. Label Normal yang ada pada dataset ini adalah sebanyak 50049 dan Label Anomali sebanyak 923. Contoh dari anomali yang ada pada dataset dapat dilihat pada Gambar 3.2.



Gambar 3.2. Contoh Anomali pada Dataset

3.1.1.2 Pengecekan Keteraturan *Timestamp*

Untuk memastikan bahwa seluruh data diukur pada interval yang sama, dilakukan pengecekan interval *timestamp*. Pengecekan dilakukan dengan menerapkan aturan bahwa data diukur pada interval 30 menit, dan dimulai dari pukul 00:00:00 pada sebuah tanggal. Maka, baris-baris yang *timestamp*-nya tidak sesuai menandakan bahwa baris tersebut invalid. Hasil pengecekan menandakan ada dua baris yang memiliki *timestamp* yang menyimpang, yaitu pada datetime “2022-08-18 20:15:00” dan “2022-08-18 20:45:00”. Baris-baris tersebut menyebabkan inkonsistensi interval antar titik, seperti yang dapat dilihat pada Gambar 3.3.

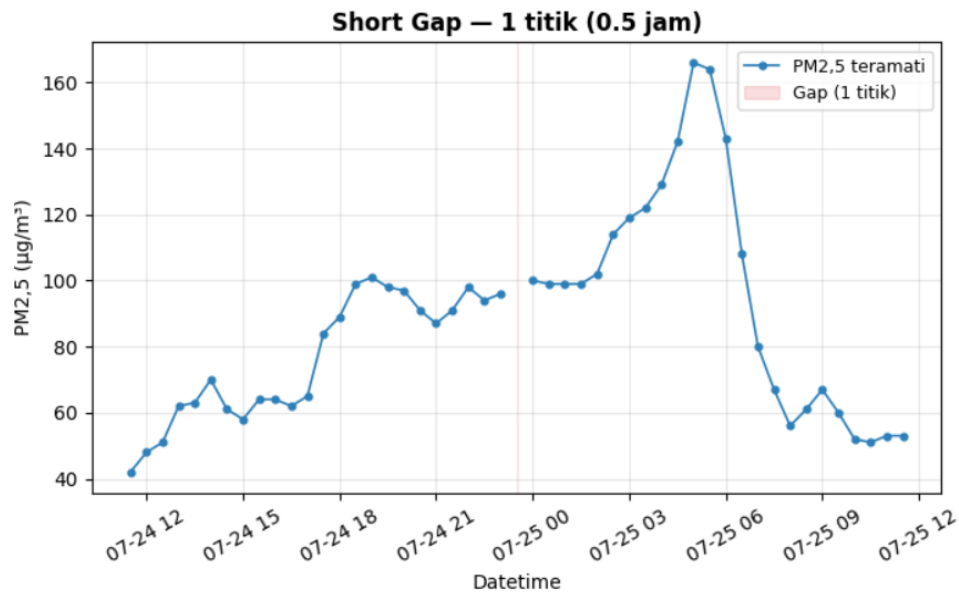


Gambar 3.3. Inkonsistensi Interval akibat Off-Grid Timestamp

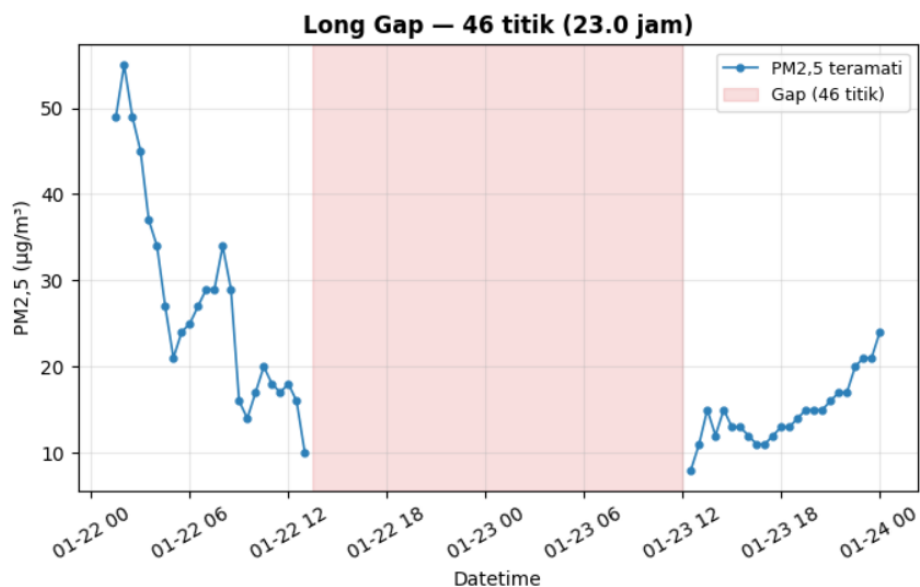
3.1.1.3 Pemeriksaan Missing Values

Berhubung data yang digunakan merupakan hasil pengukuran pada interval 30 menit dari tahun 2021-2023, seharusnya terdapat baris data sebanyak 52560. Namun, pada data asli, baris data hanya tersedia sebanyak 50.972. Hal ini menandakan bahwa adanya baris data yang tidak tercatat. Oleh karena itu, meskipun pada data asli tidak ada nilai PM2.5 yang *missing*,

tetap dilakukan pemeriksaan *Missing Values* pada data yang telah di-*reindex* ke interval 30 menit. Pemeriksaan *Missing Values* dilakukan dengan mencari jumlah nilai PM2.5 yang *missing* pada data-data yang muncul setelah *reindex*. Hal-hal yang dicek antara lain adalah jumlah titik yang *missing*, jumlah entitas *Missing Values* dengan panjang > 12 titik (*Long Gap*), dan juga jumlah entitas *Missing Values* dengan panjang ≤ 12 titik (*Short Gap*). Hasil pengecekan menunjukkan adanya *Missing Values* sebanyak 1590 baris, yang membentuk 30 *Long Gap* dan 85 *Short Gap*. Contoh dari *Short Gap* dapat dilihat pada Gambar 3.4 dan contoh *Long Gap* dapat dilihat pada Gambar 3.5.



Gambar 3.4. Contoh Short Gap



Gambar 3.5. Contoh Long Gap

3.1.1.4 Descriptive Statistics dari PM2.5

Statistik deskriptif dihitung terhadap nilai konsentrasi PM2.5 pada data asli (sebelum *reindex*). Berdasarkan hasil perhitungan, nilai rata-rata konsentrasi PM2.5 adalah $38,76 \mu\text{g}/\text{m}^3$, sedangkan nilai minimum yang ditemukan adalah $0,00 \mu\text{g}/\text{m}^3$. Median bernilai $29,00 \mu\text{g}/\text{m}^3$

dan Persentil ke-99 bernilai $139,00 \mu\text{g}/\text{m}^3$. Namun, nilai maksimum mencapai $67.601 \mu\text{g}/\text{m}^3$. Sementara itu, standar deviasi mencapai $323,90 \mu\text{g}/\text{m}^3$.

3.1.1.5 Pemeriksaan Nilai PM2.5 Ekstrem

Perbedaan yang cukup signifikan antara nilai persentil ke-99 dan nilai maksimum mengindikasikan bahwa terdapat nilai PM2.5 yang menyimpang jauh dari nilai-nilai lainnya (ekstrem). Pengecekan nilai ekstrem dilakukan dengan cara mengurutkan nilai-nilai PM2.5 dari yang terbesar sampai yang terkecil. Pada hasil pengecekan, didapati 10 nilai terbesar. Pada 10 nilai terbesar, terdapat selisih yang signifikan antara peringkat ke-7 ($394 \mu\text{g}/\text{m}^3$) dan juga peringkat ke-6 ($640 \mu\text{g}/\text{m}^3$) dan seterusnya. Hal ini dapat menandakan bahwa di antara nilai-nilai yang tinggi, terdapat beberapa nilai yang sangat ekstrem dikarenakan sangat jauh dari data mayoritas. Detail mengenai sepuluh nilai PM2.5 tertinggi dapat dilihat pada Tabel 3.1.

Tabel 3.1. 10 Nilai PM2.5 Teratas

Datetime	PM2.5	Label
2021-12-08 15:30:00	67601	1
2021-12-08 15:00:00	22284	1
2021-12-08 16:00:00	11001	1
2021-12-08 16:30:00	10425	1
2021-12-08 14:00:00	4309	1
2021-12-08 14:30:00	640	1
2021-11-06 23:00:00	394	1
2023-05-02 06:00:00	347	1
2023-01-29 01:30:00	342	1
2023-05-02 06:30:00	331	1

3.1.1.6 Pemeriksaan Rangkaian Nilai Nol

Pada data asli, didapati beberapa rangkaian nilai PM2.5 sebesar 0 secara berturut-turut. Oleh karena itu, dilakukan pengecekan nilai nol yang terjadi. Ditemukan 57 titik yang termasuk ke dalam rangkaian nilai nol. Titik-titik tersebut membentuk 7 entitas rangkaian nilai nol (dapat dilihat pada Tabel 3.2).

Tabel 3.2. Entitas Rangkaian Nol

Waktu Mulai	Waktu Selesai	Jumlah Titik	Nilai PM2.5 $\mu\text{g}/\text{m}^3$
2 Desember 2022 17.00	2 Desember 2022 19.30	6	0,0
21 September 2023 19.00	21 September 2023 19.30	2	0,0
25 September 2023 02.30	25 September 2023 03.00	2	0,0
27 September 2023 06.30	27 September 2023 07.00	2	0,0
29 September 2023 21.30	29 September 2023 22.00	2	0,0
30 September 2023 07.00	30 September 2023 08.00	3	0,0
30 September 2023 14.00	1 Oktober 2023 09.30	40	0,0

3.1.1.7 Pemeriksaan Rangkaian Nilai Statis

Selain dari rangkaian nilai nol, didapati juga rangkaian nilai statis. Rangkaian ini terdiri dari nilai yang bukan nol, namun berturut-turut sama. Jumlah titik-titik yang termasuk ke dalam rangkaian nilai statis adalah sebanyak 1347 titik, yang kemudian membentuk 219 entitas rangkaian nilai statis. Contoh-contoh dari entitas rangkaian nilai statis dapat dilihat pada Tabel 3.3.

Tabel 3.3. Entitas Rangkaian Nilai Statis

Waktu Mulai	Waktu Selesai	Jumlah Titik	Nilai PM2.5 $\mu\text{g}/\text{m}^3$
6 Januari 2021 22.30	7 Januari 2021 00.00	4	20,0
7 Januari 2021 06.30	7 Januari 2021 08.30	5	5,0
7 Januari 2021 14.00	7 Januari 2021 16.00	5	5,0
10 Januari 2021 02.30	10 Januari 2021 04.00	4	7,0
10 Januari 2021 10.00	10 Januari 2021 12.00	5	4,0
12 Januari 2021 01.00	12 Januari 2021 03.30	6	4,0
13 Januari 2021 11.00	13 Januari 2021 14.30	8	5,0
18 Januari 2021 12.00	18 Januari 2021 13.30	4	8,0
19 Januari 2021 04.00	19 Januari 2021 05.30	4	11,0
27 Januari 2021 04.00	27 Januari 2021 05.30	4	4,0

3.1.2 Preprocessing

Berdasarkan hasil temuan pada tahap *Exploratory Data Analysis* (EDA), dilakukan *preprocessing* agar data sesuai dengan kebutuhan penelitian. Tahapan ini mencakup, perbaikan interval waktu, penanganan *missing values*, nilai ekstrem, rangkaian nilai nol, dan rangkaian nilai statis. Selain itu, dilakukan pembagian data secara temporal serta normalisasi nilai PM2.5.

3.1.2.1 Perbaikan interval waktu

Ditemukan beberapa *timestamp* yang tidak sesuai dengan interval pencatatan 30 menit, yaitu *timestamp* yang tidak berada pada menit ke-00 atau ke-30 setiap jam (karena pengukuran dimulai pada pukul 00:00:00). Baris dengan *timestamp* tersebut di-drop. Setelah itu, data diurutkan berdasarkan waktu dan dilakukan *reindexing* menggunakan interval 30 menit untuk membentuk urutan *timestamp* yang lengkap. Proses *reindexing* ini menghasilkan baris baru dengan nilai kosong pada waktu-waktu yang sebelumnya tidak tercatat/terdapat pada data.

3.1.2.2 Penanganan Nilai Statis

Pada dataset asli, didapati nilai statis yang terdiri atas rangkaian nilai nol dan rangkaian nilai konstan non-nol. Kedua jenis rangkaian tersebut ditangani menggunakan aturan yang berbeda. Apabila nilai nol muncul dua titik atau lebih secara berurutan, seluruh titik dalam rangkaian tersebut dianggap sebagai *missing values* yang memang tercatat dan diubah menjadi NaN seluruhnya. Sementara itu, pada rangkaian nilai konstan non-nol, ditetapkan batas maksimum sebanyak empat titik berturut-turut. Jika panjang rangkaian melebihi batas tersebut, empat titik pertama tetap dipertahankan, sedangkan titik-titik setelahnya diubah menjadi NaN.

3.1.2.3 Penanganan Nilai Ekstrem

Di antara nilai-nilai PM2.5 yang tertinggi, didapati nilai-nilai yang perbedaannya sangat signifikan, selisihnya ratusan sampai ribuan. Karena nilai-nilai tersebut berada pada *train set* dan berpotensi mengganggu proses pelatihan model, ditetapkan batas maksimum sebesar $400 \mu\text{g}/\text{m}^3$. Nilai PM2.5 pada *train set* yang melebihi batas tersebut diubah menjadi NaN.

3.1.2.4 Train, Validation, dan Test Split

Data PM2.5 yang digunakan memiliki rentang waktu dari 2021-2023 atau sekitar 36 bulan. Sekitar 60% dari data (± 22 bulan) dibagi ke dalam *train set*, *validation set* 20% (7 bulan), dan *test* 20% (7 bulan). Split dilakukan secara urut agar tidak merusak pola temporal.

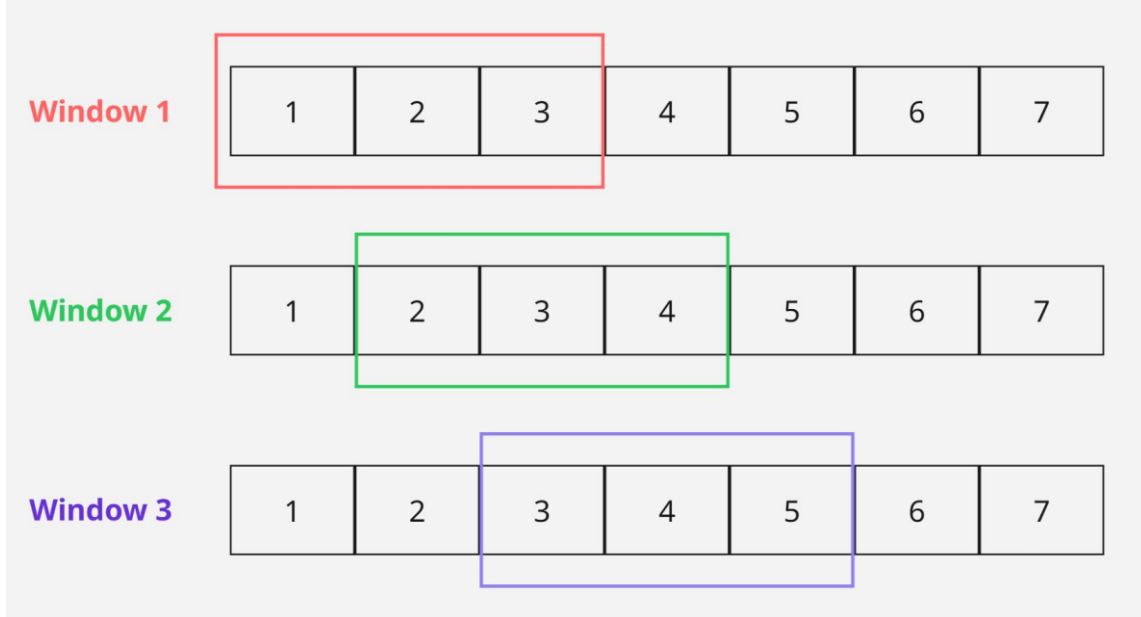
3.1.2.5 Penanganan Missing Values

Reindexing menghasilkan beberapa baris data baru dengan nilai PM2.5 NaN. Hal ini diatasi dengan interpolasi berbasis waktu menggunakan metode “*time*”. Interpolasi dilakukan pada *train set* dan *validation set*. Untuk memastikan bahwa data hasil interpolasi tidak membuat pola baru yang bersifat terlalu sintesis, interpolasi hanya diterapkan pada rangkaian *missing values* dengan panjang maksimal 12 titik berturut-turut. Untuk missing value yang panjangnya melebihi 12 titik, akan dibiarkan sebagai NaN. Selain itu, interpolasi tidak dilakukan pada data NaN yang dihasilkan oleh penanganan nilai statis dan nilai ekstrem.

3.1.2.6 Min-max Normalization

Setelah data dibagi menjadi *train set*, *validation set*, dan *test set*, dilakukan normalisasi menggunakan metode *Min-Max Normalization*. Normalisasi bertujuan menyetarakan skala nilai PM2.5 agar sesuai dengan proses augmentasi dan pelatihan model REDLAMP. Nilai minimum dan maksimum dihitung hanya dari *train set*, kemudian digunakan untuk mentransformasikan seluruh subset data. Dengan proses tersebut, nilai pada *train set* dipetakan ke dalam rentang 0 hingga 1. Penggunaan parameter yang hanya berasal dari *train set* dilakukan untuk mencegah terjadinya *data leakage* dari *validation set* dan *test set*.

3.1.3 Pembentukan Window (Sliding Window)



Gambar 3.6. Sliding Window (Benson et al., 2020)

Model REDLAMP menerima masukan dalam bentuk *window* dengan panjang L . Untuk mempertahankan kontinuitas temporal antar-*window*, dilakukan pembentukan *window* menggunakan *stride* sebesar satu sehingga setiap *window* yang berdekatan saling tumpang tindih (seperti pada Gambar 3.6). Proses *sliding window* diterapkan secara terpisah pada data *train*, *validation*, dan *test* untuk mencegah terbentuknya *window* yang melintasi batas antarsubset. Integritas setiap *window* dijaga dengan mengecualikan *window* yang mengandung nilai NaN atau memiliki proporsi titik hasil interpolasi lebih dari 10% dari total panjang *window*.

3.1.4 Augmentasi Data

Augmentasi data diterapkan pada *window* yang berasal dari *train* dan *validation set* untuk membentuk data *multiclass*. Setiap *window* pada *train* dan *validation set* diaugmentasi untuk membentuk 1 kelas normal + 11 kelas pseudo-anomali (dapat dilihat pada subbab 2.2.6). Sebagaimana dilakukan oleh Fung et al. (2024) yang membentuk *validation set* dari data normal dan anomali sintetis untuk pemilihan model tanpa memanfaatkan label anomali asli, *validation set* pada penelitian ini turut diaugmentasi sehingga dapat digunakan untuk memantau *validation loss* karena *validation loss* REDLAMP hanya terdefinisi pada instance teraugmentasi. Pendekatan ini memungkinkan model mempelajari berbagai bentuk anomali, tidak hanya membedakan data normal dan anomali secara biner.

Untuk setiap *window*, dipilih suatu segmen berdasarkan indeks awal **st** dan indeks akhir **ed**. Fungsi augmentasi kemudian diterapkan pada segmen tersebut sesuai dengan jenis pseudo-anomali yang dibentuk. Karena data yang digunakan bersifat univariat, proses augmentasi hanya dilakukan pada satu dimensi (PM2.5). Dihasilkan tiga komponen dari masing-masing augmentasi, yaitu *window* hasil modifikasi, label penanda kelas augmentasi (*onehot*), dan *anomaly mask* yang menandai posisi titik yang mengalami perubahan.

Secara umum, setiap fungsi augmentasi menerima beberapa parameter utama. Parameter x menyatakan *window* yang akan dimodifikasi, **dim** menyatakan indeks dimensi data, **st** dan **ed** menyatakan batas awal dan akhir segmen augmentasi, **L** menyatakan panjang *window*,

sedangkan **X_split** digunakan sebagai sumber segmen tambahan pada augmentasi *mixture*. Detail parameter pada masing-masing fungsi pseudo-anomali dapat dilihat pada tabel berikut:

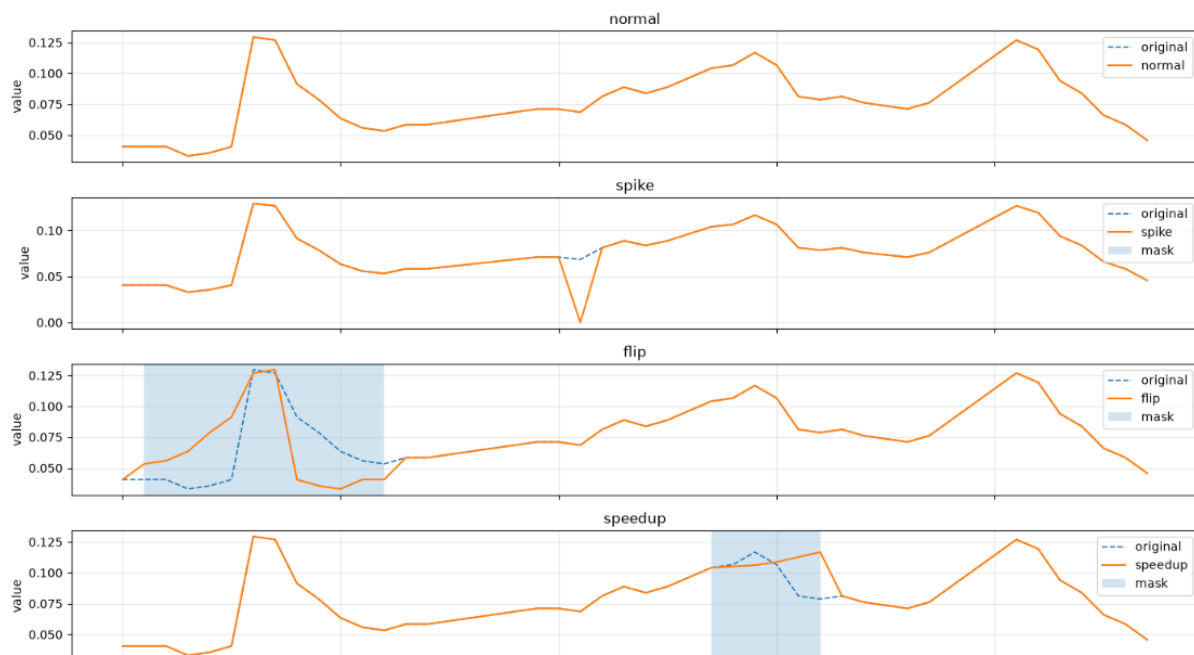
Tabel 3.4. Parameter Augmentasi Pseudo-Class

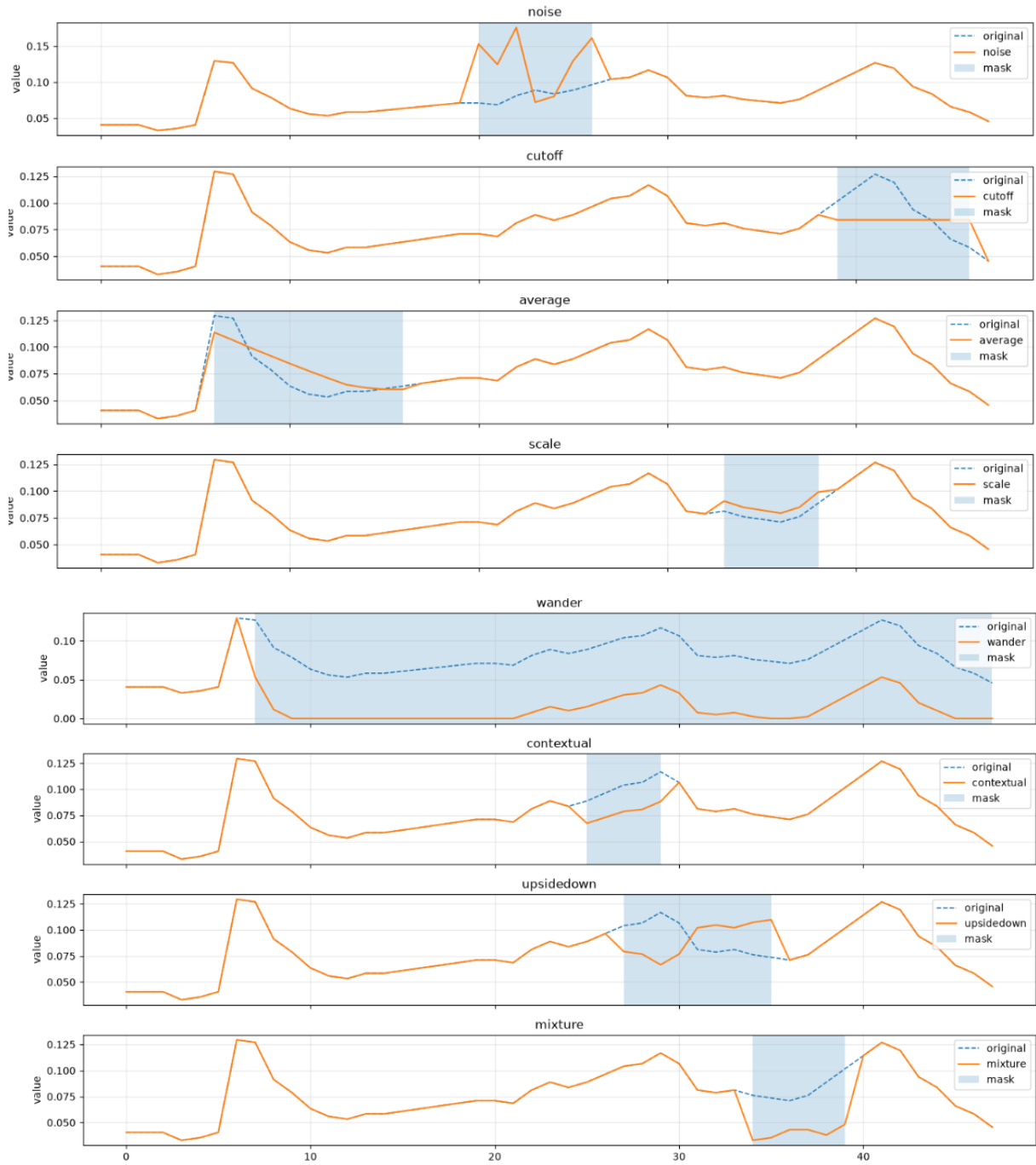
Jenis augmentasi	Parameter	Mekanisme
Normal	Tanpa modifikasi	Window asli dipertahankan sebagai kelas normal.
Spike	$a \sim \mathcal{N}(0, 0.3)$	Nilai (a) ditambahkan pada satu titik.
Noise	$\epsilon_t \sim \mathcal{N}(0, 0.1)$	Noise Gaussian ditambahkan pada seluruh titik dalam segmen.
Cutoff	$c \sim \mathcal{U}(\min(S), \max(S))$	Seluruh nilai pada segmen (S) diganti dengan satu nilai konstan acak.
Average	$w = \text{round}(L/5)$	Segmen dihaluskan menggunakan <i>moving average</i> . Nilai w ditetapkan minimal 2 dan maksimal sebesar panjang segmen.
Scale	$a \sim \mathcal{N}(1, 0.2)$	Nilai pada segmen dikalikan dengan faktor skala (a).
Flip	Tanpa parameter tambahan	Urutan nilai pada segmen dibalik secara temporal.
Speedup	Mode 2x atau half	Pola temporal segmen diubah melalui kompresi atau pemanjangan berbasis interpolasi.
Wander	$a = sr,$ $r \sim \mathcal{U}(0.06, 0.12),$ $s \in \{-1, 1\}$	Tren linear ditambahkan pada segmen dan diikuti pergeseran nilai setelah titik awal augmentasi.
Contextual	$a \sim \mathcal{N}(1, 0.2^2),$ $b \sim \mathcal{N}(0, 0.1^2)$	Segmen ditransformasikan menggunakan Persamaan (ax+b).
Upside-down	$m = \text{mean}(S)$	Nilai pada segmen dibalik terhadap rata-rata menggunakan transformasi (2m-S).
Mixture	$t' \sim \mathcal{U}\{1, 2, \dots, N_{split}\}$	Segmen diganti menggunakan segmen dari window lain pada data latih yang dipilih secara acak.

Input : Himpunan window (data latih/validasi), daftar kelas augmentasi, rentang rasio panjang segmen, dan seed
Output: Himpunan window teraugmentasi, label kelas (one-hot), dan anomaly mask
1. Terima himpunan window beserta daftar kelas augmentasi dan parameter augmentasi. 2. Tetapkan seed acak untuk menjamin reproducibility. 3. Ulangi proses untuk setiap window pada himpunan masukan. 4. Ulangi proses untuk setiap kelas augmentasi yang dipilih. 5. Salin window asli, inisialisasi anomaly mask bernilai nol, dan bentuk label one-hot sesuai kelas. 6. Salin window asli, inisialisasi anomaly mask bernilai nol, dan bentuk label one-hot sesuai kelas. 7. Simpan window tanpa modifikasi bila kelas adalah normal, kemudian lanjut ke kelas berikutnya. 8. Pilih dimensi yang akan dimodifikasi untuk kelas pseudo-anomali. 9. Tentukan indeks awal dan akhir segmen augmentasi berdasarkan rasio panjang terhadap window (khusus spike, dipilih satu titik). 10. Terapkan fungsi augmentasi sesuai kelas pada segmen terpilih. 11. Tandai posisi perubahan pada anomaly mask, lalu simpan window hasil augmentasi, label, dan anomaly mask. 12. Tandai posisi perubahan pada anomaly mask, lalu simpan window hasil augmentasi, label, dan anomaly mask. 13. Kembalikan seluruh window teraugmentasi beserta label dan anomaly mask. 14. Selesai.

Kode Semu 3.1. Algoritma Augmentasi

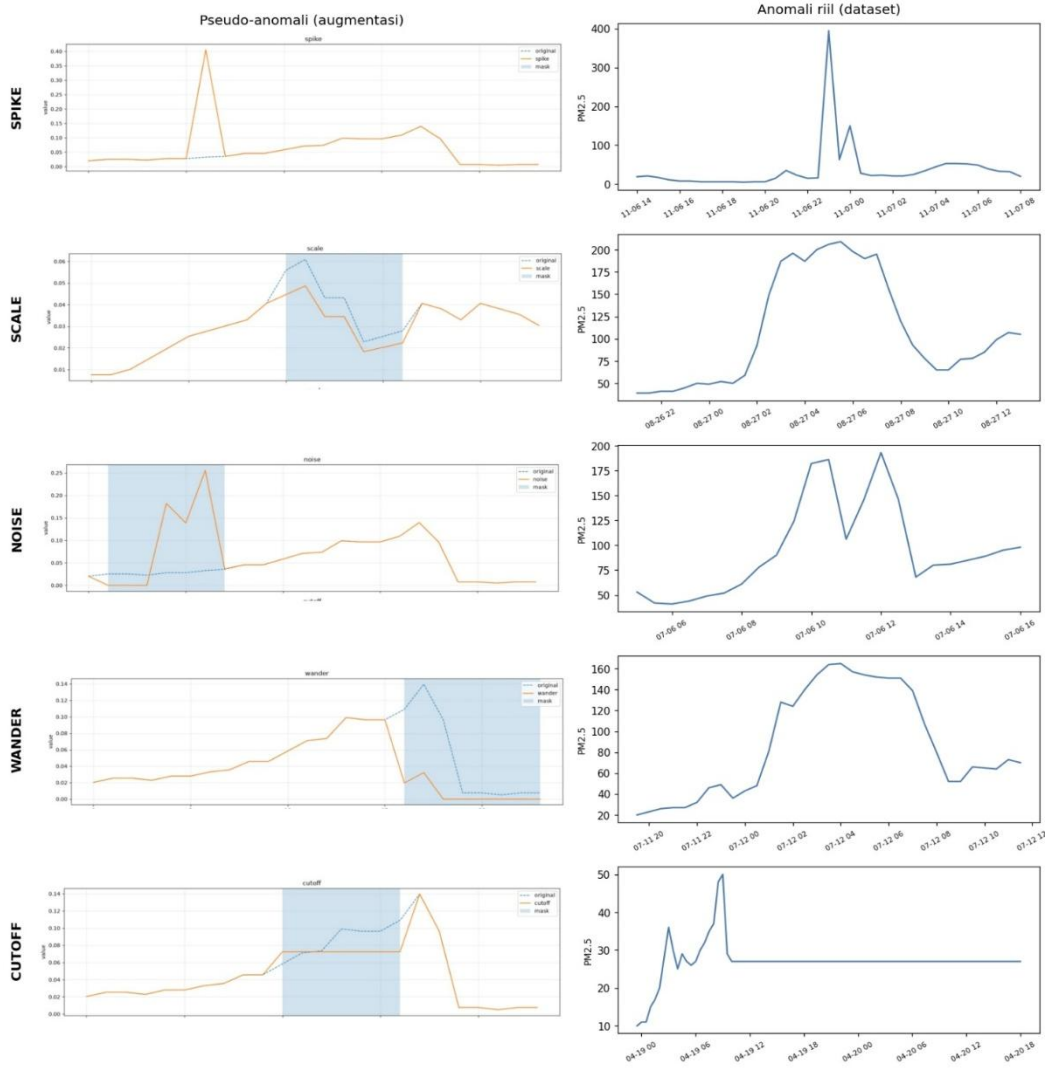
Pada setiap window, proses augmentasi dilakukan untuk seluruh kelas yang digunakan dalam skenario eksperimen. Untuk setiap kelas, fungsi augmentasi diterapkan pada segmen yang dipilih, kemudian dihasilkan window hasil modifikasi, label kelas, serta *anomaly mask* yang menandai lokasi perubahan. Alur pembentukan ketiga komponen tersebut ditunjukkan pada *pseudo-code* augmentasi REDLAMP pada Kode Semu 3.1. Untuk memastikan *reproducibility*, ditetapkan *seed* 42 untuk augmentasi *train set* dan *seed* 43 untuk augmentasi *validation set*.





Gambar 3.7. Contoh Hasil Augmentasi sebuah Window

Untuk memberikan gambaran terhadap hasil augmentasi, Gambar 3.7 menunjukkan contoh hasil augmentasi pada satu *window* normal (*sample* index 1000) yang diproses melalui sebelas fungsi pseudo-anomali seperti pada Tabel 3.4. Pada setiap subplot, garis putus-putus biru menunjukkan nilai *window* asli sebelum augmentasi, garis oranye menunjukkan nilai *window* setelah augmentasi diterapkan, dan area berwarna biru muda menandai *anomaly mask*.



Gambar 3.8. Contoh Perbandingan Hasil Augmentasi dengan Anomali pada Dataset

Gambar 3.8 menampilkan contoh bentuk beberapa jenis pseudo-anomali hasil augmentasi berdampingan dengan contoh segmen data PM2.5 yang menunjukkan pola serupa. Kolom kiri memuat *window* hasil augmentasi untuk kelas *spike*, *scale*, *noise*, *wander*, dan *cutoff*, dengan garis putus-putus biru sebagai *window* asli, garis oranye sebagai hasil augmentasi, dan area biru muda sebagai *anomaly mask*. Kolom kanan memuat contoh segmen dari dataset dengan bentuk penyimpangan yang menyerupai masing-masing jenis tersebut.

3.1.5 Label refurbishment

Setelah proses augmentasi, setiap *window* dan variasinya memiliki *one-hot label* yang menunjukkan kelas normal atau jenis augmentasi yang diterapkan. Untuk mencegah model yang *overconfident* terhadap label tersebut, dilakukan *label refurbishment* dengan mengonversi *hard label* menjadi *soft label*. Proses ini dilakukan secara *element-wise* pada setiap elemen vektor label (dapat dilihat pada Persamaan 2.18 dan 2.19). Mula-mula, elemen pada kelas target dikalikan dengan faktor $(1 - \alpha - K\beta)$. Selanjutnya, seluruh elemen kelas diberi nilai dasar sebesar β , yang pada penelitian ini ditetapkan sebesar 0,01. Khusus pada elemen kelas normal, ditambahkan juga nilai α sebesar 0,1. Dengan mekanisme tersebut, kelas target tetap memiliki probabilitas terbesar, sedangkan sebagian probabilitas didistribusikan ke kelas normal dan kelas lainnya. Jumlah seluruh elemen pada setiap *soft label* tetap bernilai 1.

3.1.6 Training

Setelah proses augmentasi dan *label refurbishment*, model REDLAMP dilatih menggunakan *train set* yang telah dibentuk menjadi *batch*. Setiap *batch* terdiri atas *window* hasil augmentasi, *soft label*, dan *anomaly mask*. *Window* hasil augmentasi diproses oleh *encoder* untuk menghasilkan representasi laten. Representasi tersebut selanjutnya digunakan oleh *decoder* untuk merekonstruksi *input window*. Selain itu, *Classifier* memprediksi kelas dari *input window*.

Proses *training* menggunakan dua *loss function*, yaitu *masked reconstruction loss* sebagaimana dirumuskan pada Persamaan 2.20 dan *cross-entropy loss* pada Persamaan 2.21. *Masked reconstruction loss* menghitung kesalahan rekonstruksi hanya pada bagian *window* yang tidak ditandai sebagai pseudo-anomali oleh *anomaly mask*. Sementara itu, *cross-entropy loss* mengukur perbedaan antara probabilitas kelas yang dihasilkan *classifier* dan *soft label* hasil *label refurbishment*. Kedua komponen tersebut digabungkan menggunakan parameter pembobot γ untuk menghasilkan *total loss*. Nilai *total loss* kemudian digunakan dalam proses *backpropagation* dan pembaruan parameter model.

Pada setiap epoch, model terlebih dahulu di-*train* menggunakan *train set*. Setelah seluruh *batch train set* diproses, model dievaluasi menggunakan *validation set* yang telah melalui proses augmentasi dan *label refurbishment* yang sama. Pada tahap *validation*, parameter model tidak diperbarui. Nilai *validation loss* digunakan untuk memantau kemampuan generalisasi model dan menentukan bobot model terbaik. Bobot dengan nilai *validation loss* terendah disimpan untuk digunakan pada tahap inferensi.

3.1.6.1 Hyperparameter Tuning

Sebelum proses *train* akhir dilakukan, diterapkan proses *hyperparameter tuning* untuk memperoleh konfigurasi model yang paling sesuai dengan data penelitian. *Tuning* dilakukan menggunakan metode *grid search*, yaitu dengan melatih dan mengevaluasi model pada seluruh kombinasi nilai hyperparameter yang telah ditentukan. Parameter yang divariasikan meliputi *batch size*, dimensi laten, konfigurasi *convolution channels*, dan *learning rate*. Sementara itu, parameter lain seperti *dropout*, γ , *weight decay*, α , dan β dipertahankan tetap selama proses pencarian.

Setiap kombinasi hyperparameter dievaluasi berdasarkan nilai *validation loss* terendah. Proses *tuning* menggunakan jumlah *epoch* yang lebih sedikit dibandingkan pelatihan akhir untuk mengurangi beban komputasi, yaitu 15 *epoch*. Selain itu, diterapkan *early stopping* agar pelatihan suatu konfigurasi dihentikan apabila *validation loss* tidak mengalami perbaikan dalam sejumlah epoch berturut-turut. *Early stopping* dilakukan dengan *patience* sebesar 4 *epoch*. Konfigurasi dengan nilai *validation loss* terbaik dipilih sebagai konfigurasi akhir model.

Tabel ruang pencarian hyperparameter dapat dituliskan sebagai berikut:

Tabel 3.5. Ruang Pencarian Hyperparameter Tuning

Hyperparameter	Nilai yang diuji
<i>Batch size</i>	64, 128
Dimensi laten	32, 64
Kanal konvolusi	([64,128,128,128]), ([128,128,256,256])
<i>Learning rate</i>	1×10^{-3} , 5×10^{-4}
<i>Dropout</i>	0,2
γ	0,1
<i>Weight decay</i>	1×10^{-5}
α	0,1
β	0,01
Maksimum epoch tuning	15
<i>Patience</i>	4
<i>Minimum improvement</i>	1×10^{-5}
<i>Random seed</i>	42

Berdasarkan ruang pencarian pada Tabel 3.5, terdapat 16 konfigurasi dari hasil kombinasi parameter-parameter yang ada.

3.1.6.2 Train With Best Configuration

Setelah konfigurasi terbaik diperoleh, model dilatih kembali menggunakan konfigurasi tersebut pada *train set* dan divalidasi menggunakan *validation set*. Proses pembaruan parameter dilakukan menggunakan *optimizer* AdamW. Pelatihan dilakukan hingga jumlah *epoch* maksimum yang ditetapkan atau dihentikan lebih awal apabila *validation loss* tidak mengalami perbaikan selama sejumlah *epoch* sesuai nilai *patience*. Pada fase *training* menggunakan konfigurasi terbaik, *max epoch* ditetapkan pada 100 *epoch* dengan *patience* 10 *epoch*. Bobot model dengan nilai *validation loss* terendah disimpan sebagai *best model*. Model tersebut kemudian digunakan pada tahap inferensi terhadap *test set*. Untuk menjaga *reproducibility* eksperimen, digunakan nilai *random seed* 42.

3.1.7 Inferensi

Setelah fase *train* selesai, model terbaik digunakan untuk melakukan inferensi pada *window* yang berasal dari *test set*. Berbeda dengan *train* dan *validation set*, *window* pada *test set* tidak melalui proses augmentasi maupun *label refurbishment*. Setiap *test window* diproses oleh encoder untuk menghasilkan representasi laten. Representasi tersebut kemudian diteruskan ke decoder untuk menghasilkan rekonstruksi *window* dan ke classifier untuk memperoleh probabilitas kelas normal serta kelas-kelas pseudo-anomali.

Dari *output decoder*, dihitung *reconstruction error* yang dihitung sebagai perbedaan antara *input window* dan hasil rekonstruksinya. Nilai *error* yang lebih tinggi menunjukkan bahwa pola pada *window* semakin sulit direkonstruksi berdasarkan representasi yang dipelajari model. Sementara itu, *output* dari *classifier* dikonversi menjadi probabilitas kelas menggunakan fungsi *softmax*. Probabilitas kelas pseudo-anomali tersebut digunakan untuk membentuk komponen skor anomali berbasis klasifikasi.

Sebelum skor klasifikasi dihitung, diterapkan *Frequent Anomaly Adjustment* (FAA) sebagaimana dirumuskan pada Persamaan 2.24 dan 2.25. Mekanisme ini digunakan karena

beberapa kelas pseudo-anomali dapat sering memperoleh probabilitas tinggi pada data uji meskipun pola tersebut tidak selalu merepresentasikan penyimpangan yang bermakna. Untuk setiap kelas pseudo-anomali, dihitung rata-rata probabilitas prediksi pada seluruh *test window*. Kelas dengan rata-rata probabilitas yang melebihi *threshold* (δ) diperlakukan sebagai *pseudo-normal class*. Kontribusi kelas tersebut kemudian dihilangkan dari perhitungan *adjusted anomaly-class score*.

Dengan demikian, *adjusted anomaly-class score* diperoleh dengan menjumlahkan probabilitas kelas pseudo-anomali yang tidak termasuk dalam kelompok *pseudo-normal*. Kelas normal tidak disertakan dalam penjumlahan tersebut. Pada penelitian ini, nilai *threshold* (δ) ditetapkan sebagai nilai $1/k$, di mana k adalah jumlah kelas pseudo-anomali yang digunakan pada augmentasi (termasuk kelas normal). Penerapan nilai *threshold* sebesar $1/k$ bertujuan agar kelas pseudo-anomali yang kontribusinya dihapus hanyalah kelas-kelas yang rata-rata probabilitasnya melebihi distribusi kelas, sehingga *threshold* (δ) tidak terlalu agresif.

Selanjutnya, *reconstruction error* dan *adjusted anomaly-class score* dinormalisasi secara terpisah menggunakan normalisasi Min–Max agar berada pada skala yang sebanding. Kedua komponen tersebut kemudian digabungkan untuk menghasilkan skor anomali akhir pada setiap window. Skor yang lebih tinggi menunjukkan bahwa window memiliki tingkat penyimpangan yang lebih besar terhadap pola yang dipelajari model.

3.1.8 Evaluasi

Evaluasi dilakukan menggunakan label yang terdapat pada *test set*. Adapun label yang tersedia merupakan label penanda bahwa suatu data tidak valid. Label 0 menandakan data normal/valid dan Label 1 menandakan data tidak valid (yang dianggap sebagai anomali). Label anomali ini tidak digunakan pada fase *train* maupun *validation*, tetapi hanya digunakan untuk mengukur kemampuan model dalam memberikan skor yang lebih tinggi pada titik-titik yang ditandai tidak valid (Label 1).

Skor anomali yang dihasilkan REDLAMP merupakan skor anomali pada masing-masing window yang dijadikan *input*. Oleh karena itu, skor anomali dipetakan kembali kepada masing-masing titik pada *test set* dengan menggunakan endpoint mapping, di mana skor anomali dipetakan kepada titik terakhir pada window tersebut. Titik-titik yang tidak memperoleh skor karena berada pada bagian awal deret waktu, termasuk dalam window yang mengandung NaN, atau tidak memenuhi kriteria integritas window, tidak disertakan dalam perhitungan metrik. Label dan skor kemudian diselaraskan berdasarkan timestamp yang sama sebelum evaluasi dilakukan.

Performa model dievaluasi menggunakan empat metrik, yaitu ROC-AUC, AUC-PR, VUS-ROC, dan VUS-PR. ROC-AUC dan AUC-PR digunakan untuk mengevaluasi kemampuan pemeringkatan skor anomali pada tingkat titik terhadap berbagai nilai *threshold*. Selain evaluasi berbasis titik, digunakan VUS-ROC dan VUS-PR untuk mempertimbangkan karakteristik temporal dari label anomali. Kedua metrik tersebut mengevaluasi skor pada berbagai nilai *threshold* dan berbagai ukuran *buffer* di sekitar rentang anomali aktual.

3.1.9 Skenario Pengujian

Pada penelitian ini, diterapkan beberapa skenario pengujian terhadap Model REDLAMP. Pengujian dilakukan pada tiga variasi ukuran window. Untuk setiap ukuran window, diterapkan tiga variasi jumlah kelas augmentasi yang masing-masing terdiri atas satu

kelas normal dan beberapa kelas pseudo-anomali. Dengan konfigurasi tersebut, didapati 9 skenario pengujian yang dapat dilihat pada Tabel 3.6.

Tabel 3.6. Skenario Pengujian

<i>Window Length</i>	Jumlah Kelas Augmentasi
24	12
	6
	2
48	12
	6
	2
96	12
	6
	2

3.2 Bahan dan Peralatan yang Digunakan

Pada penelitian ini, digunakan bahan berupa dataset PM2.5 univariat seperti yang dibahas pada subbab 3.1.1. Untuk mendukung pengerjaan penelitian, digunakan beberapa perangkat keras dan lunak sebagai alat penelitian. Detail masing-masing perangkat dapat dilihat pada Tabel 3.7, 3.8, 3.9, dan 3.10.

Tabel 3.7. Perangkat Keras yang Digunakan

Perangkat	Spesifikasi
Laptop	MSI GL68SE
Prosesor	Intel Core i7 Generasi ke-8
GPU	NVIDIA GeForce RTX 2060
RAM	16GB
Penyimpanan	SSD 1 TB

Tabel 3.8. Platform yang Digunakan

Platform	Spesifikasi
Cloud Computing	Kaggle
CPU	Virtual CPU
GPU	NVIDIA Tesla T4

Penyimpanan	Google Drive (Cloud) dan Kaggle
-------------	---------------------------------

Tabel 3.9. Perangkat Lunak yang Digunakan

Perangkat	Spesifikasi
Sistem Operasi	Windows 11
Bahasa Pemrograman	Python
Versi Bahasa	3.10
Environment	Jupyter Notebook

Tabel 3.10. Pustaka Pendukung yang Digunakan

Pustaka	Versi
NumPy	1.26.4
Pandas	3.0.3
Matplotlib	3.11.0
Scikit-learn	1.5.2
PyTorch	2.10.0+cu128
VUS	0.0.6

3.3 Implementasi

3.3.1 Load Dataset

Input : Direktori dataset berisi berkas Excel bulanan (tahun 2021-2023)
Output: DataFrame gabungan.
1. Tetapkan direktori dataset serta daftar tahun dan bulan yang akan dimuat. 2. Siapkan penampung kosong untuk menampung data setiap bulan. 3. Ulangi proses untuk setiap kombinasi tahun dan bulan. 4. Baca berkas Excel bulanan sesuai tahun dan bulan dari direktori dataset. 5. Rapikan nama kolom dengan menghapus spasi berlebih. 6. Ambil kolom waktu, PM2.5, dan anomaly, lalu ubah namanya menjadi <i>datetime</i>, <i>pm25</i>, dan <i>label</i>. 7. Konversi kolom <i>datetime</i>, <i>pm25</i>, dan <i>label</i> ke tipe data yang sesuai. 9. Buang baris yang memiliki nilai kosong pada kolom <i>datetime</i>, <i>pm25</i>, atau <i>label</i>. 10. Tambahkan informasi tahun, bulan, dan nama berkas sumber pada tiap baris. 11. Simpan data bulan tersebut ke dalam penampung. 12. Gabungkan seluruh data bulanan menjadi satu DataFrame. 13. Kembalikan DataFrame gabungan sebagai hasil pemuatan dataset. 14. Selesai.

Kode Semu 3.2. Load Dataset

Kode Semu 3.2 menunjukkan proses pemuatan *dataset* dari *file Excel* bulanan sepanjang periode 2021 hingga 2023. Pemuatan dilakukan dengan menelusuri setiap kombinasi tahun dan bulan, lalu membaca *file* yang sesuai dari direktori *dataset*. Sebelum diproses lebih lanjut, nama kolom dirapikan dengan menghapus spasi berlebih agar tidak menimbulkan kesalahan saat pemanggilan kolom. Dari setiap *file*, hanya kolom waktu, PM2.5, dan anomaly yang diambil, kemudian namanya diseragamkan menjadi *datetime*, *pm25*, dan *label* supaya konsisten di seluruh proses. Ketiga kolom tersebut dikonversi ke tipe data yang sesuai, dan baris yang mengandung nilai kosong dibuang untuk menjaga integritas data. Agar asal data tetap dapat ditelusuri, informasi tahun, bulan, dan nama *file* sumber ditambahkan pada setiap baris. Seluruh data bulanan akhirnya digabungkan menjadi satu *DataFrame* yang menjadi masukan bagi seluruh tahap *preprocessing*.

3.3.2 Pemeriksaan Timestamp

Input : DataFrame hasil load dataset dengan kolom <i>datetime</i>
Output: DataFrame yang hanya berisi timestamp pada grid 30 menit
1. Tandai baris yang timestamp-nya tidak berada pada grid 30 menit, yaitu yang menitnya bukan 0 atau 30, atau detiknya bukan 0. 2. Pisahkan baris off-grid tersebut lalu tampilkan jumlah dan isinya sebagai pemeriksaan. 3. Buang seluruh baris off-grid karena data target berada pada grid 30 menit. 4. Urutkan kembali data berdasarkan <i>datetime</i> dan tata ulang indeksnya. 5. Kembalikan DataFrame yang seluruh timestamp-nya berada pada grid 30 menit. 6. Selesai.

Kode Semu 3.3. Pemeriksaan Timestamp Off-Grid

Kode Semu 3.3 menunjukkan pemeriksaan *timestamp* yang tidak berada pada *grid* 30 menit. Data target pada penelitian ini memiliki interval tetap 30 menit, sehingga setiap *timestamp* seharusnya jatuh tepat pada menit ke-0 atau ke-30 dengan detik bernilai nol. Baris yang tidak memenuhi ketentuan tersebut ditandai sebagai *off-grid*, lalu dipisahkan dan ditampilkan lebih dulu agar jumlah dan isinya dapat diperiksa sebelum dibuang. Setelah itu, seluruh baris *off-grid* dihapus supaya data hanya berisi *timestamp* yang konsisten dengan *grid*

30 menit. Data kemudian diurutkan kembali berdasarkan *datetime* dan indeksnya ditata ulang sehingga rapi dan seragam.

Input : DataFrame dengan kemungkinan timestamp duplikat
Output: DataFrame dengan datetime non-duplikat
1. Tandai dan pisahkan baris yang memiliki datetime duplikat, lalu tampilkan jumlah dan sebagiannya sebagai pemeriksaan. 2. Bila terdapat duplikat, gabungkan baris berdasarkan datetime menggunakan agregasi konservatif: pm25 dirata-ratakan, label diambil nilai maksimum, dan nama file sumber digabungkan. 3. Urutkan kembali data berdasarkan datetime dan tata ulang indeksnya. 4. Kembalikan DataFrame yang seluruh datetime-nya unik. 5. Selesai.

Kode Semu 3.4. Pengecekan Timestamp Duplikat

Kode Semu 3.4 menunjukkan penanganan *timestamp* yang muncul lebih dari satu kali. Setiap *datetime* seharusnya hanya mewakili satu pengamatan, sehingga baris yang memiliki *datetime* duplikat terlebih dahulu ditandai, dipisahkan, dan ditampilkan sebagai pemeriksaan. Apabila duplikat memang ditemukan, baris-baris tersebut digabungkan berdasarkan *datetime* dengan pendekatan agregasi yang konservatif, yaitu nilai *pm25* dirata-ratakan, *label* diambil nilai maksimum agar penanda anomali tidak hilang ketika salah satu baris menandainya, dan nama *file* sumber digabungkan agar asal data tetap dapat ditelusuri. Melalui langkah ini, setiap *datetime* dijamin hanya diwakili oleh satu baris tunggal.

Input : DataFrame dengan datetime unik (mungkin belum lengkap gridnya)
Output: DataFrame dengan grid waktu 30 menit yang lengkap, disertai penanda timestamp yang hilang dan label evaluasi
1. Bentuk grid waktu lengkap dengan interval 30 menit dari awal 2021 hingga akhir 2023. 2. Indeks ulang data pada grid waktu lengkap tersebut sehingga timestamp yang hilang muncul sebagai baris kosong. 3. Tandai baris yang nilai pm25-nya kosong sebagai timestamp yang hilang. 4. Salin label asli menjadi label evaluasi, lalu kosongkan label evaluasi pada timestamp yang hilang. 5. Hapus kolom label lama yang tidak lagi diperlukan. 6. Kembalikan DataFrame dengan grid waktu lengkap beserta penanda timestamp hilang dan label evaluasi. 7. Selesai.

Kode Semu 3.5. Reindeks Timestamp

Kode Semu 3.5 menunjukkan proses *reindexing* untuk memastikan data memiliki *grid* waktu yang lengkap dengan interval 30 menit sepanjang 2021 hingga 2023. Mula-mula dibentuk *grid* waktu ideal yang mencakup seluruh *timestamp* yang seharusnya ada, kemudian data diindeks ulang terhadap *grid* tersebut sehingga *timestamp* yang sebelumnya tidak tercatat akan muncul sebagai baris kosong. Baris yang nilai *pm25*-nya kosong lalu ditandai sebagai *missing timestamp*. Label asli disalin menjadi label evaluasi, dan khusus pada *timestamp* yang hilang, label evaluasi dikosongkan agar baris tanpa data tidak ikut dinilai saat evaluasi. Kolom label lama yang tidak lagi diperlukan kemudian dihapus. Hasilnya, data memiliki *grid* waktu yang seragam sekaligus penanda eksplisit terhadap *timestamp* yang hilang.

3.3.3 Pemeriksaan *Missing Values*

Input : DataFrame dengan penanda timestamp yang hilang
Output: DataFrame yang setiap missing timestamp-nya diberi identitas rangkaian, panjang rangkaian, dan penanda long gap
1. Tetapkan batas gap pendek, yaitu 12 titik (setara 6 jam). 2. Urutkan data berdasarkan datetime dan tata ulang indeksinya. 3. Inisialisasi kolom identitas rangkaian dan panjang rangkaian missing. 4. Ambil seluruh baris yang ditandai sebagai timestamp hilang. 5. Kelompokkan missing timestamp yang berurutan menjadi satu rangkaian dengan memberi identitas rangkaian setiap kali jeda antar-timestamp bukan 30 menit. 6. Hitung panjang tiap rangkaian, lalu petakan kembali identitas dan panjang rangkaian ke DataFrame utama. 7. Tandai missing timestamp sebagai long gap bila panjang rangkaiannya melebihi batas gap pendek. 8. Kembalikan DataFrame beserta identitas rangkaian, panjang rangkaian, dan penanda long gap. 9. Selesai.

Kode Semu 3.6. Pemeriksaan *Missing Values*

Kode Semu 3.6 menunjukkan proses pengelompokan *missing timestamp* menjadi rangkaian sekaligus pembedaannya menjadi *gap* pendek dan *gap* panjang. Terlebih dahulu ditetapkan batas *gap* pendek sebesar 12 titik yang setara dengan 6 jam. Data diurutkan berdasarkan *datetime*, lalu seluruh baris yang bertanda hilang dikumpulkan. Rangkaian *missing* dibentuk dengan memberi identitas baru setiap kali jeda antar-timestamp bukan 30 menit, sehingga *missing timestamp* yang berurutan dikelompokkan menjadi satu rangkaian yang sama. Panjang setiap rangkaian kemudian dihitung dan dipetakan kembali ke data utama. Berdasarkan panjang tersebut, suatu *missing timestamp* ditandai sebagai *long gap* apabila panjang rangkaiannya melebihi batas *gap* pendek. Pembedaan ini penting karena *gap* pendek masih dapat ditangani melalui interpolasi, sedangkan *gap* panjang menandakan kekosongan data yang terlalu lebar untuk diisi sehingga perlu diperlakukan secara berbeda pada tahap berikutnya.

3.3.4 Pemeriksaan *Static Values*

Input : DataFrame dengan grid waktu lengkap dan kolom pm25
Output: DataFrame yang setiap flatline-nya diberi identitas rangkaian, panjang rangkaian, posisi titik dalam rangkaian, serta penanda flatline nol, flatline non-nol, dan flatline panjang
1. Tetapkan batas panjang rangkaian statis untuk nilai nol dan nilai non-nol. 2. Urutkan data berdasarkan datetime dan tata ulang indeksinya. 3. Bentuk rangkaian flatline baru setiap kali nilai berubah, jeda antar-titik bukan 30 menit, atau salah satu titik hilang, lalu beri identitas rangkaian. 4. Kecualikan missing timestamp agar tidak dihitung sebagai flatline. 5. Hitung panjang tiap rangkaian dan posisiurut setiap titik di dalam rangkaian. 6. Tandai flatline nol bila nilainya nol dan panjang rangkaiannya mencapai batas statis nol. 7. Tandai flatline non-nol bila nilainya bukan nol dan panjang rangkaiannya mencapai batas statis non-nol. 8. Tandai flatline panjang bila titik termasuk flatline nol atau flatline non-nol. 9. Kembalikan DataFrame beserta seluruh penanda flatline tersebut. 10. Selesai.

Kode Semu 3.7. Pemeriksaan Nilai Statis

Kode Semu 3.7 menunjukkan proses deteksi *flatline*, yaitu rangkaian nilai *pm25* yang statis selama beberapa titik berturut-turut. Sebelum pemrosesan, ditetapkan dua ambang panjang rangkaian, masing-masing untuk nilai nol dan nilai non-nol, karena keduanya memiliki karakteristik berbeda. Data diurutkan berdasarkan *datetime*, lalu sebuah rangkaian *flatline*

dianggap terputus setiap kali nilai berubah, jeda antar-titik bukan 30 menit, atau salah satu titik merupakan data yang hilang, sehingga hanya nilai identik yang benar-benar berurutan dan teramati yang dikelompokkan menjadi satu rangkaian. *Missing timestamp* sengaja dikecualikan supaya kekosongan data tidak keliru dihitung sebagai *flatline*. Untuk setiap rangkaian, dihitung panjangnya serta posisi urut setiap titik di dalamnya, yang nantinya berguna untuk menentukan titik mana yang dipertahankan atau dibuang. Berdasarkan panjang rangkaian, suatu titik ditandai sebagai *flatline nol* bila bernilai nol dan mencapai ambang statis nol, atau *flatline non-nol* bila bukan nol dan mencapai ambang statis non-nol; keduanya kemudian digabung menjadi penanda *flatline panjang*. Perlu dicatat bahwa rangkaian statis non-nol tidak seluruhnya dibuang, melainkan hanya titik setelah ambang yang dihapus, sehingga informasi awal rangkaian tetap dipertahankan.

3.3.5 Pemeriksaan *Extreme Values*

Input : DataFrame dengan grid waktu lengkap dan kolom pm25
Output: 50 baris dengan nilai pm25 tertinggi
1. Urutkan data berdasarkan pm25 dari nilai terbesar ke terkecil. 2. Ambil 50 baris teratas beserta kolom <i>datetime</i>, pm25, label evaluasi, dan file sumber. 3. Tampilkan hasilnya untuk pemeriksaan nilai pm25 tertinggi. 4. Selesai.

Kode Semu 3.8. Pemeriksaan Nilai PM2.5 Tertinggi

Kode Semu 3.8 menunjukkan pemeriksaan terhadap nilai *pm25* tertinggi pada data. Data diurutkan berdasarkan *pm25* dari yang terbesar, kemudian 50 baris teratas ditampilkan bersama kolom *datetime*, *pm25*, label evaluasi, dan *file* sumber. Pemeriksaan ini bersifat eksploratif dan bertujuan mengenali keberadaan nilai *pm25* yang tampak ekstrem, sekaligus menelusuri kapan nilai tersebut muncul dan dari *file* mana asalnya. Informasi ini menjadi dasar untuk menentukan ambang nilai *pm25* yang dianggap tidak wajar pada tahap penanganan nilai ekstrem berikutnya.

3.3.6 Pembentukan Nilai PM2.5 bersih

Input : DataFrame dengan penanda missing timestamp, flatline, dan nilai ekstrem
Output: DataFrame dengan kolom pm25 bersih, label evaluasi bersih, dan penanda titik pm25 tidak valid
1. Salin pm25 asli menjadi pm25 bersih sebagai titik awal pembersihan. 2. Kosongkan pm25 bersih pada missing timestamp. 3. Kosongkan pm25 bersih pada seluruh titik flatline nol yang panjang. 4. Kosongkan pm25 bersih hanya pada titik flatline non-nol setelah melewati batas statis non-nol, lalu tandai titik yang dipotong tersebut. 5. Kosongkan pm25 bersih pada titik bernilai ekstrem tinggi. 6. Salin label evaluasi menjadi label evaluasi bersih, lalu kosongkan pada titik yang pm25 bersihnya kosong. 7. Tandai setiap titik yang pm25 bersihnya kosong sebagai pm25 tidak valid. 8. Kembalikan DataFrame beserta pm25 bersih, label evaluasi bersih, dan penanda pm25 tidak valid. 9. Selesai.

Kode Semu 3.9. Pembentukan Nilai PM2.5 Clean

Kode Semu 3.9 menunjukkan proses pembersihan nilai *pm25* dengan menyatukan seluruh penanda yang telah dibentuk pada tahap sebelumnya. Nilai *pm25* asli terlebih dahulu disalin menjadi *pm25* bersih, kemudian titik-titik yang dianggap tidak layak dipakai dikosongkan secara bertahap. *Missing timestamp*, *flatline nol* yang panjang, dan nilai ekstrem tinggi seluruhnya dikosongkan karena tidak mencerminkan pengamatan yang wajar. Berbeda

dari keduanya, *flatline* non-nol tidak dibuang seutuhnya; hanya titik yang berada setelah batas statis non-nol yang dikosongkan, sedangkan titik awal rangkaian tetap dipertahankan agar informasi awal tidak hilang, dan titik yang dipotong ini ditandai secara khusus. Setelah *pm25* bersih terbentuk, label evaluasi disalin menjadi label evaluasi bersih, lalu dikosongkan pada setiap titik yang nilai *pm25* bersihnya kosong sehingga titik tanpa data valid tidak ikut dinilai. Terakhir, seluruh titik yang *pm25* bersihnya kosong ditandai sebagai *pm25* tidak valid, yang menjadi acuan untuk mengeluarkan titik-titik tersebut dari pemrosesan model maupun evaluasi.

3.3.7 Train, Validation, Test Split

Input : DataFrame gabungan dengan grid waktu lengkap dan kolom pm25 bersih
Output: Tiga subset data, yaitu train, validation, dan test
1. Ambil data pada rentang Januari 2021 hingga sebelum November 2022 sebagai train set. 2. Ambil data pada rentang November 2022 hingga sebelum Juni 2023 sebagai validation set. 3. Ambil data pada rentang Juni 2023 hingga akhir 2023 sebagai test set. 4. Kembalikan ketiga subset tersebut. 5. Selesai.

Kode Semu 3.10. Train, Validation, Test Split

Kode Semu 3.10 menunjukkan pembagian data menjadi tiga subset berdasarkan rentang waktu secara kronologis. *Train set* mencakup data dari Januari 2021 hingga sebelum November 2022, *validation set* dari November 2022 hingga sebelum Juni 2023, dan *test set* dari Juni 2023 hingga akhir 2023. Pembagian dilakukan menurut urutan waktu, bukan secara acak, agar tidak terjadi kebocoran informasi masa depan ke dalam data latih dan agar evaluasi mencerminkan kondisi penggunaan nyata ketika model memprediksi data yang muncul setelah periode pelatihan. Ketiga subset ini menjadi dasar bagi pembentukan *window* yang dilakukan secara terpisah pada masing-masing bagian.

3.3.8 Interpolasi Missing Values

Input : Subset data (train/validation) dengan kolom pm25 bersih dan penanda missing
Output: Subset data yang short gap-nya terisi hasil interpolasi, sedangkan long gap tetap kosong
1. Tetapkan batas gap pendek sebesar 12 titik. 2. Urutkan data berdasarkan datetime dan tata ulang indeksnya. 3. Hitung nilai interpolasi berbasis waktu untuk seluruh titik kosong, hanya pada kekosongan yang diapit nilai valid (tidak pada ujung awal maupun akhir). 4. Salin pm25 bersih menjadi kolom pm25 hasil interpolasi. 5. Tandai titik yang merupakan missing timestamp dengan panjang rangkaian tidak melebihi batas gap pendek sebagai short gap. 6. Isi hanya titik short gap dengan nilai interpolasi, sedangkan long gap dibiarkan tetap kosong. 7. Kembalikan subset data beserta kolom pm25 hasil interpolasi. 8. Selesai.

Kode Semu 3.11. Interpolasi Missing Values

Kode Semu 3.11 menunjukkan proses interpolasi yang diterapkan hanya pada short gap, dilakukan secara terpisah untuk setiap subset agar interpolasi tidak melintasi batas antar-subset. Setiap subset diurutkan berdasarkan datetime, lalu nilai interpolasi berbasis waktu dihitung untuk titik-titik kosong. Interpolasi dibatasi hanya pada kekosongan yang diapit oleh nilai valid, sehingga kekosongan pada ujung awal maupun akhir data tidak ikut diisi. Nilai *pm25*

bersih kemudian disalin menjadi kolom hasil interpolasi, dan hanya titik yang tergolong short gap, yaitu missing timestamp dengan panjang rangkaian tidak melebihi batas 12 titik, yang diisi dengan nilai interpolasi. Sebaliknya, long gap sengaja dibiarkan tetap kosong karena rentang kekosongannya terlalu lebar untuk diisi secara andal. Dengan pendekatan ini, kekosongan singkat dapat dipulihkan tanpa memaksakan nilai buatan pada bagian data yang hilang dalam durasi panjang.

3.3.9 Finalisasi Kolom Pemodelan

Input : Subset data dengan kolom pm25 bersih dan pm25 hasil interpolasi
Output: Subset data dengan nilai pm25 final untuk model, penanda titik hasil interpolasi, dan penanda titik yang dapat dipakai model
1. Tetapkan nilai pm25 hasil interpolasi sebagai nilai pm25 final untuk model. 2. Tandai titik sebagai hasil interpolasi bila merupakan missing timestamp yang semula kosong pada pm25 bersih namun terisi setelah interpolasi. 3. Tandai titik sebagai dapat dipakai model bila nilai pm25 akhirnya tidak kosong. 4. Kembalikan subset data beserta nilai pm25 final dan kedua penanda tersebut. 5. Selesai.

Kode Semu 3.12. Finalisasi Clean Split

Kode Semu 3.12 menunjukkan tahap finalisasi setiap subset setelah interpolasi *short gap* dilakukan. Nilai *pm25* hasil interpolasi ditetapkan sebagai nilai *pm25* final yang akan dipakai untuk pemodelan REDLAMP. Selanjutnya, setiap titik ditandai sebagai hasil interpolasi apabila semula merupakan *missing timestamp* yang kosong pada *pm25* bersih tetapi berhasil terisi setelah interpolasi; penanda ini berguna untuk membedakan nilai pengamatan asli dari nilai buatan. Titik yang nilai *pm25* akhirnya tidak kosong kemudian ditandai sebagai titik yang dapat dipakai model. Kedua penanda ini menjadi acuan pada pembentukan *window*, khususnya untuk mengecualikan *window* yang mengandung nilai kosong atau memiliki proporsi titik hasil interpolasi yang terlalu tinggi.

3.3.10 Min-Max Normalization

Input : Subset train, validation, dan test dengan nilai pm25 final untuk model
Output: Ketiga subset dengan tambahan kolom pm25 hasil normalisasi
1. Inisialisasi scaler Min-Max. 2. Ambil nilai pm25 model dari train set yang tidak kosong sebagai dasar penyesuaian scaler. 3. Sesuaikan (fit) scaler hanya pada nilai train set tersebut. 4. Terapkan (transform) scaler yang sama untuk menormalisasi pm25 model pada train, validation, dan test set. 5. Kembalikan ketiga subset beserta kolom pm25 hasil normalisasi. 6. Selesai.

Kode Semu 3.13. Min-Max Normalization

Kode Semu 3.13 menunjukkan proses normalisasi nilai *pm25* menggunakan *Min-Max scaler* agar seluruh nilai berada pada rentang yang seragam. Penyesuaian *scaler* dilakukan hanya menggunakan nilai *pm25* model dari *train set* yang tidak kosong, sehingga batas minimum dan maksimum sepenuhnya ditentukan dari data latih. *Scaler* yang sama kemudian diterapkan untuk menormalisasi *train*, *validation*, dan *test set*. Pendekatan ini penting untuk mencegah kebocoran informasi dari *validation* maupun *test set* ke dalam proses penyesuaian *scaler*, sehingga statistik data uji tidak memengaruhi normalisasi. Dengan cara ini, nilai *test set* dapat saja sedikit melampaui rentang [0, 1] apabila melebihi nilai ekstrem *train set*, dan hal tersebut memang diharapkan agar evaluasi tetap mencerminkan kondisi data yang belum pernah dilihat model.

3.3.11 Pembentukan *Sliding Window*

Input : Subset data dengan kolom pm25 hasil normalisasi dan penanda titik hasil interpolasi
Output: Kumpulan window valid beserta informasi setiap window, baik yang valid maupun seluruhnya
1. Tetapkan panjang window dan batas maksimum proporsi titik hasil interpolasi per window. 2. Urutkan data berdasarkan datetime dan tata ulang indeksinya. 3. Geser window satu titik demi satu titik di sepanjang data (<i>sliding window</i>). 4. Untuk setiap posisi, ambil potongan nilai dan penanda interpolasi sepanjang window. 5. Periksa apakah window mengandung nilai kosong dan hitung jumlah titik hasil interpolasi di dalamnya. 6. Tandai window sebagai valid bila tidak mengandung nilai kosong dan jumlah titik interpolasinya masih di bawah batas yang ditetapkan. 7. Catat informasi setiap window, mencakup indeks, waktu awal dan akhir, jumlah titik interpolasi, serta status validitasnya. 8. Kumpulkan hanya window valid sebagai data window akhir. 9. Kembalikan kumpulan window valid beserta informasi window valid dan informasi seluruh window. 10. Selesai.

Kode Semu 3.14. Pembentukan Window (*Sliding Window*)

Kode Semu 3.14 menunjukkan pembentukan *window* melalui pendekatan *sliding window*, yang diterapkan secara terpisah pada setiap subset agar tidak terbentuk *window* yang melintasi batas antar-subset. Data terlebih dahulu diurutkan berdasarkan *datetime*, kemudian *window* digeser satu titik demi satu titik di sepanjang data. Pada setiap posisi, diambil potongan nilai *pm25* ternormalisasi beserta penanda interpolasinya sepanjang ukuran *window*. Sebuah *window* dianggap valid apabila tidak mengandung nilai kosong sekaligus memiliki jumlah titik hasil interpolasi yang masih di bawah batas maksimum, yaitu 10% dari panjang *window*. Kriteria ini menjaga agar *window* yang dipakai tidak didominasi oleh nilai buatan hasil interpolasi. Untuk setiap *window* dicatat informasi berupa indeks, waktu awal dan akhir, jumlah titik interpolasi, serta status validitasnya, sehingga baik *window* valid maupun keseluruhan *window* tetap dapat ditelusuri. Hanya *window* valid yang dikumpulkan sebagai masukan bagi tahap augmentasi dan pelatihan model.

3.3.12 Augmentasi Pseudo-Anomali

Input : Himpunan window X_{train} berdimensi $(N \times D \times L)$, daftar kelas augmentasi, rentang rasio panjang segmen, panjang segmen minimum, dan seed
Output: Himpunan window teraugmentasi (X_{aug}), label kelas (Y_{aug}), dan anomaly mask (W_{aug})
1. Tetapkan seed acak untuk menjamin reproducibility. 2. Gunakan seluruh daftar kelas bila kelas augmentasi tidak ditentukan secara khusus, lalu tetapkan jumlah kelas K. 3. Ulangi proses untuk setiap window pada himpunan masukan. 4. Ulangi proses untuk setiap kelas augmentasi yang dipilih. 5. Salin window asli, inisialisasi anomaly mask bernilai nol, dan bentuk label kelas (<i>one-hot</i> atau indeks kelas). 6. Simpan window tanpa modifikasi bila kelas adalah normal, kemudian lanjut ke kelas berikutnya. 7. Pilih secara acak 1 sampai D dimensi yang akan dimodifikasi. 8. Ulangi proses untuk setiap dimensi terpilih. 9. Untuk kelas spike, pilih satu titik acak, terapkan augmentasi spike, tandai titik tersebut pada anomaly mask, lalu lanjut ke dimensi berikutnya. 10. Untuk kelas lainnya, tentukan panjang segmen secara proporsional terhadap L (dibatasi panjang minimum dan tidak melebihi L), lalu tetapkan indeks awal dan akhir segmen secara acak. 11. Terapkan fungsi augmentasi sesuai kelas pada segmen tersebut. 12. Tandai anomaly mask sepanjang segmen, kecuali untuk kelas wander yang ditandai dari indeks awal hingga akhir window. 13. Simpan window hasil augmentasi, label, dan anomaly mask.

14. Kembalikan seluruh window teraugmentasi beserta label dan anomaly mask. 15. Selesai .
--

Kode Semu 3.15. Fungsi Augmentasi

Kode Semu 3.15 menunjukkan fungsi utama augmentasi REDLAMP yang mengoordinasikan seluruh fungsi pseudo-anomali untuk membentuk data multiclass. Proses diawali dengan penetapan *seed* agar hasil dapat direproduksi, lalu augmentasi diterapkan pada setiap *window* untuk setiap kelas yang dipilih sehingga tiap kelas memiliki rasio 1:1 terhadap jumlah *window* awal. Pada setiap iterasi, *window* asli disalin, *anomaly mask* diinisialisasi nol, dan label kelas dibentuk baik dalam bentuk *one-hot* maupun indeks kelas. Kelas normal langsung disimpan tanpa modifikasi karena merepresentasikan *window* asli. Untuk kelas pseudo-anomali, dipilih terlebih dahulu dimensi yang akan dimodifikasi; karena data bersifat univariat ($D = 1$), pemilihan ini selalu menghasilkan satu dimensi, yaitu PM2.5. Kelas *spike* diperlakukan khusus karena hanya menyisipkan lonjakan pada satu titik acak, sehingga *anomaly mask* ditandai pada titik tunggal. Untuk kelas lainnya, panjang segmen ditentukan secara proporsional terhadap panjang *window* melalui rentang rasio 0,10 hingga 0,30, dengan batas panjang minimum agar segmen tetap bermakna pada berbagai ukuran *window*; indeks awal dan akhir kemudian ditetapkan secara acak. Setelah fungsi augmentasi diterapkan, *anomaly mask* menandai posisi perubahan sepanjang segmen, dengan pengecualian pada kelas *wander* yang ditandai hingga akhir *window* karena pergeseran nilainya bersifat berkelanjutan. Seluruh *window* teraugmentasi beserta label dan *anomaly mask* dikumpulkan sebagai keluaran yang menjadi masukan bagi tahap *label refurbishment* dan pelatihan model.

3.3.13 Label Refurbishment

Input : Matriks label one-hot Y berukuran $(N \times K)$, parameter alpha dan beta, serta indeks kelas normal
--

Output: Matriks soft label Y_{soft} berukuran $(N \times K)$

1. Kalikan setiap label one-hot dengan faktor $(1 - \alpha - K \cdot \beta)$, lalu tambahkan nilai dasar beta pada seluruh elemen kelas. 2. Tambahkan nilai alpha khusus pada elemen kelas normal. 3. Kembalikan matriks soft label hasil koreksi. 4. Selesai .

Kode Semu 3.16. Label Refurbishment

Kode Semu 3.16 menunjukkan proses *label refurbishment*, yaitu konversi *hard label* menjadi *soft label* untuk mencegah model menjadi *overconfident* terhadap label augmentasi. Proses dilakukan secara *element-wise* pada setiap vektor label. Mula-mula, elemen pada kelas target dikalikan dengan faktor $(1 - \alpha - K \cdot \beta)$, kemudian seluruh elemen kelas diberi nilai dasar sebesar beta yang pada penelitian ini ditetapkan 0,01. Khusus pada elemen kelas normal, ditambahkan nilai alpha sebesar 0,1. Dengan mekanisme ini, kelas target tetap memiliki probabilitas terbesar, sedangkan sebagian probabilitas didistribusikan ke kelas normal dan kelas lainnya sehingga jumlah seluruh elemen pada setiap *soft label* tetap bernilai 1. Distribusi ulang ini mengikuti pendekatan *backward correction* yang memperhitungkan kemungkinan kesalahan pelabelan antar-kelas, mengingat pseudo-anomali dihasilkan dari data normal melalui augmentasi.

3.3.14 Implementasi Model REDLAMP

3.3.14.1 ConvBlock1D

Input : Tensor x dengan in_ch kanal masukan; parameter jumlah kanal keluaran out_ch, ukuran kernel, dan dropout
Output: Tensor hasil ekstraksi fitur dengan out_ch kanal
1. Definisikan blok konvolusi satu dimensi yang tersusun atas konvolusi 1D dengan padding menjaga panjang keluaran sama dengan masukan, diikuti batch normalization, aktivasi ReLU, dan dropout. 2. Pada proses forward, lewatkan masukan melalui blok tersebut secara berurutan. 3. Kembalikan tensor hasil blok konvolusi. 4. Selesai.

Kode Semu 3.17. Convblock1D

Kode Semu 3.17 menunjukkan definisi *ConvBlock1D*, yaitu blok konvolusi satu dimensi yang menjadi komponen penyusun *encoder* dan *decoder* pada REDLAMP. Blok ini menggabungkan empat operasi secara berurutan, yaitu konvolusi satu dimensi, *batch normalization*, aktivasi *ReLU*, dan *dropout*. Konvolusi menggunakan *padding* sebesar setengah ukuran *kernel* agar panjang keluaran tetap sama dengan panjang masukan, sehingga dimensi temporal *window* terjaga di sepanjang jaringan. *Batch normalization* menstabilkan distribusi aktivasi antar-*batch*, aktivasi *ReLU* memberikan sifat non-linear, sedangkan *dropout* berperan sebagai regularisasi untuk mengurangi *overfitting*. Ukuran *kernel* pada penelitian ini ditetapkan sebesar 3, sehingga blok berfokus pada pola lokal antar-titik yang berdekatan.

3.3.14.2 Encoder

Input : Tensor window x dengan in_channels kanal; parameter dimensi laten latent_dim, daftar jumlah kanal konvolusi conv_channels, dan dropout
Output: Representasi laten z
1. Susun beberapa blok konvolusi 1D secara bertumpuk sesuai daftar jumlah kanal, dengan jumlah kanal tiap blok mengikuti keluaran blok sebelumnya. 2. Definisikan lapisan max pooling untuk mereduksi panjang temporal setengahnya. 3. Definisikan lapisan konvolusi akhir yang memetakan keluaran ke dimensi laten. 4. Pada proses forward, lewatkan masukan melalui tumpukan blok konvolusi, lalu max pooling, kemudian konvolusi akhir untuk menghasilkan representasi laten. 5. Kembalikan representasi laten z. 6. Selesai.

Kode Semu 3.18. Encoder

Kode Semu 3.18 menunjukkan arsitektur *encoder* yang bertugas mengubah *window* masukan menjadi representasi laten. *Encoder* tersusun atas beberapa blok konvolusi satu dimensi yang ditumpuk secara berurutan, dengan jumlah kanal yang meningkat secara bertahap sesuai daftar *conv_channels*. Setiap blok mengikuti jumlah kanal keluaran dari blok sebelumnya, sehingga fitur yang diekstraksi semakin kaya di sepanjang jaringan. Setelah melalui tumpukan blok konvolusi, keluaran dilewatkan pada lapisan *max pooling* yang mereduksi panjang dimensi temporal menjadi setengahnya untuk memampatkan informasi. Terakhir, sebuah lapisan konvolusi memetakan keluaran ke dimensi laten yang ditetapkan, sehingga menghasilkan representasi laten yang selanjutnya digunakan oleh *decoder* untuk merekonstruksi *window* dan oleh *classifier* untuk memprediksi kelas.

3.3.14.3 Decoder

Input : Representasi laten z ; parameter dimensi laten `latent_dim`, panjang laten `latent_len`, panjang keluaran `output_len`, jumlah kanal keluaran `out_channels`, daftar jumlah kanal konvolusi `conv_channels`, dan `dropout`

Output: Window hasil rekonstruksi $\hat{x}$

1. **Definisikan** lapisan linear untuk memperpanjang dimensi temporal laten dari panjang laten menjadi panjang keluaran (*upsampling*).
2. **Susun** beberapa blok konvolusi 1D secara bertumpuk dengan urutan jumlah kanal dibalik dari susunan encoder.
3. **Definisikan** lapisan konvolusi akhir yang memetakan keluaran ke jumlah kanal keluaran.
4. **Pada** proses forward, perpanjang dimensi temporal laten melalui lapisan linear, lewatkan melalui tumpukan blok konvolusi, lalu konvolusi akhir untuk menghasilkan window rekonstruksi.
5. **Kembalikan** window hasil rekonstruksi $\hat{x}$.
6. **Selesai**.

Kode Semu 3.19. Decoder

Kode Semu 3.19 menunjukkan arsitektur *decoder* yang bertugas merekonstruksi *window* dari representasi laten yang dihasilkan *encoder*. Karena *encoder* memampatkan dimensi temporal melalui *max pooling*, *decoder* terlebih dahulu memperpanjang kembali panjang temporal laten menuju panjang keluaran yang diinginkan menggunakan lapisan linear sebagai *upsampling*. Setelah itu, keluaran dilewatkan melalui tumpukan blok konvolusi satu dimensi yang susunan jumlah kanalnya dibalik dari susunan pada *encoder*, sehingga struktur *decoder* mencerminkan proses kebalikan dari *encoder*. Terakhir, sebuah lapisan konvolusi memetakan keluaran ke jumlah kanal keluaran, yaitu satu kanal untuk data univariat, sehingga dihasilkan *window* rekonstruksi yang berukuran sama dengan *window* masukan. Hasil rekonstruksi ini menjadi dasar perhitungan *masked reconstruction loss* pada saat pelatihan sekaligus komponen *reconstruction score* pada tahap inferensi.

3.3.14.4 Classifier

Input : Representasi laten z ; parameter dimensi laten `latent_dim`, panjang laten `latent_len`, jumlah kelas `num_classes`, dan `dropout`

Output: Logit prediksi kelas

1. **Hitung** jumlah fitur masukan sebagai hasil kali dimensi laten dan panjang laten.
2. **Definisikan** jaringan klasifikasi berupa lapisan linear menuju 64 unit, diikuti *batch normalization*, aktivasi *ReLU*, dan *dropout*, lalu lapisan linear akhir menuju jumlah kelas.
3. **Pada** proses forward, ratakan representasi laten menjadi vektor satu dimensi, lalu lewatkan melalui jaringan klasifikasi untuk menghasilkan logit.
4. **Kembalikan** logit prediksi kelas.
5. **Selesai**.

Kode Semu 3.20. Classifier

Kode Semu 3.20 menunjukkan arsitektur *classifier* yang bertugas memprediksi kelas augmentasi dari representasi laten. Sebelum diklasifikasikan, representasi laten yang berbentuk dua dimensi (dimensi laten dan panjang laten) diratakan menjadi vektor satu dimensi, dengan jumlah fitur masukan sama dengan hasil kali dimensi laten dan panjang laten. Vektor tersebut kemudian dilewatkan melalui sebuah lapisan linear menuju 64 unit tersembunyi, diikuti *batch normalization*, aktivasi *ReLU*, dan *dropout* sebagai regularisasi, sebelum akhirnya dipetakan ke jumlah kelas melalui lapisan linear terakhir. Keluaran berupa *logit* untuk setiap kelas, yang pada saat pelatihan dibandingkan dengan *soft label* melalui *cross-entropy loss*, dan pada tahap inferensi digunakan sebagai komponen *classification score*.

3.3.14.5 Model Utama REDLAMP

Input : Tensor window x ; parameter jumlah kanal masukan, ukuran window, dimensi laten, jumlah kelas, daftar jumlah kanal konvolusi, dan dropout
Output: Window rekonstruksi $\hat{x}$, logit kelas, dan representasi laten z
1. Tetapkan ukuran window dan panjang laten sebesar setengah ukuran window (mengikuti reduksi max pooling pada encoder). 2. Inisialisasi encoder untuk menghasilkan representasi laten dari window masukan. 3. Inisialisasi decoder untuk merekonstruksi window dari representasi laten. 4. Inisialisasi classifier untuk memprediksi kelas dari representasi laten. 5. Pada proses forward, lewatkan window melalui encoder untuk memperoleh representasi laten, lalu gunakan representasi tersebut pada decoder untuk merekonstruksi window dan pada classifier untuk menghasilkan logit kelas. 6. Kembalikan window rekonstruksi, logit kelas, dan representasi laten. 7. Selesai.

Kode Semu 3.21. REDLAMP

Kode Semu 3.21 menunjukkan penggabungan seluruh komponen menjadi arsitektur REDLAMP yang utuh. Model ini menyatukan tiga modul utama, yaitu *encoder*, *decoder*, dan *classifier*, yang berbagi satu representasi laten yang sama. Panjang laten ditetapkan sebesar setengah ukuran *window*, menyesuaikan reduksi dimensi temporal akibat *max pooling* pada *encoder*. Pada proses *forward*, *window* masukan terlebih dahulu diubah menjadi representasi laten oleh *encoder*. Representasi laten tersebut kemudian dimanfaatkan secara bersamaan oleh dua cabang, yaitu *decoder* yang merekonstruksi *window* dan *classifier* yang memprediksi kelas augmentasi. Rancangan berbagi representasi laten ini memungkinkan model belajar secara serentak untuk merekonstruksi pola normal sekaligus membedakan jenis pseudo-anomali, sehingga keluaran berupa *window* rekonstruksi, *logit* kelas, dan representasi laten dapat digunakan bersama pada perhitungan *loss* saat pelatihan maupun *anomaly scoring* saat inferensi.

3.3.15 Implementasi Loss Function

Input : Window asli x , window rekonstruksi $\hat{x}$, anomaly mask w , logit kelas, soft label y_{soft} , dan parameter pembobot γ
Output: Total loss beserta komponen reconstruction loss dan cross-entropy loss
1. Masked reconstruction loss: 2. Bentuk mask normal sebagai kebalikan anomaly mask $(1 - w)$. 3. Hitung selisih rekonstruksi hanya pada bagian normal, kuadratkan, lalu rata-ratakan terhadap jumlah titik normal. 4. 5. Soft cross-entropy loss: 6. Hitung log-softmax dari logit, lalu hitung cross-entropy terhadap soft label dan rata-ratakan antar-sampel. 7. 8. Total loss: 9. Gabungkan cross-entropy loss dan reconstruction loss dengan bobot γ dan $(1 - \gamma)$. 10. 11. Kembalikan total loss beserta kedua komponennya. 12. 13. Selesai.

Kode Semu 3.22. Loss Function

Kode Semu 3.22 menunjukkan perhitungan *loss* yang menggabungkan dua komponen, yaitu *masked reconstruction loss* dan *soft cross-entropy loss*. *Masked reconstruction loss* mengukur kesalahan rekonstruksi hanya pada bagian *window* yang tidak ditandai sebagai pseudo-anomali oleh *anomaly mask*. Untuk itu, dibentuk *mask* normal sebagai kebalikan dari

anomaly mask, sehingga selisih rekonstruksi hanya dihitung pada titik-titik normal, lalu dikuadratkan dan dirata-ratakan terhadap jumlah titik normal. Pembatasan ini penting agar model belajar merekonstruksi pola normal dan tidak dipaksa merekonstruksi titik yang memang sengaja dijadikan anomali. Sementara itu, *soft cross-entropy loss* mengukur perbedaan antara distribusi probabilitas kelas hasil *classifier* dan *soft label* dari *label refurbishment*, dihitung melalui *log-softmax* dan dirata-ratakan antar-sampel. Kedua komponen tersebut digabungkan menggunakan parameter pembobot gamma untuk menghasilkan total *loss*, yang kemudian dipakai dalam *backpropagation* dan pembaruan parameter model.

3.3.16 Hyperparameter Tuning

Input : Ruang pencarian hyperparameter (<code>param_grid</code>) dan parameter tetap (<code>fixed_params</code>)
Output: Daftar seluruh konfigurasi hyperparameter beserta identitas tiap konfigurasi
1. Tetapkan ruang pencarian hyperparameter yang divariasikan, yaitu batch size, latent dimension, dropout, konfigurasi kanal konvolusi, dan learning rate. 2. Tetapkan parameter tetap yang tidak divariasikan, yaitu gamma dan weight decay. 3. Bentuk seluruh kombinasi nilai dari ruang pencarian secara Cartesian product. 4. Untuk setiap kombinasi, satukan dengan parameter tetap dan bentuk identitas konfigurasi yang mencerminkan nilai tiap hyperparameter. 5. Kumpulkan seluruh konfigurasi tersebut. 6. Kembalikan daftar konfigurasi hyperparameter. 7. Selesai.

Kode Semu 3.23. Hyperparameter Tuning

Kode Semu 3.23 menunjukkan pembentukan seluruh konfigurasi *hyperparameter* yang akan diuji selama *tuning*. Ruang pencarian mencakup *hyperparameter* yang divariasikan, yaitu *batch size*, *latent dimension*, konfigurasi kanal konvolusi, dan *learning rate*, sedangkan gamma dan *weight decay* ditetapkan sebagai parameter tetap. Seluruh kombinasi nilai dari ruang pencarian dibentuk melalui *Cartesian product*, sehingga tiap konfigurasi merepresentasikan satu perpaduan unik dari nilai-nilai *hyperparameter* tersebut. Setiap konfigurasi kemudian digabungkan dengan parameter tetap dan diberi identitas yang mencerminkan nilai masing-masing *hyperparameter*, sehingga tiap percobaan dapat dibedakan dan ditelusuri dengan mudah. Pendekatan pencarian menyeluruh ini memastikan setiap kombinasi dievaluasi secara sistematis pada tahap *hyperparameter tuning*.

Input : Daftar konfigurasi hyperparameter serta data train dan validation yang telah diaugmentasi (<i>window</i> , <i>soft label</i> , dan <i>anomaly mask</i>)
Output: Kumpulan hasil tuning untuk setiap konfigurasi
1. Tetapkan jumlah epoch maksimum dan batas kesabaran (<i>patience</i>) untuk early stopping. 2. Ulangi proses untuk setiap konfigurasi hyperparameter. 3. Latih dan evaluasi model pada satu konfigurasi menggunakan data train dan validation, dengan early stopping berdasarkan validation loss. 4. Catat hasil konfigurasi, mencakup epoch terbaik, validation loss terbaik, beserta komponen reconstruction loss dan cross-entropy loss-nya. 5. Kumpulkan seluruh hasil konfigurasi. 6. Kembalikan kumpulan hasil tuning. 7. Selesai.

Kode Semu 3.24. Loop Hyperparameter Tuning

Kode Semu 3.24 menunjukkan pelaksanaan hyperparameter tuning atas seluruh konfigurasi yang telah dibentuk. Untuk membatasi biaya komputasi, setiap konfigurasi dilatih dengan jumlah epoch maksimum yang terbatas dan dilengkapi early stopping berdasarkan

patience, sehingga pelatihan dihentikan lebih awal apabila validation loss tidak lagi membaik. Setiap konfigurasi dilatih menggunakan data train teraugmentasi dan dievaluasi pada data validation teraugmentasi, dengan seed yang ditetapkan agar hasil dapat direproduksi. Hasil dari tiap konfigurasi dicatat, meliputi epoch terbaik, validation loss terbaik, serta rincian komponen reconstruction loss dan cross-entropy loss. Seluruh hasil tersebut kemudian dikumpulkan untuk dibandingkan, sehingga konfigurasi hyperparameter dengan performa terbaik dapat dipilih untuk pelatihan akhir.

3.3.17 Load Best Model

Input : Konfigurasi hyperparameter terbaik dan bobot model terbaik hasil tuning
Output: Model REDLAMP terbaik yang siap digunakan untuk inferensi
1. Inisialisasi model REDLAMP menggunakan konfigurasi hyperparameter terbaik. 2. Muat kembali bobot model terbaik yang tersimpan dari proses tuning. 3. Alihkan model ke mode evaluasi. 4. Kembalikan model REDLAMP terbaik yang siap dipakai. 5. Selesai.

Kode Semu 3.25. Load Best Model

Kode Semu 3.25 menunjukkan proses pemuatan model REDLAMP terbaik hasil *hyperparameter tuning*. Model terlebih dahulu diinisialisasi ulang menggunakan konfigurasi *hyperparameter* terbaik, kemudian bobot terbaik yang telah disimpan pada saat *tuning* dimuat kembali ke dalam model tersebut. Setelah bobot dimuat, model dialihkan ke mode evaluasi agar komponen seperti *dropout* dan *batch normalization* berperilaku sesuai tahap inferensi, bukan tahap pelatihan. Dengan langkah ini, diperoleh model yang parameternya sudah optimal dan siap digunakan untuk tahap inferensi maupun *anomaly scoring* pada data uji.

3.3.18 Train Final dengan Best Model

Input : Konfigurasi hyperparameter terbaik serta data train dan validation yang telah diaugmentasi (window, soft label, dan anomaly mask)
Output: Hasil pelatihan akhir berupa model REDLAMP terlatih beserta catatan pelatihannya
1. Tetapkan jumlah epoch maksimum dan batas kesabaran (<i>patience</i>) yang lebih besar dibanding tahap tuning. 2. Latih model menggunakan konfigurasi hyperparameter terbaik pada data train teraugmentasi, dengan pemantauan validation loss dan early stopping. 3. Kembalikan hasil pelatihan akhir berupa model terlatih beserta catatan pelatihannya. 4. Selesai.

Kode Semu 3.26. Train Akhir dengan Best Model

Kode Semu 3.26 menunjukkan pelatihan akhir model menggunakan konfigurasi *hyperparameter* terbaik yang telah dipilih. Berbeda dari tahap *tuning* yang dibatasi agar hemat komputasi, pelatihan akhir diberi jumlah *epoch* maksimum yang jauh lebih besar dan *patience* yang lebih longgar, sehingga model memiliki kesempatan lebih luas untuk konvergen. Pelatihan tetap menggunakan data *train* teraugmentasi dengan pemantauan *validation loss* dan *early stopping* agar pelatihan berhenti ketika performa pada data *validation* tidak lagi membaik, sehingga *overfitting* dapat dihindari. Penetapan *seed* menjaga agar proses pelatihan dapat direproduksi. Hasil akhir berupa model terlatih beserta catatan pelatihannya, yang selanjutnya dipakai untuk tahap inferensi dan evaluasi pada data uji.

3.3.19 Inferensi pada *Test Set*

Input : Model REDLAMP terlatih dan test loader berisi window uji
Output: Kumpulan window asli, window rekonstruksi, dan logit kelas untuk seluruh data uji
1. Alihkan model ke mode evaluasi dan nonaktifkan perhitungan gradien. 2. Ulangi proses untuk setiap batch pada test loader. 3. Lewatkan batch window melalui model untuk memperoleh window rekonstruksi dan logit kelas. 4. Kumpulkan window asli, window rekonstruksi, dan logit dari setiap batch. 5. Gabungkan seluruh hasil batch menjadi satu kumpulan menyeluruh. 6. Kembalikan kumpulan window asli, window rekonstruksi, dan logit kelas. 7. Selesai.

Kode Semu 3.27. Fungsi Inferensi

Kode Semu 3.27 menunjukkan proses inferensi model REDLAMP pada data uji. Model dialihkan ke mode evaluasi dan perhitungan gradien dinonaktifkan agar inferensi lebih efisien serta tidak memengaruhi parameter model. Data uji diproses per *batch* melalui *test loader*, dan setiap *batch* dilewatkan pada model untuk memperoleh *window* rekonstruksi beserta *logit* kelas. Keluaran dari setiap *batch* dikumpulkan, lalu digabungkan menjadi satu kumpulan menyeluruh yang mencakup *window* asli, *window* rekonstruksi, dan *logit* untuk seluruh data uji. Ketiga keluaran ini menjadi bahan dasar bagi perhitungan *anomaly score*, karena *window* asli dan rekonstruksinya digunakan untuk *reconstruction score*, sedangkan *logit* digunakan untuk *classification score*.

3.3.20 Anomaly Scoring dan Frequent Anomaly Adjustment

Input : Window uji asli dan window uji hasil rekonstruksi
Output: Skor reconstruction error (s_mse) per window
1. Hitung selisih antara window asli dan window rekonstruksi, lalu kuadratkan. 2. Jumlahkan selisih kuadrat pada seluruh titik dalam setiap window sehingga diperoleh satu skor per window. 3. Kembalikan skor reconstruction error per window. 4. Selesai.

Kode Semu 3.28. Reconstruction Error

Kode Semu 3.28 menunjukkan perhitungan *reconstruction score* untuk setiap *window* uji. Skor ini diperoleh dari selisih antara *window* asli dan *window* hasil rekonstruksi yang dikuadratkan, kemudian dijumlahkan pada seluruh titik dalam *window* sehingga menghasilkan satu nilai skor per *window*. Karena model dilatih untuk merekonstruksi pola normal dengan baik, *window* yang mengandung anomali cenderung sulit direkonstruksi dan menghasilkan *reconstruction error* yang lebih tinggi. Dengan demikian, skor ini berperan sebagai indikator anomali berbasis rekonstruksi, di mana nilai yang lebih besar menunjukkan kemungkinan anomali yang lebih tinggi.

Input : Matriks probabilitas kelas per window (probs) berukuran (n_windows × n_classes), ambang delta, dan indeks kelas normal
Output: Skor classification (s_ce) per window beserta informasi kelas yang dibuang dan dipertahankan

1. Tentukan indeks seluruh kelas anomali, yaitu seluruh kelas selain kelas normal.
2. Hitung rata-rata probabilitas tiap kelas di seluruh window.
3. Jika hanya terdapat dua kelas (biner), lewati FAA dan gunakan probabilitas kelas pseudo-anomali langsung sebagai skor classification.
4. Jika multiclass, buang kelas anomali yang rata-rata probabilitasnya melebihi ambang delta dengan menolak probabilitasnya, karena dianggap terlalu sering muncul (false anomaly).
5. Tetapkan kelas anomali yang tersisa sebagai kelas yang dipertahankan.
6. Hitung skor classification dengan menjumlahkan probabilitas kelas anomali yang dipertahankan; bila tidak ada yang tersisa, tetapkan skor bernilai nol.
7. Kembalikan skor classification beserta informasi kelas yang dibuang dan dipertahankan.
8. Selesai.

Kode Semu 3.29. Frequent Anomaly Adjustment

Kode Semu 3.29 menunjukkan perhitungan *classification score* yang dilengkapi mekanisme *Frequent Anomaly Adjustment* (FAA). Skor ini pada dasarnya menjumlahkan probabilitas kelas-kelas anomali sebagai indikasi seberapa besar sebuah *window* dinilai menyimpang dari pola normal. Namun, sebagian kelas pseudo-anomali dapat terlalu sering diprediksi sehingga justru menyerupai pola normal dan berpotensi menjadi *false anomaly*. Untuk mengatasinya, dihitung rata-rata probabilitas tiap kelas di seluruh *window*, lalu kelas anomali yang rata-ratanya melampaui ambang delta dibuang dengan menolak probabilitasnya, karena kemunculannya yang terlalu sering dianggap tidak informatif. Skor *classification* kemudian dihitung dari penjumlahan probabilitas kelas anomali yang tersisa. Sebagai pengecualian, ketika hanya terdapat dua kelas, FAA dilewati dan probabilitas kelas pseudo-anomali digunakan secara langsung, sebab pembuangan kelas pada kondisi biner akan menghilangkan satu-satunya sinyal anomali. *Threshold* delta ditetapkan sebesar $1/K$ (jumlah kelas pseudo-anomali, termasuk kelas normal).

Input : Skor reconstruction (*s_mse*) dan skor classification (*s_ce*) per window

Output: Tiga varian skor akhir, yaitu reconstruction-only, classification-only, dan combined

1. Normalisasi skor reconstruction dan skor classification ke rentang $[0, 1]$ menggunakan min-max.
2. Bentuk varian skor reconstruction-only dari skor reconstruction ternormalisasi.
3. Bentuk varian skor classification-only dari skor classification ternormalisasi.
4. Bentuk varian skor combined sebagai gabungan berbobot sama antara kedua skor ternormalisasi tersebut.
5. Kembalikan ketiga varian skor akhir.
6. Selesai.

Kode Semu 3.30. Anomaly Scoring

Kode Semu 3.30 menunjukkan pembentukan skor anomali akhir dari dua komponen, yaitu *reconstruction score* dan *classification score*. Karena kedua skor berada pada skala yang berbeda, masing-masing terlebih dahulu dinormalisasi ke rentang $[0, 1]$ menggunakan *min-max* agar dapat digabungkan secara adil. Dari kedua skor ternormalisasi tersebut dibentuk tiga varian skor, yaitu *reconstruction-only* yang hanya memakai skor rekonstruksi, *classification-only* yang hanya memakai skor klasifikasi, dan *combined* yang menggabungkan keduanya dengan bobot yang sama. Ketiga varian ini disediakan agar kontribusi tiap komponen *scoring* dapat dibandingkan secara terpisah pada tahap evaluasi, sehingga dapat diketahui skema *scoring* mana yang paling efektif dalam mendeteksi anomali pada data uji.

3.3.21 Evaluasi

3.3.21.1 Endpoint Mapping

Input : Skor per window, informasi window, jumlah titik data (n_points), dan ukuran window
Output: Skor per titik, dengan skor ditempatkan pada titik akhir tiap window
1. Inisialisasi seluruh skor titik dengan nilai kosong. 2. Ulangi proses untuk setiap window beserta skornya. 3. Tentukan titik akhir window sebagai indeks awal ditambah ukuran window dikurangi satu. 4. Tempatkan skor window pada titik akhir tersebut bila indeksnya berada dalam rentang data. 5. Kembalikan skor per titik. 6. Selesai.

Kode Semu 3.31. Endpoint Mapping

Kode Semu 3.31 menunjukkan pemetaan skor dari tingkat *window* ke tingkat titik, yang diperlukan karena evaluasi dilakukan per titik sedangkan skor dihasilkan per *window*. Setiap *window* diwakili oleh satu skor, dan skor tersebut ditempatkan pada titik akhir *window* menggunakan pendekatan *endpoint*, yaitu titik terakhir dari rentang *window*. Titik akhir dipilih karena mencerminkan hasil pengamatan hingga titik terkini pada *window*, sesuai dengan skenario deteksi yang menilai kondisi pada ujung *window*. Titik yang tidak menjadi titik akhir *window* mana pun dibiarkan bernilai kosong sehingga tidak ikut dinilai. Dengan pemetaan ini, skor per *window* dapat disejajarkan dengan label per titik untuk keperluan evaluasi.

3.3.21.2 Point-Level Evaluation

Input : Label sebenarnya per titik (y_true) dan skor anomali per titik (score)
Output: Nilai ROC-AUC dan PR-AUC
1. ROC-AUC: 2. Kembalikan nilai kosong bila data kosong atau hanya memuat satu kelas. 3. Hitung dan kembalikan nilai ROC-AUC dari label dan skor. 4. 5. PR-AUC: 6. Kembalikan nilai kosong bila data kosong atau hanya memuat satu kelas. 7. Bentuk kurva precision-recall dari label dan skor. 8. Hitung dan kembalikan luas di bawah kurva precision-recall. 9. 10. Selesai.

Kode Semu 3.32. ROC-AUC dan AUC-PR

Kode Semu 3.32 menunjukkan perhitungan dua metrik evaluasi berbasis titik, yaitu ROC-AUC dan PR-AUC, yang keduanya dilengkapi pemeriksaan pengaman agar tidak menghasilkan kesalahan pada kasus data yang tidak memadai. Sebelum perhitungan, dipastikan bahwa data tidak kosong dan memuat setidaknya dua kelas, karena kedua metrik hanya terdefinisi jika terdapat titik normal sekaligus titik anomali; bila syarat ini tidak terpenuhi, dikembalikan nilai kosong. ROC-AUC mengukur kemampuan skor membedakan titik anomali dari titik normal di seluruh ambang keputusan. Sementara itu, PR-AUC dihitung dengan terlebih dahulu membentuk kurva *precision-recall* dari label dan skor, lalu mengambil luas di bawah kurva tersebut. PR-AUC menjadi metrik yang lebih informatif pada kondisi data yang sangat tidak seimbang seperti pada penelitian ini, karena berfokus pada performa terhadap kelas anomali yang jumlahnya jauh lebih sedikit.

3.3.21.3 VUS Evaluation

Input : DataFrame evaluasi berisi kolom label, skor endpoint, dan identitas segmen evaluasi; serta buffer maksimum VUS
Output: Rangkaian label dan skor untuk perhitungan VUS beserta informasi ringkasnya
1. Ambil hanya titik yang label, skor, dan identitas segmennya tidak kosong; hentikan proses bila tidak ada titik valid. 2. Kumpulkan daftar identitas segmen evaluasi yang unik. 3. Tetapkan panjang guard sebesar buffer maksimum VUS ditambah satu. 4. Tetapkan skor guard sedikit lebih rendah daripada skor valid terendah. 5. Ulangi proses untuk setiap segmen evaluasi. 6. Ambil label dan skor titik pada segmen tersebut, lalu tambahkan ke rangkaian. 7. Sisipkan guard region berupa label nol dan skor guard di antara dua segmen yang berurutan (tidak setelah segmen terakhir). 8. Gabungkan seluruh bagian menjadi satu rangkaian label dan satu rangkaian skor. 9. Kembalikan rangkaian label, rangkaian skor, dan informasi ringkas evaluasi. 10. Selesai.

Kode Semu 3.33. VUS Input (Guarded)

Kode Semu 3.33 menunjukkan pembentukan masukan untuk perhitungan VUS beserta penerapan *guard region*. Mula-mula, hanya titik yang memiliki label, skor *endpoint*, dan identitas segmen yang valid yang disertakan, sehingga titik tanpa data tidak memengaruhi evaluasi. Titik-titik tersebut kemudian disusun kembali per segmen evaluasi menurut identitas segmennya. *Guard region* digunakan karena data evaluasi dapat terdiri atas beberapa segmen valid yang terpisah akibat adanya NaN atau *window* yang tidak valid. Di antara dua segmen yang berurutan disisipkan *guard region* berupa titik berlabel nol dengan skor yang sedikit lebih rendah dari skor valid terendah, sepanjang buffer maksimum VUS ditambah satu. Penyisipan ini mencegah *buffer* pada perhitungan VUS melintasi *gap* invalid antar-segmen, sehingga evaluasi tetap merepresentasikan struktur temporal yang benar. Seluruh label dan skor akhirnya digabungkan menjadi satu rangkaian yang siap dipakai pada perhitungan VUS-ROC dan VUS-PR.

Input : DataFrame evaluasi, daftar kolom skor endpoint untuk tiap varian, label titik, dan mask titik yang dievaluasi
Output: Tabel hasil evaluasi berisi metrik point-level dan VUS untuk setiap varian skor
1. Ulangi proses untuk setiap varian skor (<i>reconstruction-only</i>, <i>classification-only</i>, dan <i>combined</i>). 2. Ambil skor varian pada titik-titik yang dievaluasi. 3. Hitung metrik point-level, yaitu ROC-AUC dan AUC-PR, dari label dan skor titik. 4. Bentuk rangkaian label dan skor VUS beserta guard region untuk varian tersebut. 5. Pastikan rangkaian label dan skor berukuran sama serta memuat lebih dari satu kelas. 6. Hitung metrik VUS, yaitu VUS-ROC dan VUS-PR, dari rangkaian tersebut. 7. Catat seluruh metrik dan informasi evaluasi untuk varian tersebut. 8. Gabungkan seluruh hasil varian menjadi satu tabel evaluasi. 9. Kembalikan tabel hasil evaluasi. 10. Selesai.

Kode Semu 3.34. Evaluasi Akhir

Kode Semu 3.34 menunjukkan proses evaluasi akhir yang dilakukan terhadap tiga varian skor, yaitu *reconstruction-only*, *classification-only*, dan *combined*, sehingga kontribusi masing-masing komponen *scoring* dapat dibandingkan. Untuk setiap varian, dihitung dua kelompok metrik. Metrik *point-level* berupa ROC-AUC dan AUC-PR dihitung hanya pada titik-titik yang memenuhi kriteria evaluasi, yaitu titik yang memiliki label dan skor valid. Selain itu, dihitung metrik VUS-ROC dan VUS-PR dengan terlebih dahulu membentuk rangkaian

label dan skor yang dilengkapi *guard region* antar-segmen, lalu dipastikan bahwa rangkaian tersebut berukuran konsisten dan memuat setidaknya dua kelas sebelum metrik dihitung. Seluruh metrik untuk tiap varian, beserta informasi pendukung seperti jumlah titik evaluasi, jumlah titik anomali, cakupan evaluasi, dan jumlah segmen, kemudian dikumpulkan menjadi satu tabel. Tabel inilah yang menjadi dasar analisis untuk menentukan skema *scoring* yang paling efektif pada setiap skenario pengujian.

BAB 4

HASIL DAN PEMBAHASAN

4.1 Hasil Skenario Pengujian

4.1.1 Hasil Hyperparameter Tuning pada Skenario Acuan

Sebelum seluruh skenario pengujian dievaluasi, dilakukan *hyperparameter tuning* pada skenario acuan. Skenario acuan yang digunakan adalah skenario dengan ukuran *window* $L = 48$ dan jumlah kelas $K = 12$. Ukuran *window* $L = 48$ dipilih karena berada di tengah variasi ukuran *window* yang diuji, sedangkan $K = 12$ merepresentasikan konfigurasi penuh REDLAMP yang terdiri atas satu kelas normal dan sebelas kelas pseudo-anomali. Dengan demikian, skenario ini digunakan sebagai dasar untuk mencari konfigurasi model yang kemudian diterapkan pada seluruh skenario pengujian.

Hyperparameter tuning dilakukan menggunakan *grid search* terhadap kombinasi nilai *batch size*, *latent dimension*, *convolution channels*, dan *learning rate*. Sementara itu, parameter *dropout*, *gamma*, dan *weight decay* dipertahankan tetap sesuai rancangan eksperimen. Setiap konfigurasi dievaluasi berdasarkan nilai *validation loss*. Konfigurasi dengan nilai *validation loss* terendah dipilih sebagai konfigurasi terbaik untuk proses *training final* pada seluruh skenario.

Hasil *hyperparameter tuning* menunjukkan bahwa konfigurasi terbaik menggunakan *batch size* sebesar 128, *latent dimension* sebesar 64, *convolution channels* [128, 128, 256, 256], *learning rate* sebesar 0,0005, *dropout* sebesar 0,2, *gamma* sebesar 0,1, dan *weight decay* sebesar 0,00001.

Tabel 4.1. Hasil Hyperparameter Tuning Terbaik

Hyperparameter	Nilai
Batch size	128
Latent dimension	64
Convolution channels	[128, 128, 256, 256]
Learning rate	0,0005
Dropout	0,2
Gamma	0,1
Weight decay	0,00001

4.1.2 Hasil Skenario Pengujian Menggunakan Parameter Acuan

Setelah konfigurasi terbaik diperoleh, dilakukan *training final* dan evaluasi pada seluruh skenario pengujian. Pada skenario $K = 2$, kelas yang digunakan terdiri atas kelas normal dan satu kelas pseudo-anomali, yaitu *spike*. Pada skenario $K = 6$, kelas yang digunakan terdiri atas kelas normal dan lima kelas pseudo-anomali, yaitu *spike*, *noise*, *cutoff*, *scale*, dan *wander*. Pada skenario $K = 12$, digunakan konfigurasi penuh REDLAMP yang terdiri atas kelas normal dan sebelas kelas pseudo-anomali. Hasil evaluasi seluruh skenario menggunakan *Total Anomaly Score* ditunjukkan pada Tabel 4.2.

Tabel 4.2. Hasil Seluruh Skenario

L	K	Pseudo-Anomali	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
24	12	full REDLAMP	0,8042	0,1543	0,8017	0,1322
	6	spike, noise, cutoff, scale, wander	0,8413	0,1800	0,8267	0,1821
	2	spike	0,8472	0,1579	0,8425	0,1360
48	12	full REDLAMP	0,7805	0,1741	0,8068	0,2035
	6	spike, noise, cutoff, scale, wander	0,8489	0,1751	0,8567	0,1825
	2	spike	0,8178	0,1396	0,8325	0,1187
96	12	full REDLAMP	0,7704	0,1781	0,7991	0,1757
	6	spike, noise, cutoff, scale, wander	0,7953	0,1591	0,8223	0,1445
	2	spike	0,8150	0,1578	0,8367	0,1560

Berdasarkan Tabel 4.2, nilai VUS-PR tertinggi diperoleh oleh skenario $L = 24$ dan jumlah kelas $K = 6$. Skenario ini memperoleh nilai VUS-PR sebesar 0,1800. Nilai tersebut menjadi nilai tertinggi dibandingkan seluruh skenario lain pada metrik VUS-PR.

Sementara itu, nilai VUS-ROC tertinggi diperoleh oleh skenario dengan $L = 48$ dan jumlah kelas $K = 6$. Skenario ini memperoleh nilai VUS-ROC sebesar 0,8489. Skenario yang sama juga memperoleh nilai point ROC-AUC tertinggi, yaitu sebesar 0,8567. Adapun nilai point AUC-PR tertinggi diperoleh oleh skenario dengan $L = 48$ dan jumlah kelas $K = 12$ dengan nilai sebesar 0,2035.

Selain nilai metrik, setiap skenario juga memiliki jumlah window dan cakupan evaluasi yang berbeda. Perbedaan ini dipengaruhi oleh ukuran window yang digunakan. Semakin besar ukuran window, semakin panjang subsekuens yang harus valid agar dapat digunakan dalam evaluasi. Ringkasan jumlah window dan cakupan evaluasi ditunjukkan pada Tabel 4.3.

Tabel 4.3. Jumlah Window dan Cakupan Evaluasi

L	Train Windows	Val Windows	Test Windows	Test Timeline Points	Point Eval	Point Anomaly	Evaluation Coverage
24	28.607	9.546	9.601	10.272	9.601	368	0,9347
48	27.171	9.058	9.210	10.272	9.210	361	0,8966
96	24.548	8.329	8.518	10.272	8.518	361	0,8292

Berdasarkan Tabel 4.3, ukuran window $L = 24$ menghasilkan jumlah titik evaluasi terbesar, yaitu 9.601 titik dari total 10.272 titik pada test set, dengan evaluation coverage sebesar 0,9347. Pada ukuran window $L = 48$, jumlah titik evaluasi berkurang menjadi 9.210 titik dengan coverage sebesar 0,8966. Sementara itu, pada ukuran window $L = 96$, jumlah titik evaluasi menjadi 8.518 titik dengan coverage sebesar 0,8292.

Jumlah titik evaluasi yang berbeda tersebut sesuai dengan aturan pembentukan *window* pada metodologi penelitian. Window yang mengandung nilai NaN tidak digunakan dalam evaluasi. Dengan demikian, semakin panjang ukuran *window*, semakin besar kemungkinan suatu *window* menyentuh nilai NaN dan menjadikannya tidak valid, sehingga jumlah *window* dan titik evaluasi yang dapat digunakan menjadi lebih sedikit.

Selain itu, nilai “vus_max_buffer” pada seluruh skenario adalah 7. Nilai ini digunakan pada proses evaluasi VUS untuk membentuk *buffer* pada rentang anomali. Jumlah segmen evaluasi berbeda berdasarkan ukuran window. Pada $L = 24$, jumlah segmen evaluasi adalah 17 segmen. Pada $L = 48$, jumlah segmen evaluasi adalah 16 segmen. Pada $L = 96$, jumlah segmen evaluasi adalah 14 segmen.

Tabel 4.4. Jumlah Segmen Evaluasi pada VUS

L (Panjang Window)	N Eval Segments	VUS Max Buffer
24	17	7
48	16	7
96	14	7

4.2 Pembahasan

4.2.1 Performa Komponen Scoring

Hasil metrik yang didapati pada Tabel 4.2 diukur menggunakan *Total Anomaly Score*. *Total Anomaly Score* pada REDLAMP merupakan gabungan dari skor rekonstruksi (S_{MSE}) dan skor klasifikasi (S_{CE}) dengan bobot yang sama. Untuk memahami kontribusi masing-masing komponen pada implementasi penelitian ini, dilakukan evaluasi terpisah terhadap tiga varian

skor, yaitu *reconstruction_only*, *classification_only*, dan *combined*. Hasil evaluasi ketiga varian skor pada seluruh skenario pengujian ditunjukkan pada Tabel 4.5, 4.6, dan 4.7.

Tabel 4.5. Kontribusi Komponen Scoring pada $L = 24$

K	Varian	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
12	Reconstruction only	0,8996	0,2636	0,8947	0,2218
	Classification only	0,6983	0,0989	0,6899	0,0726
	Combined	0,8042	0,1543	0,8017	0,1322
6	Reconstruction only	0,9082	0,2890	0,9082	0,2573
	Classification only	0,7928	0,1374	0,7541	0,0975
	Combined	0,8413	0,1800	0,8267	0,1821
2	Reconstruction only	0,9150	0,2718	0,9113	0,2237
	Classification only	0,7305	0,1024	0,6840	0,0749
	Combined	0,8472	0,1579	0,8425	0,1360

Tabel 4.6. Kontribusi Komponen Scoring pada $L = 48$

K	Varian	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
12	Reconstruction only	0,9038	0,2749	0,9122	0,2752
	Classification only	0,7098	0,1048	0,7353	0,0828
	Combined	0,7805	0,1741	0,8068	0,2035
6	Reconstruction only	0,9029	0,2708	0,9115	0,2660
	Classification only	0,8254	0,1369	0,8178	0,1008
	Combined	0,8489	0,1751	0,8567	0,1825
2	Reconstruction only	0,8861	0,2369	0,9004	0,2193
	Classification only	0,6764	0,0871	0,6587	0,0633
	Combined	0,8178	0,1396	0,8325	0,1187

Tabel 4.7. Kontribusi Komponen Scoring pada $L = 96$

K	Varian	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
12	Reconstruction only	0,8862	0,2562	0,8987	0,2479
	Classification only	0,6869	0,1082	0,7070	0,0803
	Combined	0,7704	0,1781	0,7991	0,1757
6	Reconstruction only	0,8590	0,2117	0,8800	0,1926
	Classification only	0,7270	0,1162	0,7373	0,0872
	Combined	0,7953	0,1591	0,8223	0,1445
2	Reconstruction only	0,8771	0,2444	0,8979	0,2382
	Classification only	0,6129	0,0942	0,6232	0,0733
	Combined	0,8150	0,1578	0,8367	0,1560

Berdasarkan Tabel 4.5, 4.6, dan 4.7, terlihat pola yang konsisten pada seluruh kombinasi L dan K . Varian *reconstruction only* selalu menghasilkan nilai tertinggi pada keempat metrik yang diuji, diikuti oleh *combined*, kemudian *classification only* pada posisi terendah. Sebagai contoh, pada $L=48$ $K=12$ (Tabel 4.6), *reconstruction only* memperoleh VUS-ROC sebesar 0,9038, sedangkan *combined* hanya 0,7805 dan *classification only* sebesar 0,7098. Pola yang sama juga terlihat pada VUS-PR, di mana *reconstruction only* memperoleh 0,2749 dibandingkan *combined* yang hanya 0,1741.

Pola ini menunjukkan bahwa performa *classification only* secara konsisten jauh lebih rendah dibandingkan *reconstruction only* pada seluruh skenario (VUS-ROC *classification only* berkisar 0,61–0,83, sedangkan *reconstruction only* berkisar 0,86–0,92). Karena skor *combined* dihitung sebagai rata-rata kedua komponen yang telah dinormalisasi (Persamaan 2.27), selisih performa yang besar antara kedua komponen tunggal ini menyebabkan skor *combined* tertarik turun oleh komponen klasifikasi yang lebih lemah.

4.2.2 Analisis Hasil terhadap Ukuran Window

Untuk menganalisis pengaruh ukuran window terhadap performa deteksi, dihitung rata-rata nilai metrik dari combined score pada Tabel 4.5, 4.6, dan 4.7, dikelompokkan berdasarkan ukuran window L . Hasilnya ditunjukkan pada Tabel 4.8.

Tabel 4.8. Rata-rata Combined Score berdasarkan Ukuran Window

L	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
24	0,8309	0,1640	0,8236	0,1501
48	0,8157	0,1629	0,8320	0,1682
96	0,7936	0,1650	0,8194	0,1587

Berdasarkan Tabel 4.8, VUS-ROC menurun secara konsisten seiring bertambahnya ukuran window, dari 0,8309 pada L=24 menjadi 0,8157 pada L=48 dan 0,7936 pada L=96. Pola serupa terlihat pada Point ROC-AUC, meskipun nilainya pada L=48 (0,8320) sedikit lebih tinggi dibandingkan L=24 (0,8236), sebelum menurun pada L=96 (0,8194). Pola berbeda terlihat pada VUS-PR dan Point AUC-PR. VUS-PR relatif stabil pada kisaran 0,163–0,165 di ketiga ukuran window, tanpa tren penurunan yang jelas seiring bertambahnya L. Point AUC-PR justru tertinggi pada L=48 (0,1682), dibandingkan L=24 (0,1501) maupun L=96 (0,1587). Hal ini menunjukkan bahwa window yang lebih panjang menurunkan kemampuan model dalam memberi *ranking* keseluruhan titik data secara global (VUS-ROC), tetapi tidak menurunkan kemampuan model secara proporsional dalam mendeteksi anomali pada titik-titik yang benar-benar anomali (VUS-PR, Point AUC-PR).

4.2.3 Analisis Hasil terhadap Jumlah Kelas Pseudo-Anomali

Untuk menganalisis pengaruh jumlah dan jenis pseudo-anomali terhadap performa model, dihitung rata-rata nilai metrik *combined score* pada Tabel 4.2, dikelompokkan berdasarkan jumlah kelas augmentasi K (termasuk kelas normal). Hasilnya ditunjukkan pada Tabel 4.9.

Tabel 4.9. Rata-rata Combined Score Berdasarkan Pseudo-Anomali

K	Pseudo-Anomali	VUS-ROC	VUS-PR	Point ROC-AUC	Point AUC-PR
2	spike	0,8267	0,1518	0,8373	0,1369
6	spike, noise, cutoff, scale, wander	0,8285	0,1714	0,8352	0,1697
12	full REDLAMP	0,7850	0,1688	0,8025	0,1705

Berdasarkan Tabel 4.9, K=6 memperoleh nilai tertinggi pada VUS-ROC (0,8285) dan VUS-PR (0,1714), sementara K=12 memperoleh nilai terendah pada VUS-ROC (0,7850) dan Point ROC-AUC (0,8025) dibandingkan K=2 dan K=6. Pola ini menunjukkan bahwa penambahan jumlah kelas pseudo-anomali dari K=6 ke K=12 tidak meningkatkan performa pada metrik berbasis *ranking* global (ROC-AUC), meskipun jumlah kelas yang dipelajari model lebih banyak. Sementara itu, K=2 memperoleh VUS-ROC dan Point ROC-AUC yang relatif tinggi dan mendekati K=6, meskipun hanya menggunakan satu jenis pseudo-anomali (*spike*). Namun, K=2 tetap tertinggal dari K=6 pada VUS-PR (0,1518 dibandingkan 0,1714) dan Point AUC-PR (0,1369 dibandingkan 0,1697).

4.2.4 Analisis Prediksi Model

Pada subbab ini dilakukan analisis terhadap prediksi model pada skenario terbaik berdasarkan VUS-PR (Tabel 4.2), yaitu konfigurasi L=24 dan K=6. Analisis dilakukan dengan memeriksa dua *window* dari *test set*. *Window-window* yang diperiksa merupakan *window* berlabel normal dengan skor tertinggi dan *window* berlabel anomali dengan skor terendah. Skor yang digunakan adalah *Anomaly Score (Combined Score)*. *Anomaly Score* merupakan gabungan dari normalisasi *Reconstruction Error* dan *Adjusted Anomaly-Class Score* (seperti pada Persamaan 2.27).

```

FAA applied: True
s_ce shape: (9601,)
Removed classes: [4]

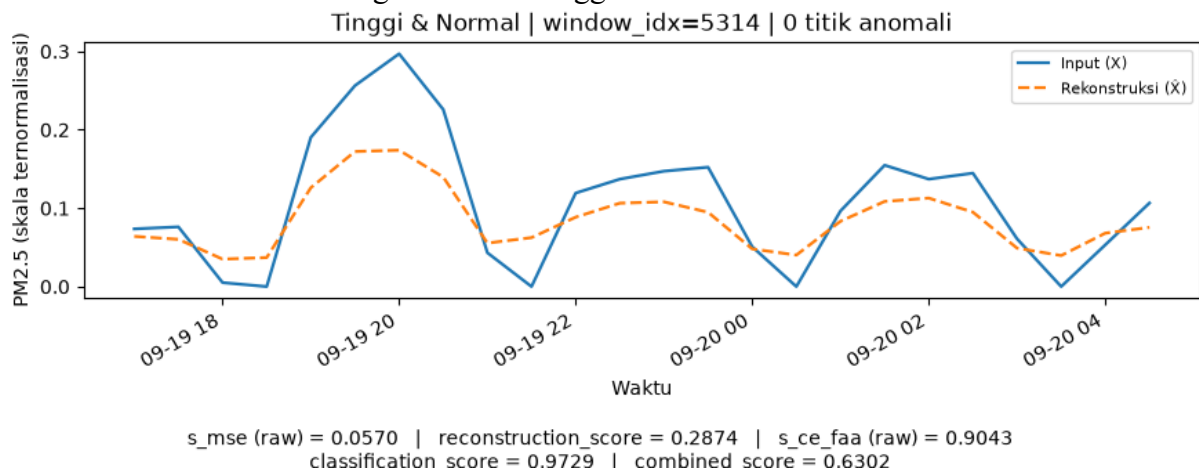
Mean probability per class:
00 normal      : 0.4471
01 spike       : 0.0386
02 noise       : 0.0644
03 cutoff      : 0.0193
04 scale       : 0.3069
05 wander      : 0.1236

```

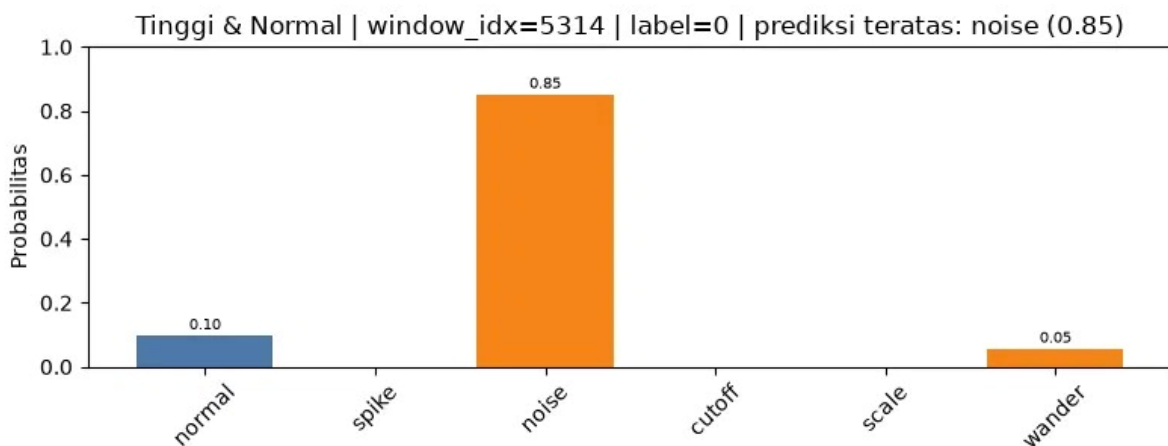
Gambar 4.1. Hasil Frequent Anomaly Adjustment pada $L = 24$ dan $K = 6$

Pada skenario ini, kelas *scale* memiliki rata-rata probabilitas tertinggi di antara kelas pseudo-anomali (0,3069), melampaui *threshold* (pada skenario ini 0,1666), sehingga teridentifikasi sebagai kelas pseudo-normal dan kontribusinya dihilangkan dari adjusted anomaly-class score (Gambar 4.1). Dengan demikian, meskipun suatu window memperoleh probabilitas *scale* yang tinggi, probabilitas tersebut tidak berkontribusi pada skor anomali.

4.2.4.1 Window Normal dengan Skor Tertinggi

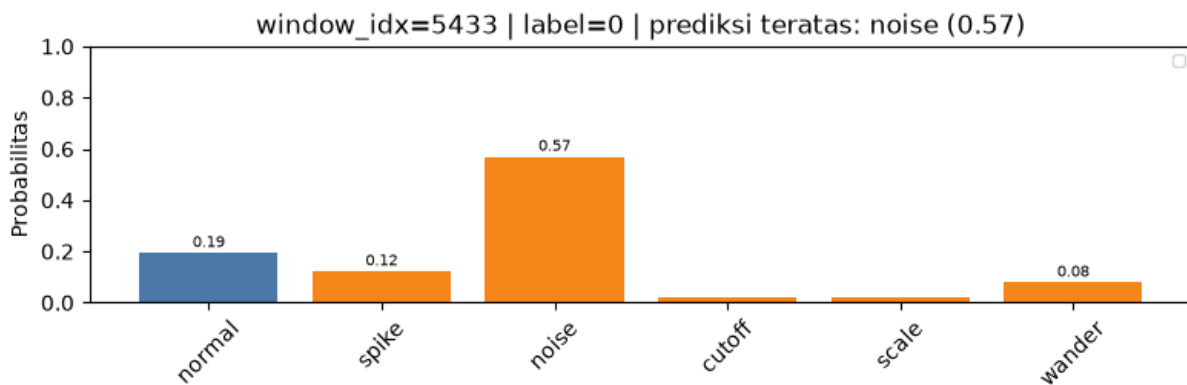


Gambar 4.2. Window Normal dengan Skor Tertinggi



Gambar 4.3. Prediksi Soft Label pada Window Normal dengan Skor Tertinggi

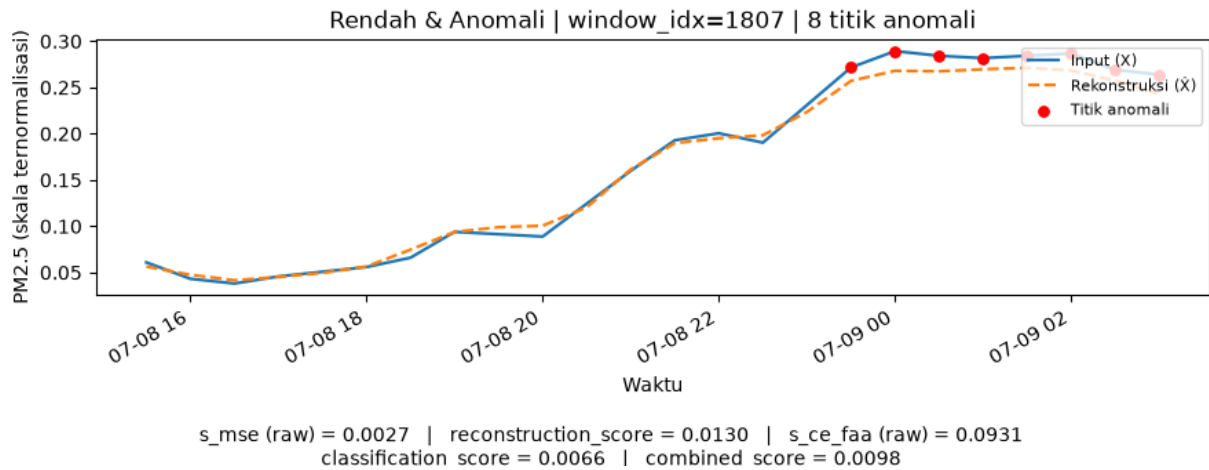
Gambar 4.2 menunjukkan *window* dengan skor tertinggi di antara *window* berlabel normal, yang berada pada *window* indeks ke-5314, dengan *combined score* sebesar 0,6302, *reconstruction score* sebesar 0,2874, dan *classification score* mencapai 0,9729. Tingginya *combined score* pada *window* ini terutama disebabkan oleh komponen klasifikasi, bukan komponen rekonstruksi. Gambar 4.3 menunjukkan probabilitas kelas yang dihasilkan *classifier* untuk *window* yang sama. *Classifier* memberikan probabilitas sangat tinggi pada kelas *noise* (0,85), diikuti normal (0,10) dan wander (0,05). Karena kelas *noise* tidak termasuk kelas yang dihapus pada tahap *Frequent Anomaly Adjustment*, kelas tersebut tetap dihitung ke dalam *adjusted anomaly-class score* (*s_ce_faa* 0,9043). Hal ini menunjukkan bahwa komponen klasifikasi dapat menaikkan skor pada pola normal, sehingga *window* normal memperoleh *combined score* yang tinggi.



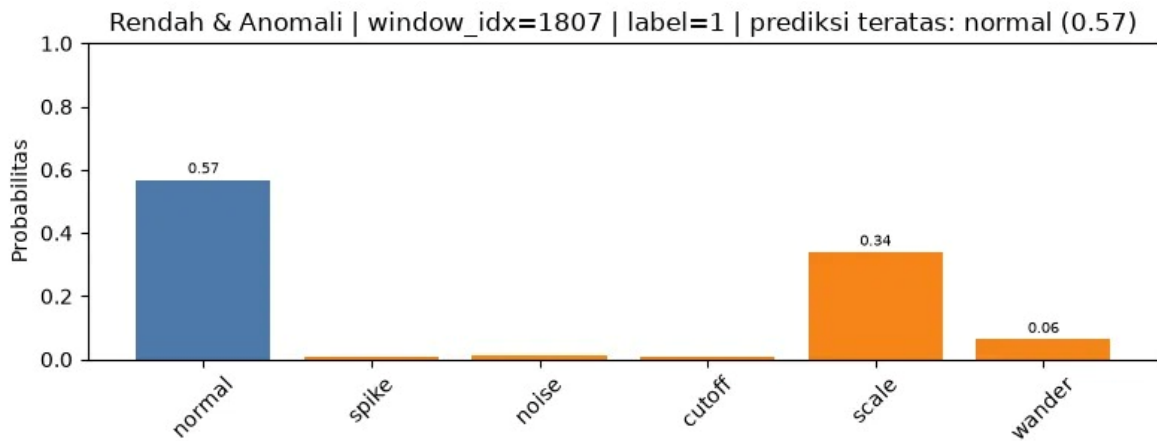
Gambar 4.4. Prediksi Classifier pada Window Normal dengan Probabilitas yang Tersebar

Gambar 4.4 menunjukkan *window* berlabel normal berskor tinggi dengan karakteristik prediksi yang berbeda dari yang ada pada Gambar 4.3. Gambar 4.4 menunjukkan *window* indeks ke-5433 yang juga berlabel normal, tetapi probabilitasnya tersebar pada kelas *noise* (0,57), normal (0,19), *spike* (0,12), *wander* (0,08), sisanya pada *cutoff* dan *scale*. Meskipun prediksi tersebar pada beberapa kelas, skor klasifikasinya tetap tinggi karena *adjusted anomaly-class score* merupakan penjumlahan probabilitas seluruh kelas pseudo-anomali yang dipertahankan (Persamaan 2.26), bukan probabilitas kelas tertinggi. Dengan demikian, skor anomali tinggi pada *window* normal dapat muncul karena dua hal, yaitu satu kelas mendominasi skor seperti pada Gambar 4.3, atau probabilitas yang tersebar terakumulasi melalui penjumlahan seperti pada Gambar 4.4.

4.2.4.2 Window Anomali dengan Skor Terendah



Gambar 4.5. Window Anomali dengan Skor Terendah



Gambar 4.6. Prediksi Soft Label pada Window Anomali dengan Skor Terendah

Gambar 4.5 menunjukkan *window* dengan *combined score* terendah di antara *window* berlabel anomali, yang berada pada *window* indeks ke-1807, dengan *combined score* 0,0098 meskipun mengandung delapan titik anomali. Berbeda dengan kasus sebelumnya, kedua komponen skor sama-sama rendah. *Reconstruction score* sebesar 0,0130 dan *classification score* sebesar 0,0066, yang keduanya berada pada posisi sangat rendah terhadap seluruh *window*. Hasil rekonstruksi hampir sepenuhnya menyerupai *input*, termasuk pada titik-titik yang berlabel anomali. Gambar 4.6 menunjukkan probabilitas kelas untuk *window* yang sama. *Classifier* memberikan probabilitas tertinggi pada kelas normal (0,57), diikuti *scale* (0,34) dan *wander* (0,06). Meskipun kelas *scale* memperoleh probabilitas yang cukup besar dan berpotensi menandai *window* ini sebagai anomali, kelas *scale* teridentifikasi sebagai kelas pseudo-normal pada tahap *Frequent Anomaly Adjustment* karena sering memperoleh probabilitas tinggi di seluruh *test set*, sehingga kontribusinya dihilangkan dari *adjusted anomaly-class score* (s_ce_faa 0,0931). Dengan demikian, anomali pada *window* ini terlewat oleh kedua komponen.

Halaman ini sengaja dikosongkan.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Deteksi anomali pada data *time-series* konsentrasi PM2.5 dari SPKU Gelora Bung Karno, Kementerian Lingkungan Hidup, yang tidak memiliki label anomali resmi, dilakukan secara *unsupervised* menggunakan label penanda data tidak valid yang sudah tersedia pada sistem sebagai acuan evaluasi, tanpa menggunakannya pada proses pelatihan model. Pendekatan ini memungkinkan model dilatih murni dari pola data, sementara kemampuan deteksinya tetap dapat diukur menggunakan label tersebut pada tahap pengujian.
2. Metode REDLAMP berhasil diterapkan sebagai pendekatan *unsupervised multiclass anomaly detection* pada data konsentrasi PM2.5 univariat. Penerapan ini meliputi pembentukan *window* melalui *sliding window*, augmentasi data menjadi satu kelas normal dan sejumlah kelas pseudo-anomali sesuai skenario pengujian, pelatihan model dengan kombinasi *masked reconstruction loss* dan *cross-entropy loss*, serta penghasilan skor anomali akhir dari kombinasi *reconstruction error* dan *adjusted anomaly-class score* setelah penerapan *Frequent Anomaly Adjustment*. Hasil evaluasi menunjukkan bahwa REDLAMP dapat membedakan titik anomali dan titik normal pada data PM2.5, dengan nilai VUS-ROC tertinggi sebesar 0,8489 dan VUS-PR tertinggi sebesar 0,1800, yang masing-masing dicapai pada skenario L=48 K=6 dan L=24 K=6.
3. Kinerja REDLAMP menunjukkan variasi yang dipengaruhi oleh ukuran *window* dan jenis pseudo-anomali yang digunakan. Pada sisi kontribusi komponen skor, komponen *reconstruction only* secara konsisten menghasilkan performa tertinggi pada keempat metrik evaluasi (VUS-ROC, VUS-PR, Point ROC-AUC, Point AUC-PR) dibandingkan *combined* dan *classification only*, pada seluruh skenario pengujian, yang menunjukkan bahwa pada implementasi penelitian ini, penggabungan kedua komponen skor dengan bobot tetap 0,5 tidak meningkatkan performa deteksi dibandingkan jika model hanya mengandalkan *reconstruction error*. Pada sisi ukuran *window*, *window* yang lebih panjang (L=96) menurunkan kemampuan model dalam memberi ranking keseluruhan titik data secara global, tercermin dari penurunan VUS-ROC rata-rata dari 0,8309 (L=24) menjadi 0,7936 (L=96), namun penurunan tersebut tidak terjadi secara proporsional pada metrik berbasis presisi (VUS-PR, Point AUC-PR), yang relatif stabil atau bahkan tertinggi pada L=48. Pada sisi jenis pseudo-anomali, konfigurasi dengan enam kelas hasil augmentasi (K=6: normal, spike, noise, cutoff, scale, wander) secara konsisten memperoleh performa tertinggi atau mendekati tertinggi pada keempat metrik dibandingkan konfigurasi dua kelas (K=2) maupun konfigurasi penuh dua belas kelas (K=12), sementara penambahan jumlah kelas pseudo-anomali dari K=6 menjadi K=12 tidak meningkatkan performa, bahkan menurunkannya pada metrik berbasis ranking global (VUS-ROC turun dari 0,8285 menjadi 0,7850).

Secara keseluruhan, penelitian ini menunjukkan bahwa REDLAMP dapat diterapkan untuk mendeteksi anomali pada data PM2.5 univariat secara *unsupervised*, tetapi performa

optimalnya bergantung pada konfigurasi ukuran *window* dan jumlah kelas pseudo-anomali yang digunakan, dengan konfigurasi *window* menengah ($L=48$) dan jumlah kelas menengah ($K=6$) menunjukkan hasil yang unggul pada penelitian ini.

5.2 Saran

Berdasarkan hasil dan keterbatasan yang ditemukan selama penelitian, berikut beberapa saran untuk pengembangan penelitian selanjutnya:

1. Penelitian selanjutnya dapat mengkaji pembobotan adaptif antara komponen *reconstruction error* dan *adjusted anomaly-class score* pada Total Anomaly Score, mengingat bobot tetap 0,5 pada penelitian ini belum optimal untuk karakteristik data PM2.5 yang digunakan.
2. Penelitian selanjutnya dapat menguji kemampuan deteksi anomali pada data PM2.5 jika menggunakan kombinasi augmentasi pseudo-class yang berbeda.
3. Penelitian selanjutnya dapat membandingkan kinerja REDLAMP dengan metode deteksi anomali *time-series* lain pada data konsentrasi PM2.5, mengingat penelitian ini hanya menerapkan satu metode tanpa pembandingan.
4. Penelitian selanjutnya dapat mengkaji penerapan REDLAMP pada data kualitas udara multivariat (misalnya dengan menambahkan parameter PM10, CO, atau NO2), untuk mengevaluasi apakah pola hasil yang ditemukan pada data univariat ini juga berlaku pada konteks multivariat.

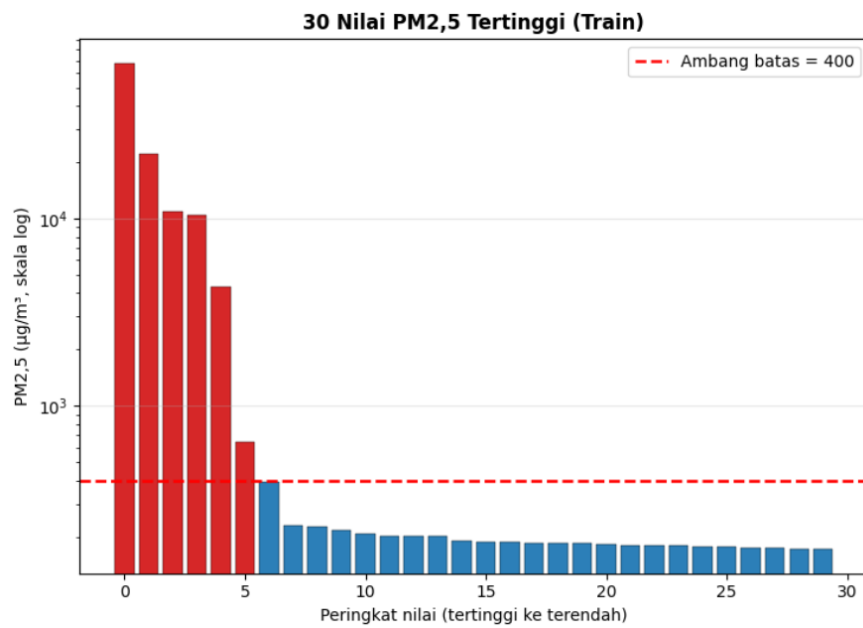
DAFTAR PUSTAKA

- Benson, B., Pan, W. D., Prasad, A., Gary, G. A., & Hu, Q. (2020). Forecasting Solar Cycle 25 Using Deep Neural Networks. *Solar Physics*, 295(5). <https://doi.org/10.1007/s11207-020-01634-y>
- Braei, M., & Wagner, S. (2020). *Anomaly Detection in Univariate Time-series: A Survey on the State-of-the-Art*. <http://arxiv.org/abs/2004.00433>
- Carmona, C. U., Aubet, F.-X., Flunkert, V., & Gasthaus, J. (2021). *Neural Contextual Anomaly Detection for Time Series*. <http://arxiv.org/abs/2107.07702>
- Damayanti, T. V., & Handriyono, R. E. (2022). MONITORING KUALITAS UDARA AMBIEN MELALUI STASIUN PEMANTAU KUALITAS UDARA WONOREJO, KEBONSARI DAN TANDES KOTA SURABAYA. *Environmental Engineering Journal ITATS ENVITATS*, 2(1).
- Fung, C., Qiu, C., Li, A., & Rudolph, M. (2024). *Model Selection of Anomaly Detectors in the Absence of Labeled Validation Data*. <http://arxiv.org/abs/2310.10461>
- Giannoni, F., Mancini, M., & Marinelli, F. (2018). *Anomaly detection models for IoT time series data Data Mining-Project Report*.
- Goswami, M., Challu, C., Callot, L., Minorics, L., & Kan, A. (2023). *Unsupervised Model Selection for Time-series Anomaly Detection*. <http://arxiv.org/abs/2210.01078>
- IQAir. (2024). World's most polluted countries. Diakses pada 22 Desember 2025 dari <https://www.iqair.com/world-most-polluted-countries>
- Lai, K.-H., Zha, D., Xu, J., Zhao, Y., Wang, G., & Hu, X. (2021). *Revisiting Time Series Outlier Detection: Definitions and Benchmarks*. Retrieved from <https://github.com/datamllab/tods/tree/benchmark>.
- Obata, K., Matsubara, Y., & Sakurai, Y. (2025). Robust and Explainable Detector of Time Series Anomaly via Augmenting Multiclass Pseudo-Anomalies. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2, 2198–2209. <https://doi.org/10.1145/3711896.3737110>
- Paparrizos, J., Boniol, P., Palpanas, T., Tsay, R. S., Elmore, A., & Franklin, M. J. (2022). *Volume Under the Surface: A New Accuracy Evaluation Measure for Time-Series Anomaly Detection* (Vol. 15, Number 11). <https://www.timeseries.org/vus>.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3). <https://doi.org/10.1371/journal.pone.0118432>
- Schmidl, S., Wenig, P., & Papenbrock, T. (2022). Anomaly Detection in Time Series: A Comprehensive Evaluation. *Proceedings of the VLDB Endowment*, 15(9), 1779–1797. <https://doi.org/10.14778/3538598.3538602>

- Vital Strategies & Institut Teknologi Bandung*. (2020). Air Pollution in Jakarta: A Source Apportionment Study (Policy Brief). Diakses 22 Desember 2025
https://www.vitalstrategies.org/wp-content/uploads/Air-Pollution-in-Jakarta-A-Source-Apportionment-Study_Policy-Brief_ENG.pdf
- Wangintan, R., & Sofyan, A. (2019). *ANALISIS KUALITAS UDARA AMBIEN DAN PENETUAN LOKASI STASIUN PEMANTAUAN KUALITAS UDARA (SPKU) DKI JAKARTA*.
- Zhang, C., Song, D., Chen, Y., Feng, X., Lumezanu, C., Cheng, W., Ni, J., Zong, B., Chen, H., & Chawla, N. V. (2018). *A Deep Neural Network for Unsupervised Anomaly Detection and Diagnosis in Multivariate Time Series Data*. <http://arxiv.org/abs/1811.08055>

LAMPIRAN

Lampiran 1. Plot Nilai PM2.5 pada Train Set



Lampiran 2. Tabel Baseline PR-Curve

L	Point Eval	Point Anomaly	PR Baseline
24	9.601	368	0,038
48	9.210	361	0,039
96	8.518	361	0,042

Lampiran 3 Tabel Window L = 48

Split	Expected Windows	Actual Windows	Dropped Windows
Train	32.065	27.171	4.894 (15%)
Val	10.129	9.058	1.071 (10%)
Test	10.225	9.210	1.015(10%)

Lampiran 4 Tabel Window L = 24

Split	Expected Windows	Actual Windows	Dropped Windows
Train	32.089	28.607	3.482 (11%)
Val	10.153	9.546	607 (6%)
Test	10.249	9.601	648(6%)

Lampiran 5 Tabel Window L = 96

Split	Expected Windows	Actual Windows	Dropped Windows
Train	32.017	24.548	7.469 (23%)
Val	10.081	8.329	1.752 (17%)
Test	10.177	8.518	1.659 (16%)

Lampiran 6 F1 Threshold and Classification Report

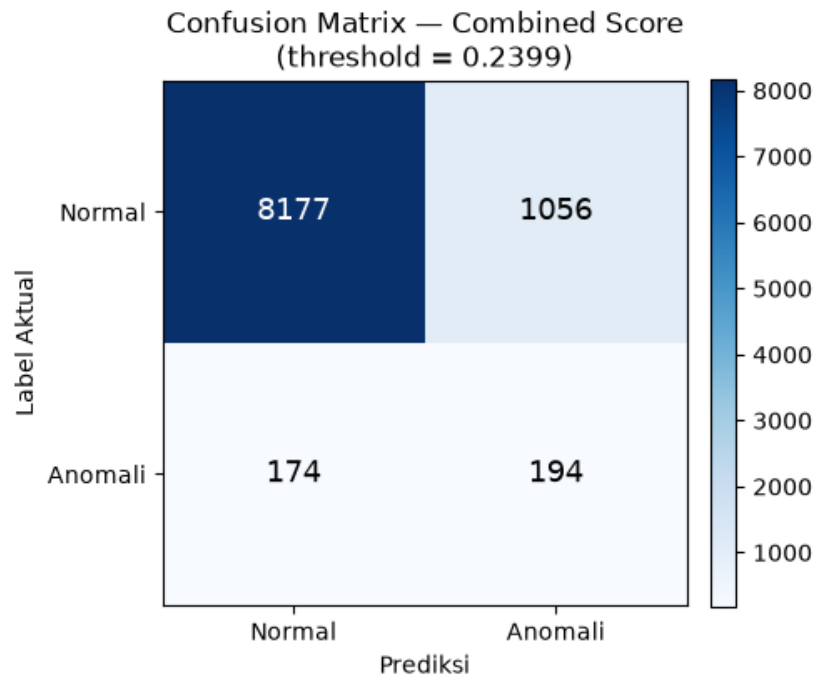
Best-F1 threshold : 0.239914
Precision @ best : 0.1552
Recall @ best : 0.5272
F1 @ best : 0.2398

Confusion Matrix (Best-F1 threshold):
[[8177 1056]
[174 194]]

Classification Report:

	precision	recall	f1-score	support
Normal	0.98	0.89	0.93	9233
Anomali	0.16	0.53	0.24	368
accuracy			0.87	9601
macro avg	0.57	0.71	0.58	9601
weighted avg	0.95	0.87	0.90	9601

Lampiran 7 Confusion Matrix pada L = 24 & K = 6



Halaman ini sengaja dikosongkan.

BIODATA PENULIS



Penulis dilahirkan di Jakarta, 13 Juli 2004, merupakan anak pertama dari 3 bersaudara. Penulis telah menempuh pendidikan formal yaitu di Al-Jannah, SDI Al-Azhar 20 Cibubur, SMPI Al-Azhar 19 Cibubur, dan SMAN 39 Jakarta. Setelah lulus dari SMA tahun 2022, Penulis melanjutkan studi ke jenjang sarjana di Institut Teknologi Sepuluh Nopember, di Departemen Teknik Informatika, Fakultas Teknologi Elektro dan Informatika Cerdas.

Di Departemen Teknik Informatika Penulis sempat aktif di beberapa kegiatan Organisasi seperti HMTC dan Schematics. Selain itu, Penulis juga aktif dalam Klub Basket TC, yang di mana Ia memenangkan Kejuaraan Bola Basket ELITE X PESRAF pada tahun 2022, 2023, dan 2024 secara berturut-turut, sebelum

akhirnya memilih pensiun dari kegiatan tersebut.